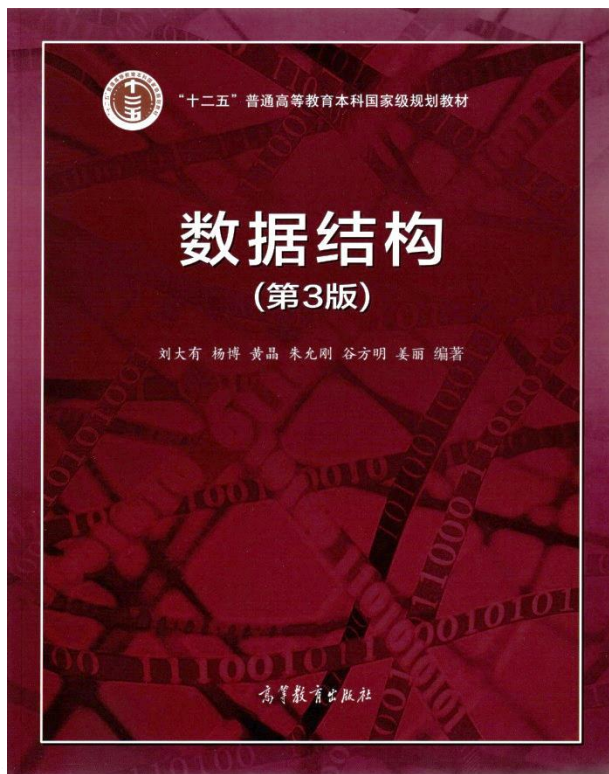
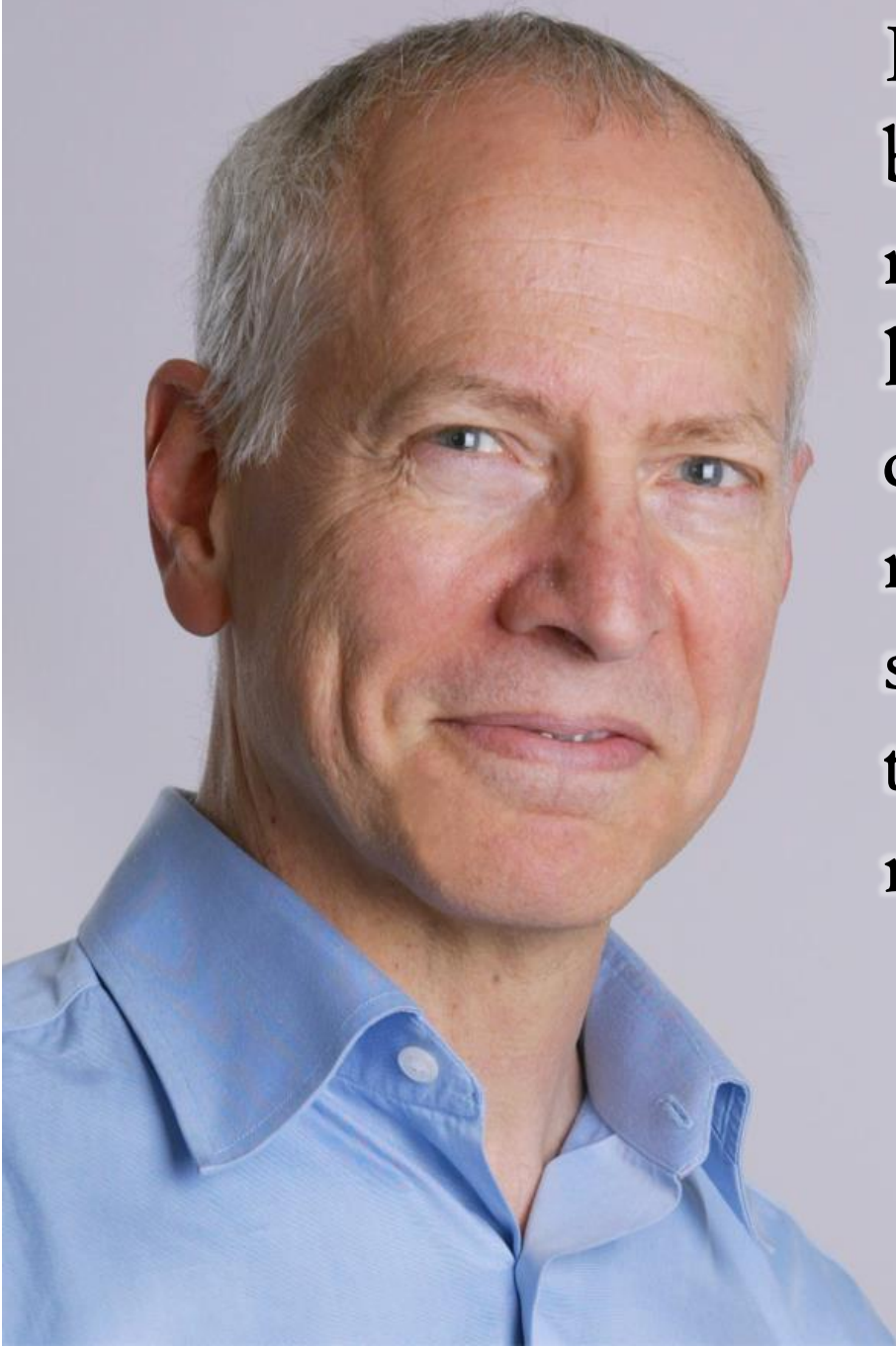




## 二叉查找树

- 二叉查找树
- 高度平衡树
- 红黑树简介
- 伸展树

数据之法  
结构之美  
算法之道

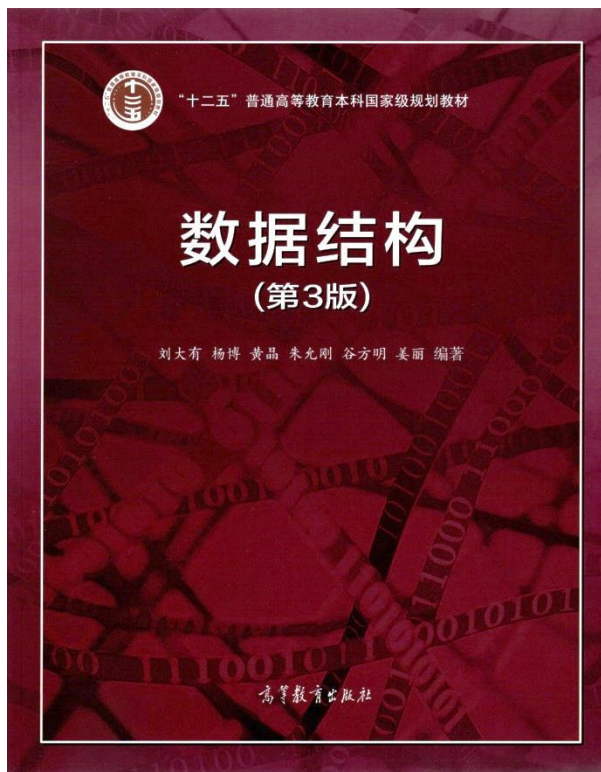


I do research in algorithm design not only because it offers possibilities for affecting the real world of computing, but also because I love the work itself, the rich and surprising connections among problems and solutions it reveals, and the opportunity it provides to share with creative, stimulating, and thoughtful colleagues in the discovery of new ideas.

——Robert Tarjan

图灵奖获得者  
普林斯顿大学教授  
美国科学院院士  
美国工程院院士





# 二叉查找树

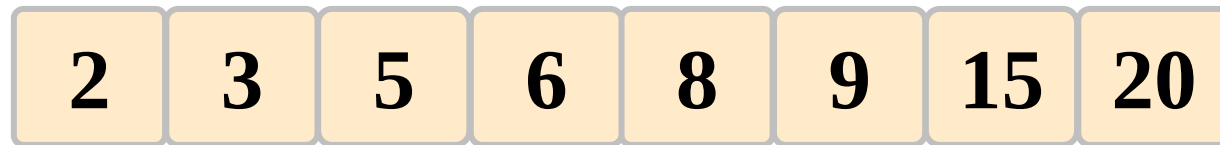
- 二叉查找树
- 高度平衡树
- 红黑树简介
- 伸展树

数据之法  
结构之美  
算法之道

zhuyungang@jlu.edu.cn

## 动机——动态查找

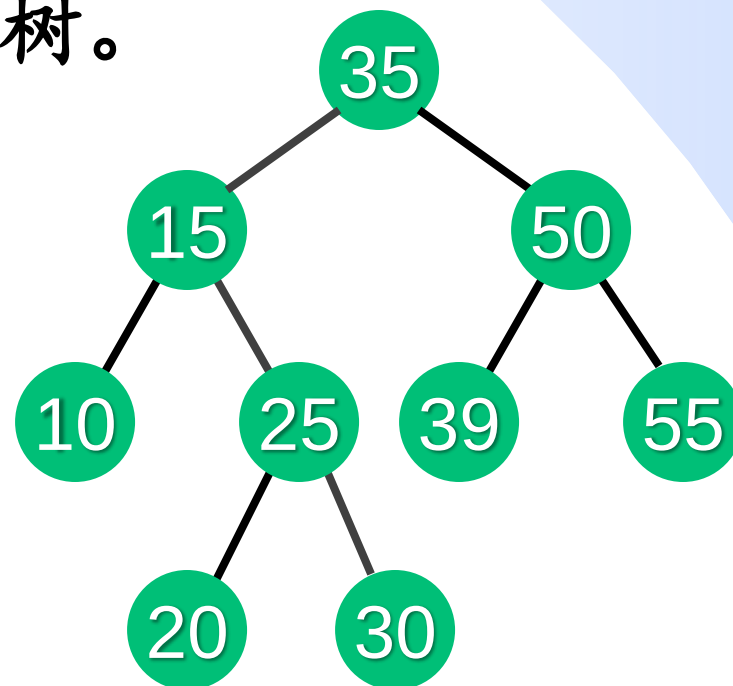
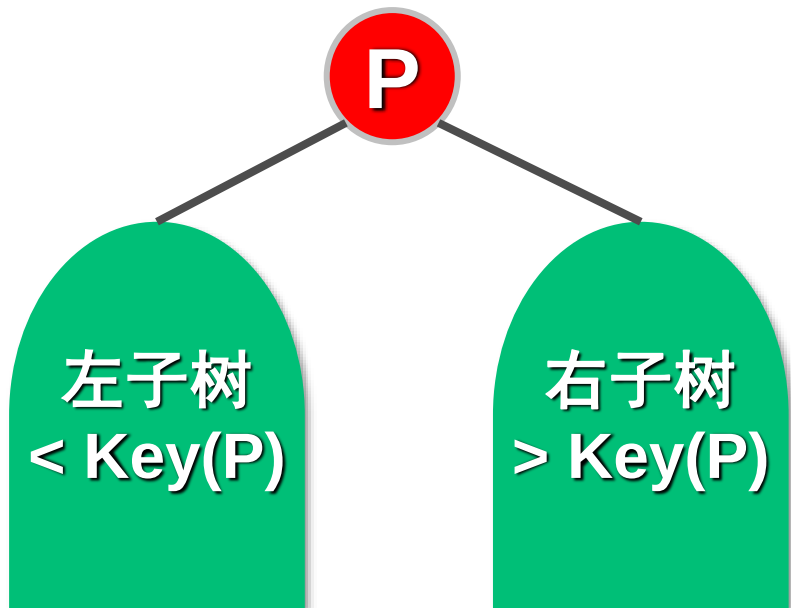
- 对有序数组的二分查找，适用于静态查找场景，若元素动态变化（插入、删除元素），为了维持数组有序，需要 $O(n)$ 时间调整。
- 希望：插入、删除、查找时间不超过 $O(\log n)$ 。



# 二叉查找树 ( *Binary Search Tree, BST* )

**定义：二叉查找树**（亦称二叉搜索树、二叉排序树）是一棵二叉树，其各结点关键词互异，且中根序列按其关键词递增排列。

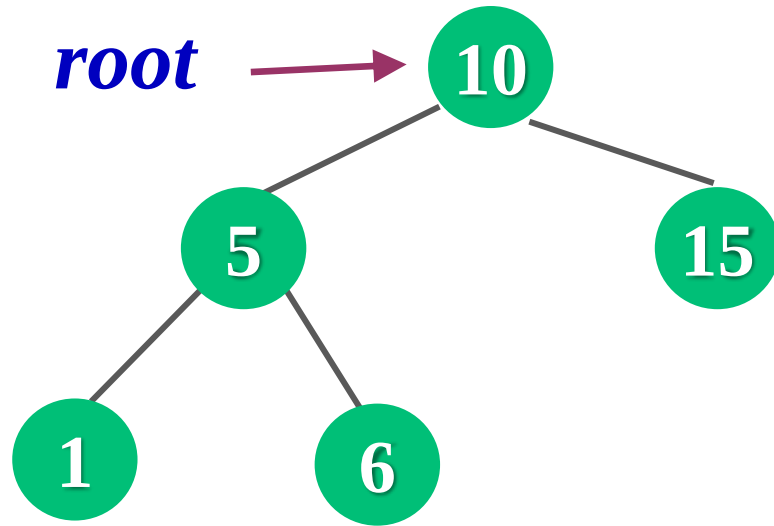
**等价描述：二叉查找树**中任一结点 $P$ ，其左子树中结点的关键词都小于 $P$ 的关键词，右子树中结点的关键词都大于 $P$ 的关键词，且结点 $P$ 的左右子树也都是二叉查找树。



## 二叉查找树 vs 二叉判定树

- 二叉判定树：描述查找过程，数据实际是存在数组里，是扩充二叉树。
- 二叉查找树：数据是真的存在二叉树里了，不是扩充二叉树。

## 二叉查找树结点结构



```
struct BSTnode {  
    int key;  
    BSTnode* left;  
    BSTnode* right;  
    BSTnode(int K){ key=K; left=right=NULL; }  
};
```

# 二叉查找树基本操作

## ➤ 核心操作

- ✓ **查找**：在二叉查找树中查找关键词为 $K$ 的结点。
- ✓ **插入**：将关键词为 $K$ 的结点插入二叉查找树，插入后仍为二叉查找树，若 $K$ 已在树中，则不插入。
- ✓ **删除**：删除关键词为 $K$ 的结点，删除后仍为二叉查找树。

## ➤ 非核心操作

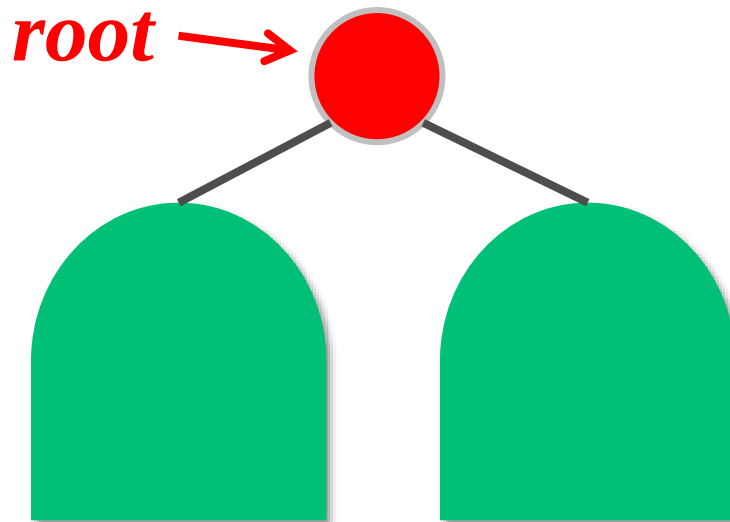
- ✓ **创建**：从空树开始逐个插入新结点。
- ✓ **排序**：中根遍历。
- ✓ **找最小元素/最大元素**：找中根序列第一个/最后一个结点。
- ✓ **找第 $k$ 小的元素**。
- ✓ .....



## 二叉查找树—查找算法（递归版本）

在以 $root$ 为根的二叉查找树中查找关键词等于 $K$ 的结点，并返回指向该结点的指针【大厂面试题[LeetCode700](https://leetcode.com/problems/search-in-a-bst/)】。

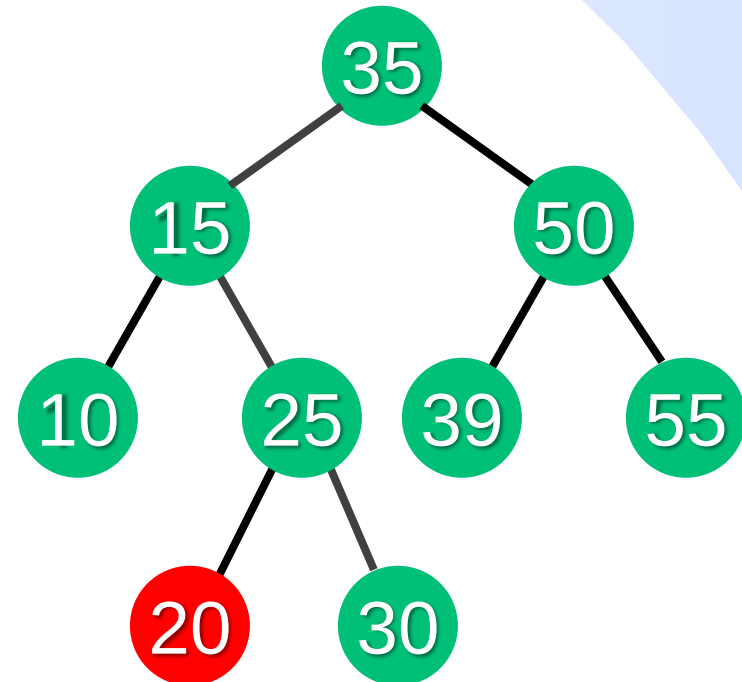
```
BSTnode* Search(BSTnode* root, int K){  
    if (root == NULL || root->key == K) return root;  
    if (K < root->key) return Search(root->left, K);  
    else return Search(root->right, K);  
}
```



## 二叉查找树—查找算法（迭代版本）

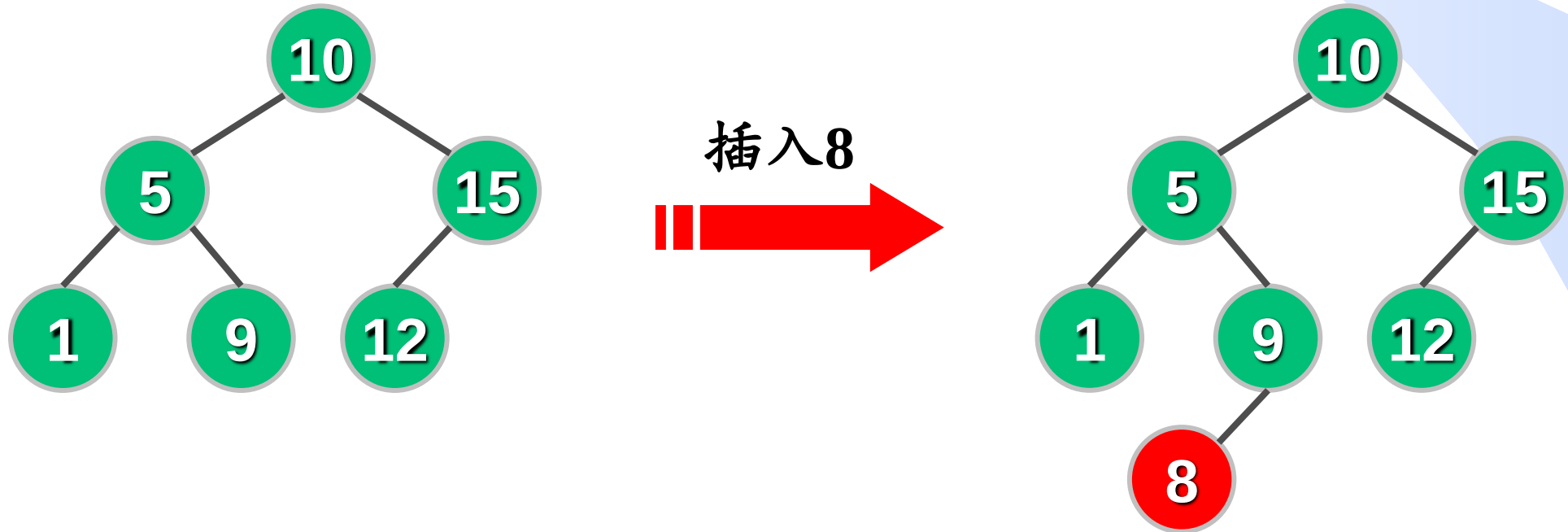
```
BSTnode* Search2(BSTnode* root, int K){ //迭代版本
    BSTnode* p = root;
    while (p != NULL) {
        if (K < p->key) p = p->left;
        else if (K > p->key) p = p->right;
        else return p;
    }
    return NULL;
}
```

时间复杂度  
 $O(h)$   
 $h$ 为树高



## 二叉查找树—插入算法

- 在以 $root$ 为根的二叉查找树中插入关键词值为 $K$ 的结点，并保证插入后仍是一棵二叉查找树，若树中已有关键词等于 $K$ 的结点，则不再重复插入。
- 查找 $K$ ，然后在查找失败处插入 $K$ 。例如：插入8



## 二叉查找树—删除算法

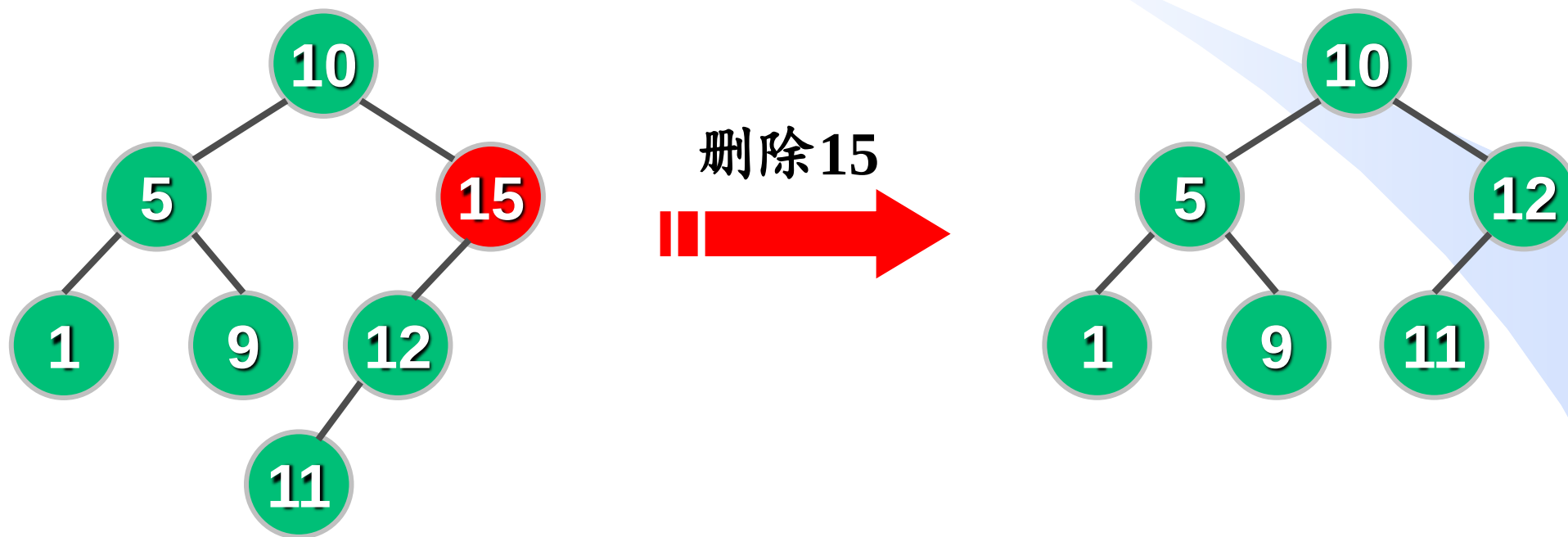
- 在以 $root$ 为根的二叉查找树中删除关键词值为 $K$ 的结点，并保证删除后仍是一棵二叉查找树，若树中无关键词等于 $K$ 的结点，则无操作。
- 在 $root$ 中找到关键词值为 $K$ 的结点，分3种情况删除 $K$ ：  
(1) 若 $K$ 无孩子（即叶结点），则直接删除。





## 二叉查找树—删除算法

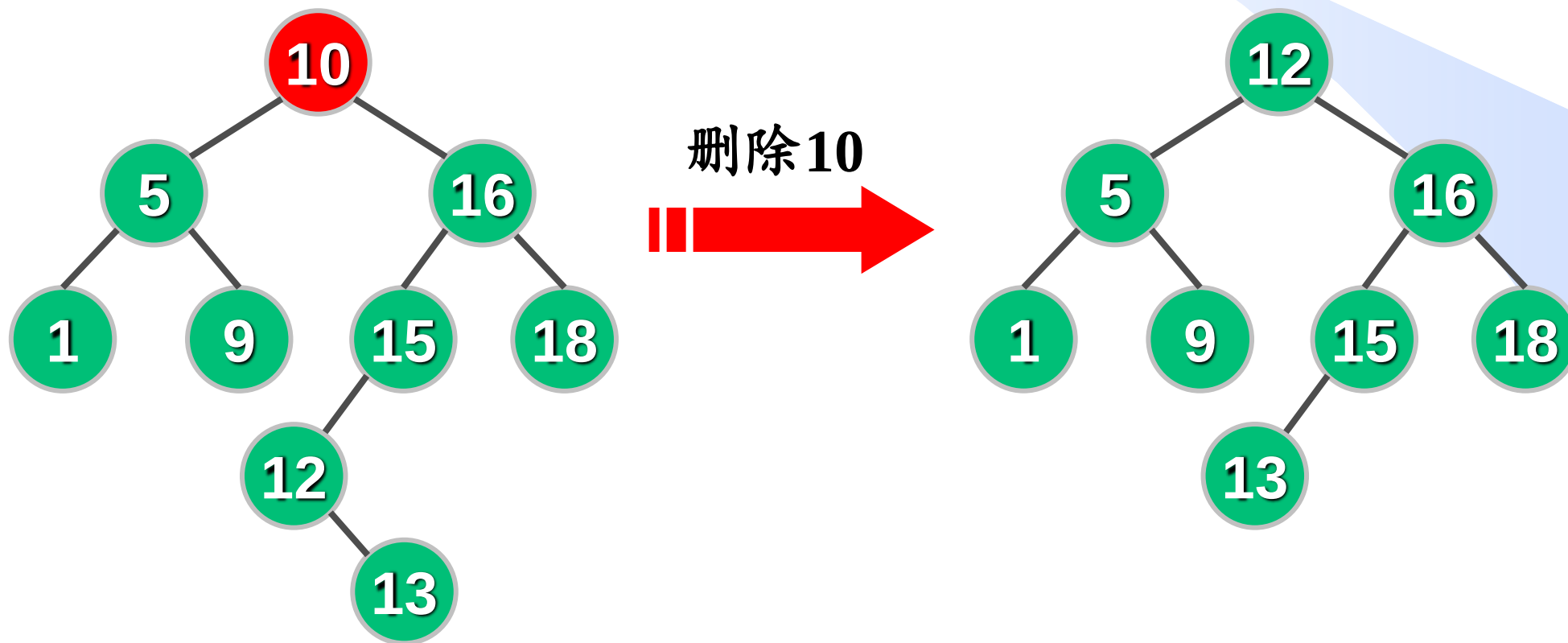
(2)  $K$ 只有1个孩子，则子承父业（让其子结点替换 $K$ ）。



$K$ 的位置由它的子结点取代

## 二叉查找树—删除算法

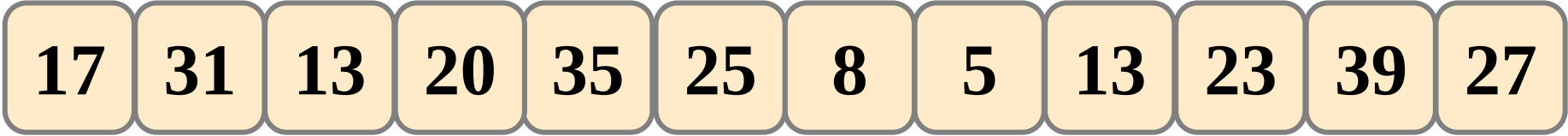
(3)  $K$ 有2个孩子，则在其右子树找关键词最小的结点 $s$ （即右子树中根序列的第一个结点，亦即 $K$ 的中根后继）替换 $K$ ，然后删除原结点 $s$ （ $s$ 只有一个右孩子或无孩子）。

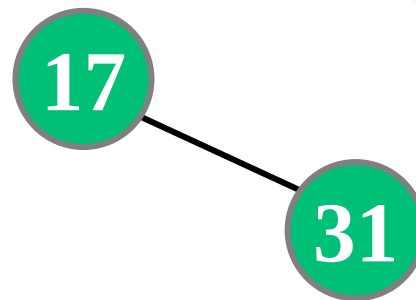


## 练习

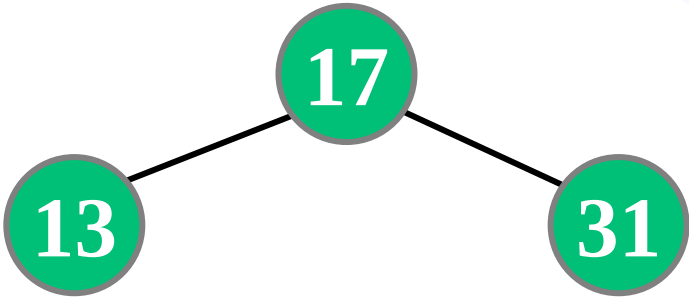
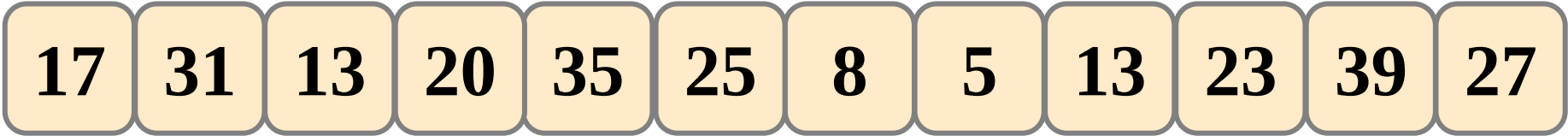
从空二叉查找树开始依次插入元素（17, 31, 13, 20, 35, 25, 8, 4, 13, 24, 40, 27），请画出最后的二叉查找树，并分别给出继续执行下列操作后的二叉查找树：

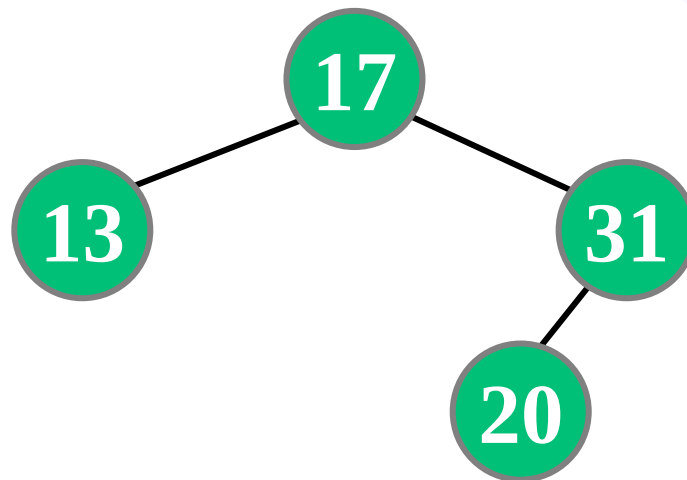
- 1) 插入数据9；
- 2) 删除结点17；
- 3) 再删除结点13.

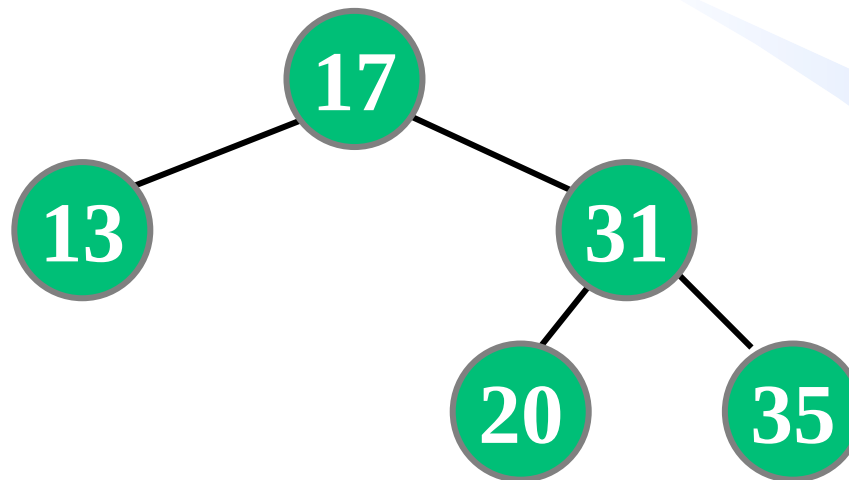


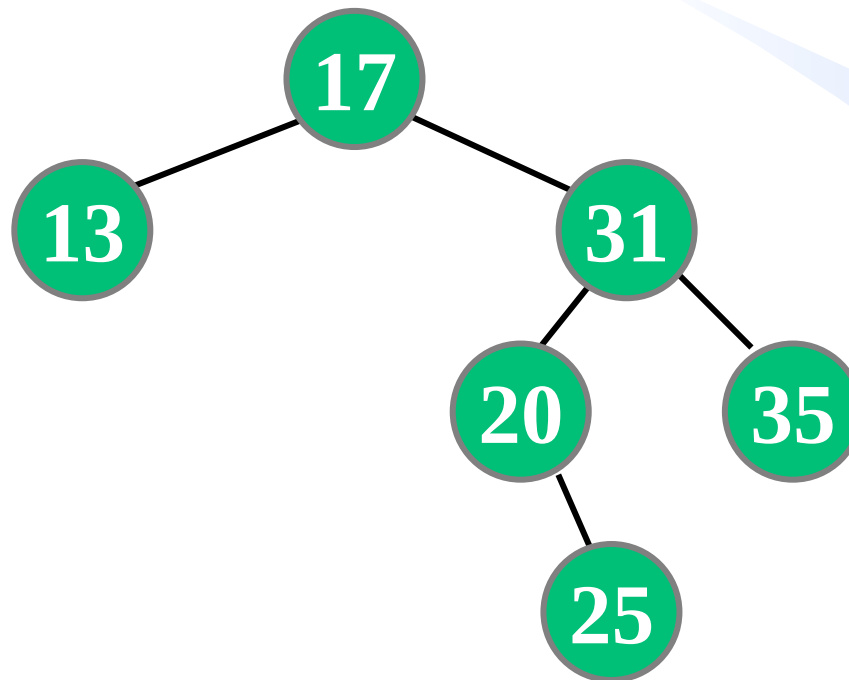


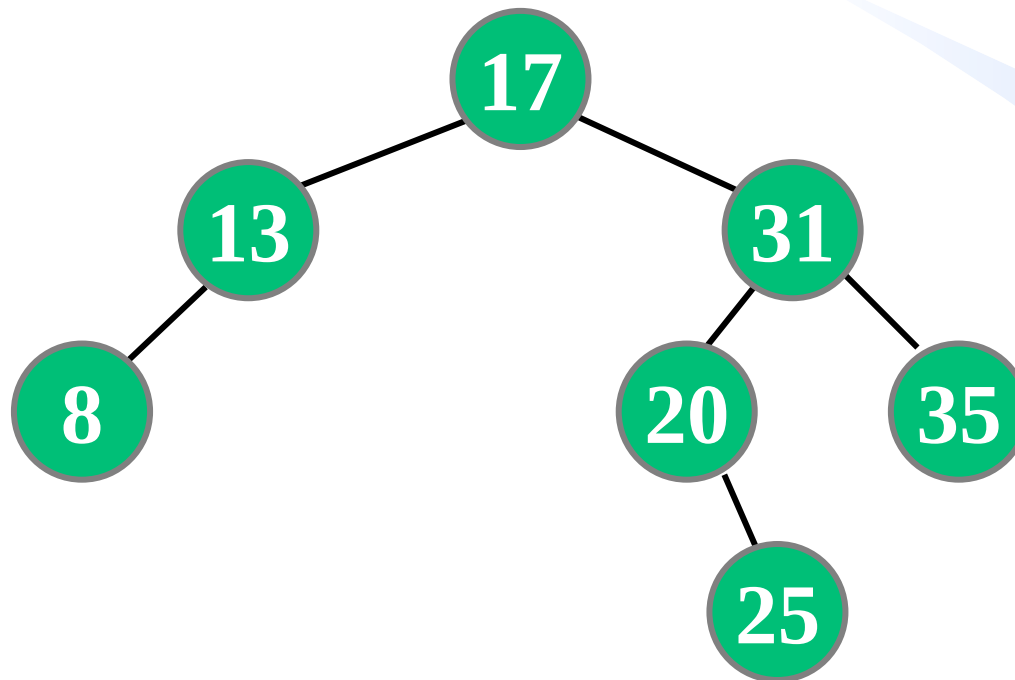


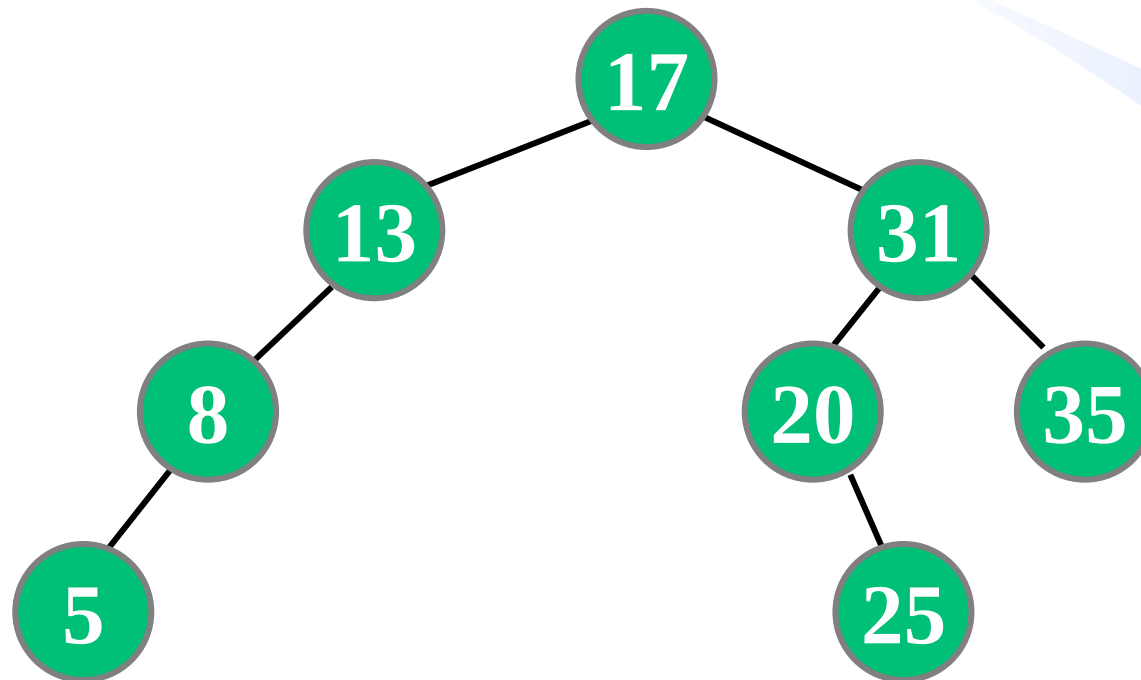


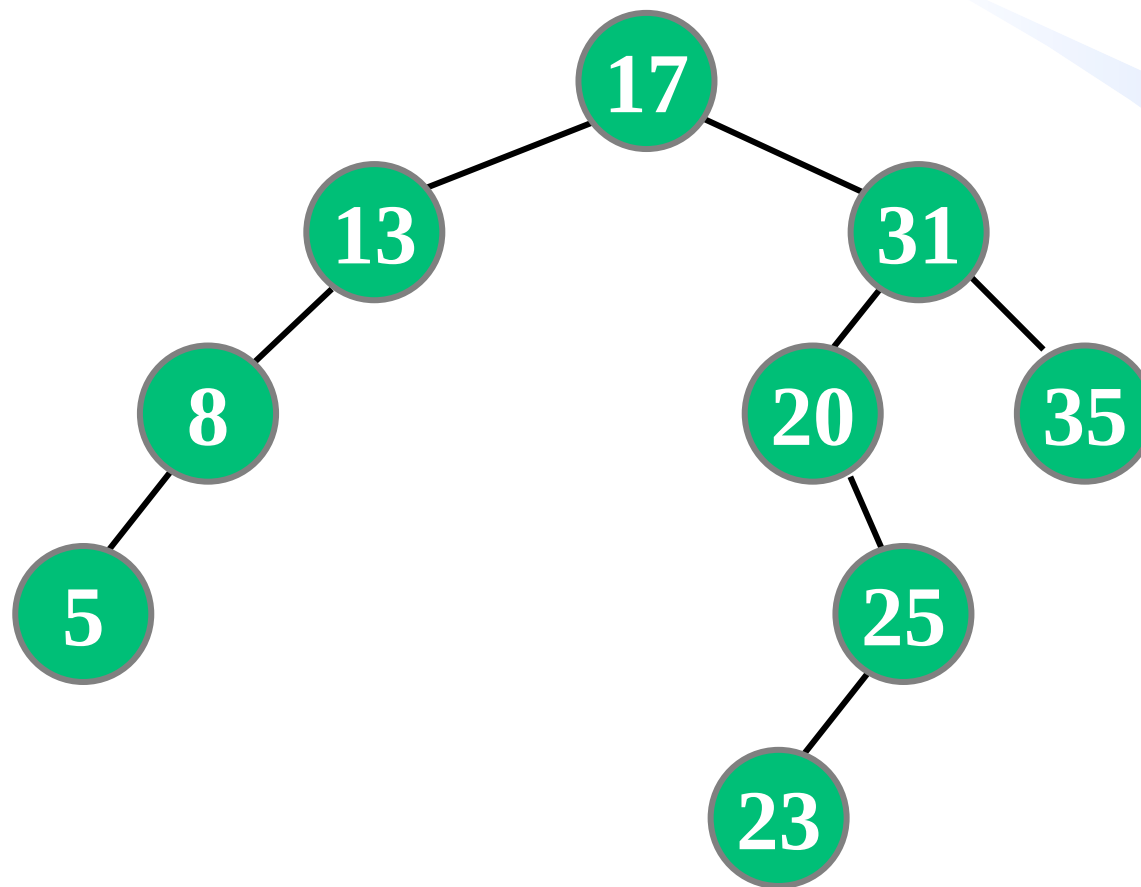




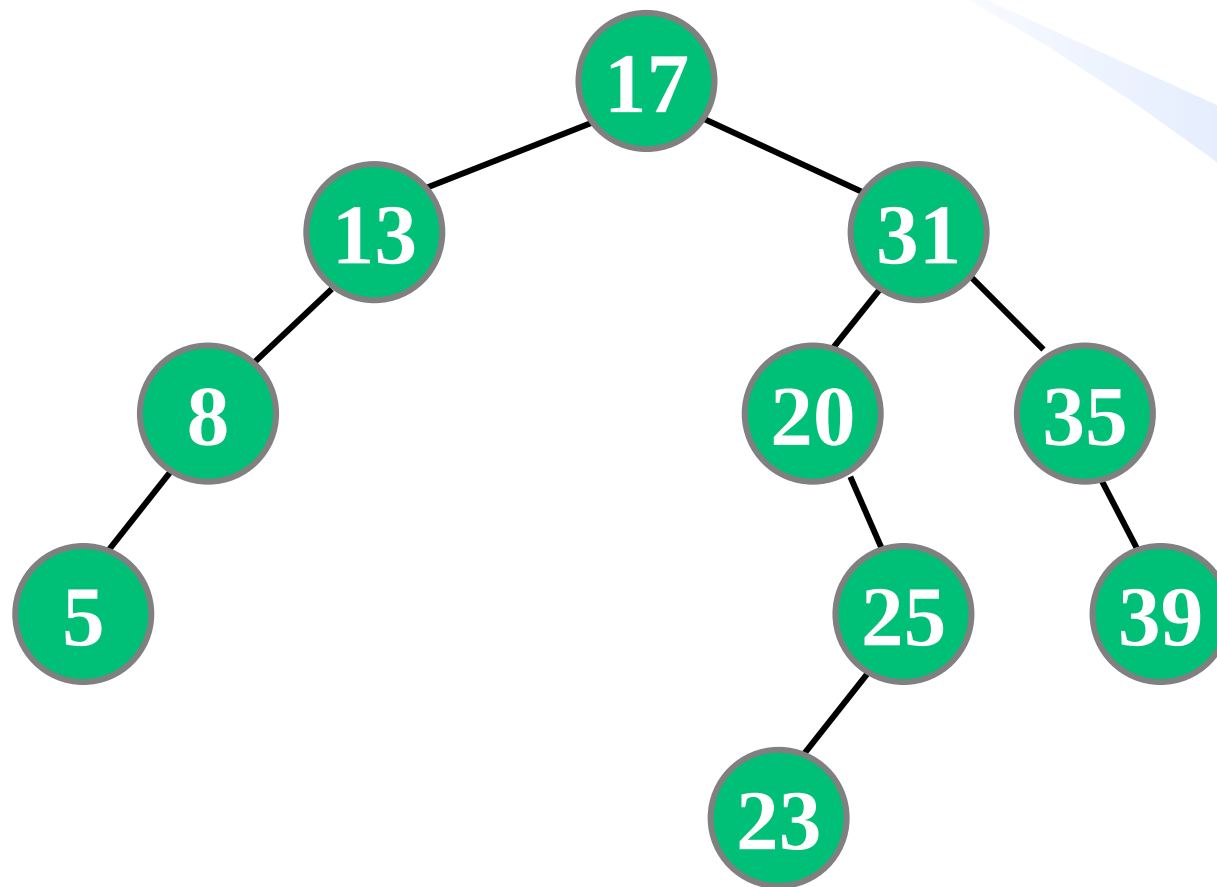


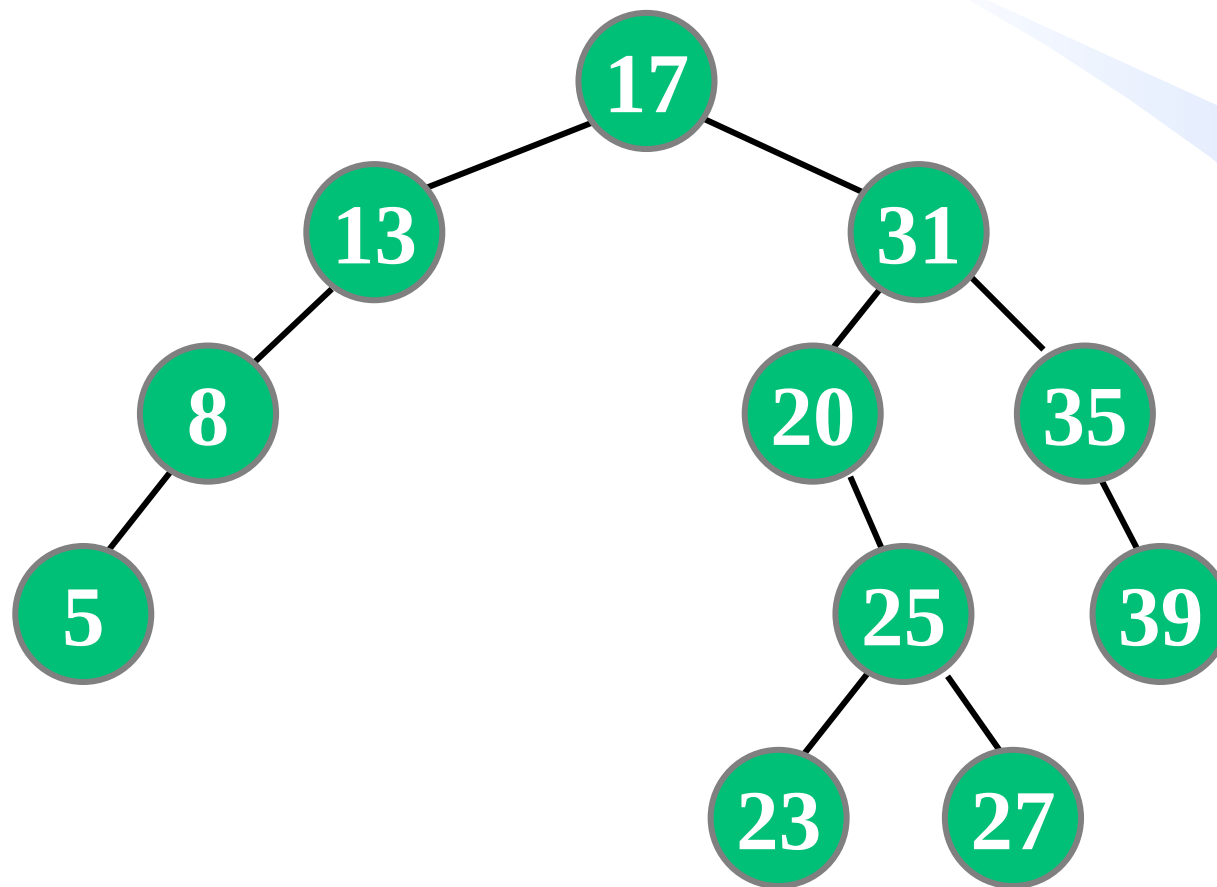






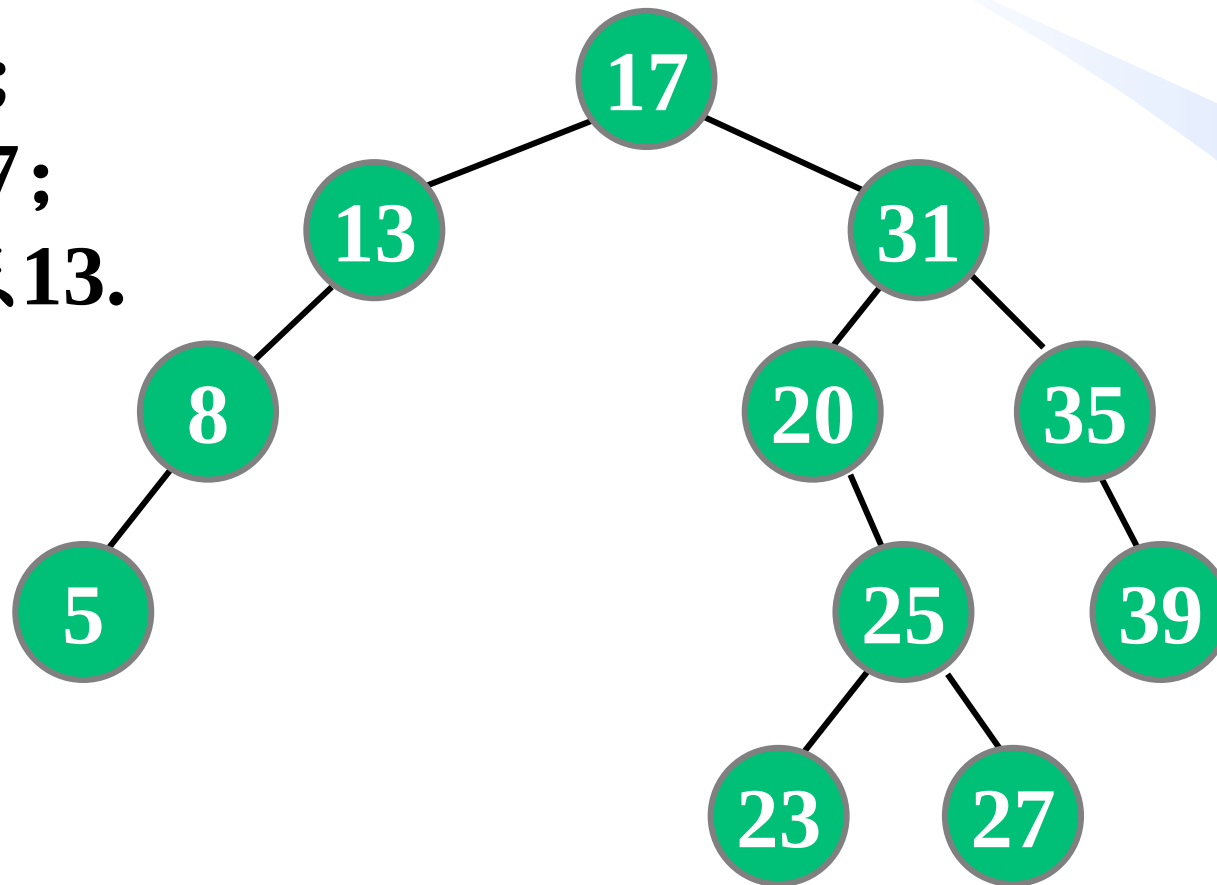






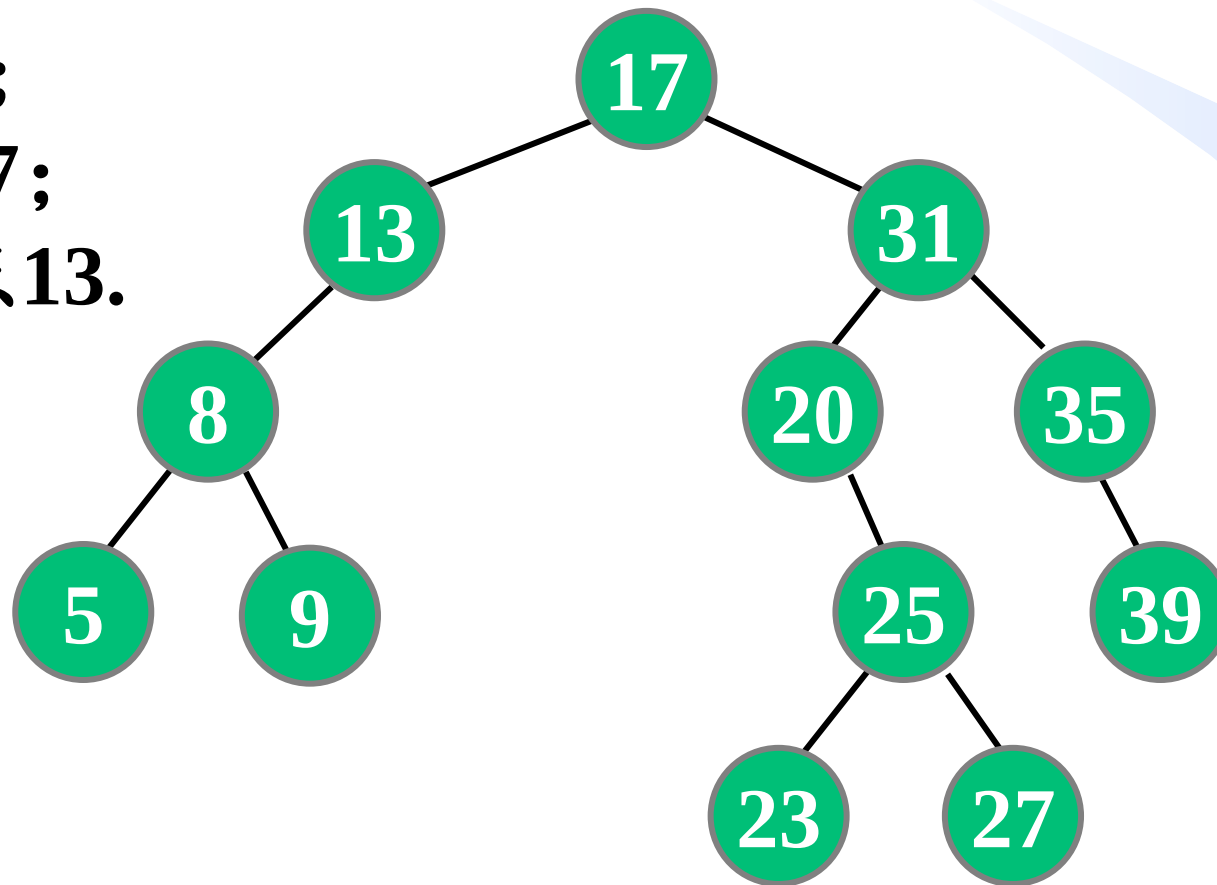


- 1) 插入数据9;
- 2) 删除结点17;
- 3) 再删除结点13.



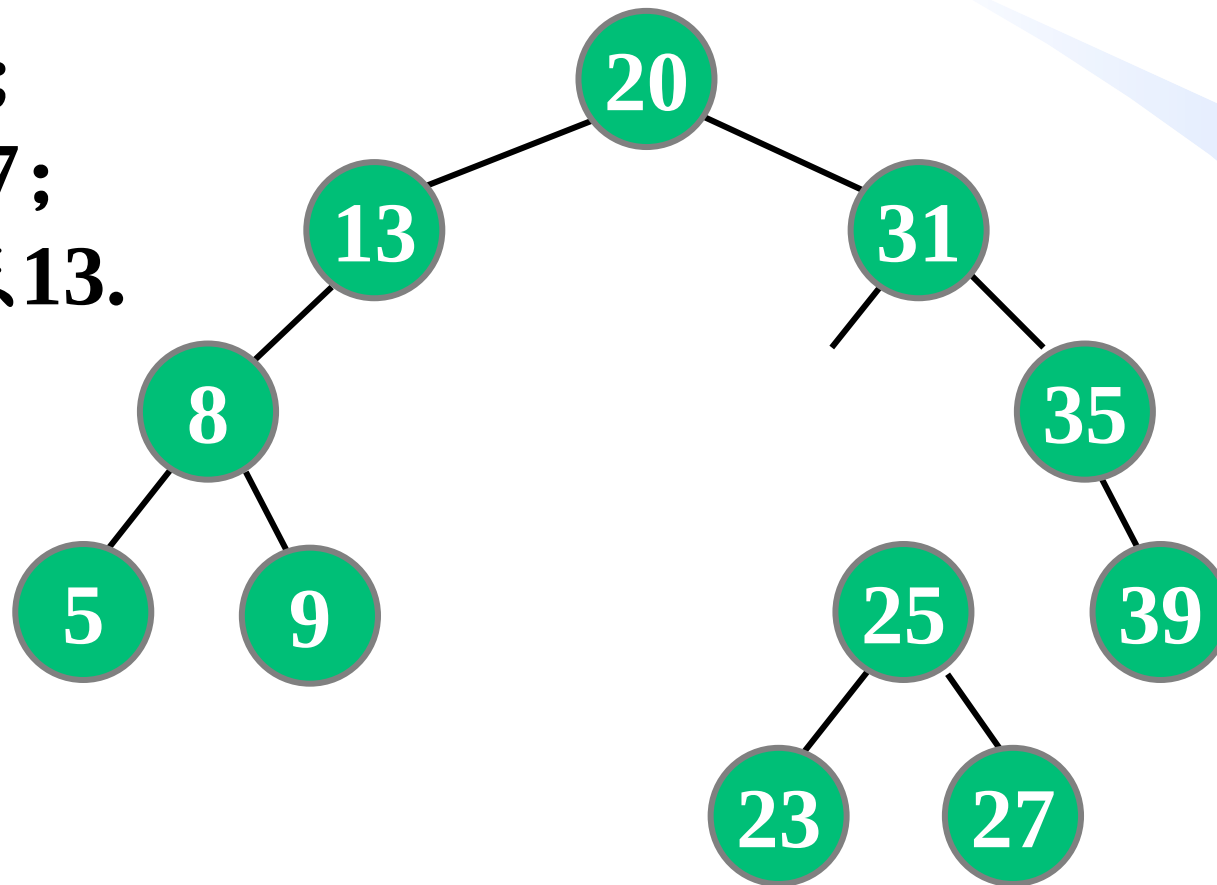


- 1) 插入数据9;
- 2) 删除结点17;
- 3) 再删除结点13.



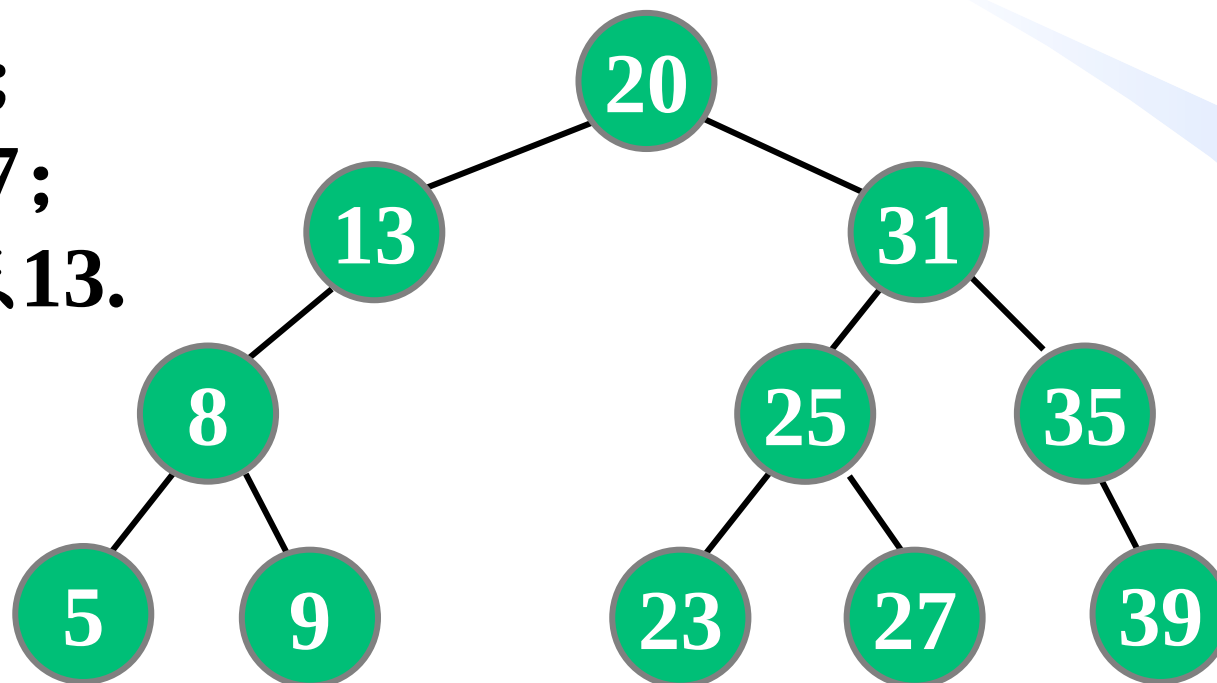


- 1) 插入数据9;
- 2) 删除结点17;
- 3) 再删除结点13.



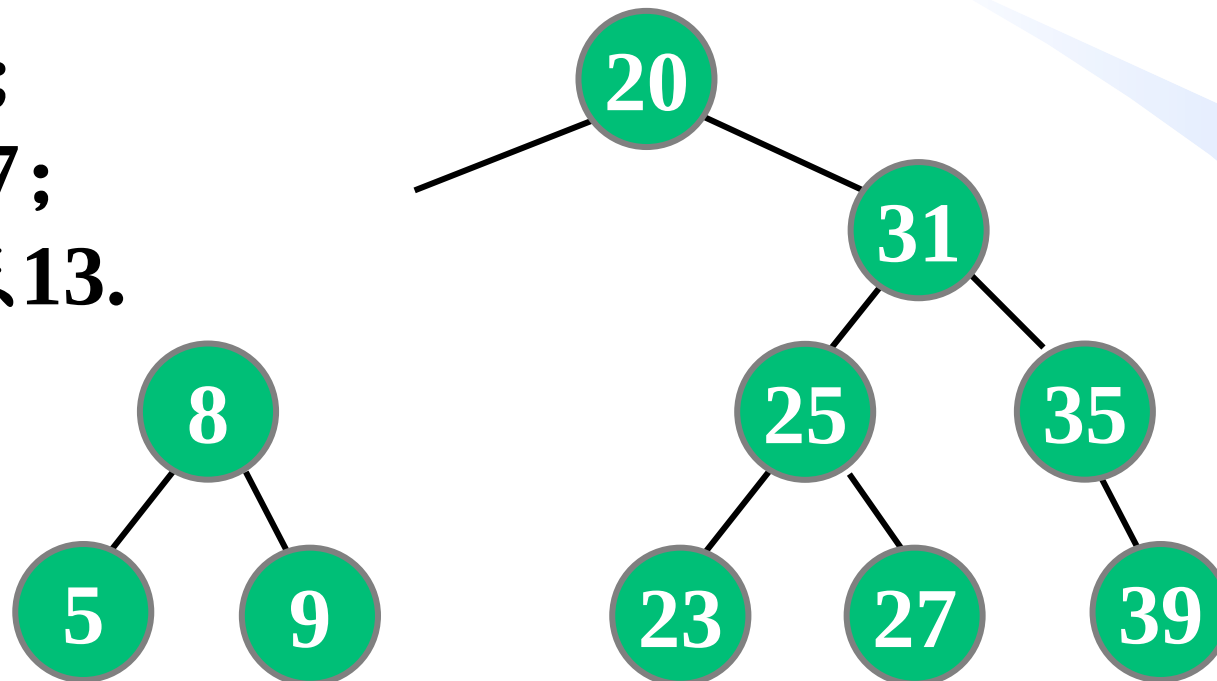


- 1) 插入数据9;
- 2) 删除结点17;
- 3) 再删除结点13.



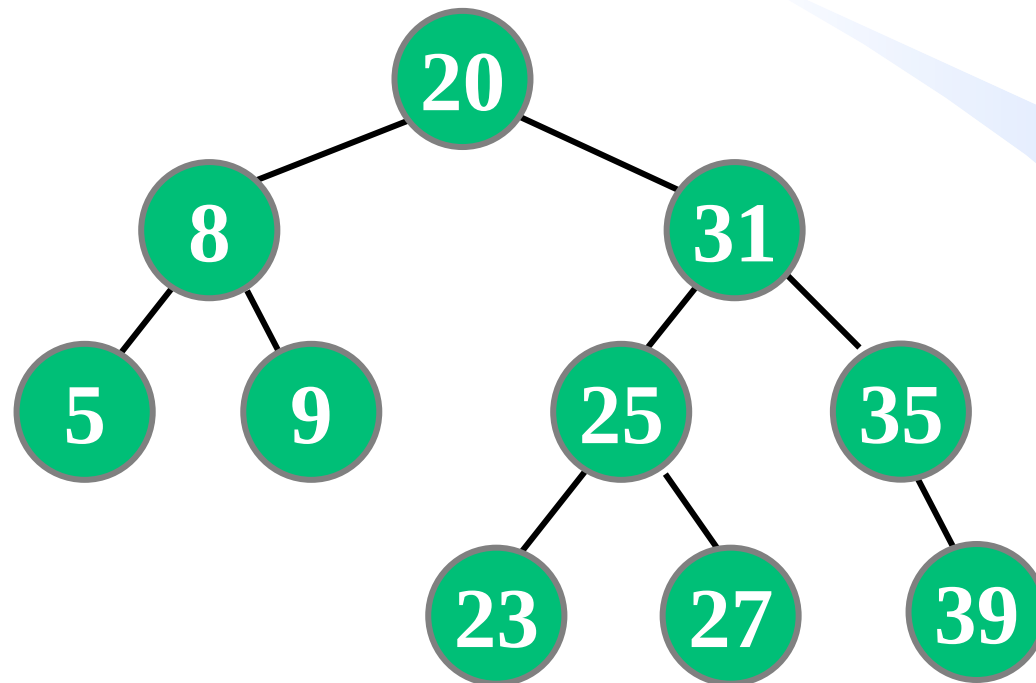


- 1) 插入数据9;
- 2) 删除结点17;
- 3) 再删除结点13.





- 1) 插入数据9;
- 2) 删除结点17;
- 3) 再删除结点13.





## 二叉查找树—插入算法的实现

- 在以 $root$ 为根的二叉查找树中插入关键词值为 $K$ 的结点，并保证插入后仍是一棵二叉查找树，若树中已有关键词等于 $K$ 的结点，则不再重复插入。【大厂面试题[LeetCode701](#)】
- 查找 $K$ ，然后在查找失败处插入 $K$ 。

## 二叉查找树—插入算法的实现

```
BSTnode* Search(BSTnode* root, int K){  
    if (root == NULL || root->key == K) return root;  
    if (K < root->key) return Search(root->left, K);  
    else return Search(root->right, K);  
}
```

为什么使用  
引用参数？

```
void Insert(BSTnode* &root, int K){  
    if (root == NULL) root = new BSTnode(K);  
    else if (K < root->key) Insert(root->left, K);  
    else if (K > root->key) Insert(root->right, K);  
}
```

时间复杂度  
 $O(h)$   
 $h$ 为树高

```
struct BSTnode {  
    int key;  
    BSTnode *left, *right;  
    BSTnode(int K){key=K; left=right=NULL;}  
};
```

## 二叉查找树—关于根指针

- 不同于查询操作，**插入**、**删除**操作将使二叉查找树的形态发生变化，在函数执行过程中，二叉树的**根指针**可能被修改，在函数退出后，这种修改应得以继续保留。
- **可行方案1**：使用根指针作为函数的“引用参数”，使函数中对根指针的修改在函数退出后仍得以保留。
- **可行方案2**：将根指针作为函数返回值，函数退出时返回当前最新的根指针。

# 类比

➤ 例如：设计一个函数对变量a的值加1。

错误方案：

```
void increase(int x){x++;}
```

主函数：

```
int a=5; increase(a);
```

可行方案1：

```
void increase(int &x){  
    x++;  
}
```

主函数：

```
int a=5;  
increase(a);
```

可行方案2：

```
int increase(int x){  
    x++; return x;  
}
```

主函数：

```
int a=5;  
a=increase(a);
```

## 二叉查找树—插入算法的两种实现方式

可行方案1（本课程所采用的方案）：

```
void Insert(BSTnode* &root, int K){  
    if (root == NULL) root = new BSTnode(K);  
    else if (K<root->key) Insert(root->left, K);  
    else if (K>root->key) Insert(root->right, K);  
}
```

可行方案2：

```
BSTnode* Insert(BSTnode* root, int K){  
    if (root == NULL) root = new BSTnode(K);  
    else if(K<root->key) root->left=Insert(root->left,K);  
    else if(K>root->key) root->right=Insert(root->right,K);  
    return root;  
}
```

## 二叉查找树—删除算法的实现

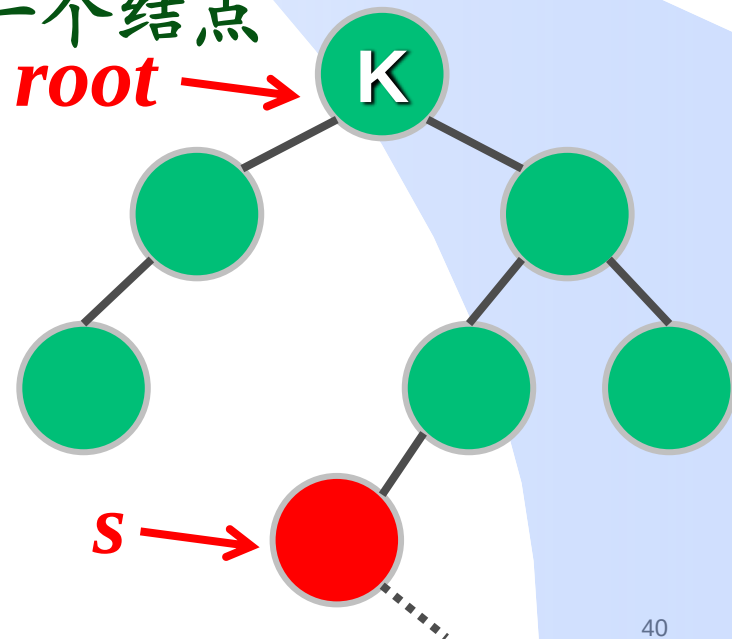
- 在以 $root$ 为根的二叉查找树中删除关键词值为 $K$ 的结点，并保证删除后仍是一棵二叉查找树，若树中无关键词等于 $K$ 的结点，则无操作。【大厂面试题[LeetCode450](#)】
- 在 $root$ 中找到关键词值为 $K$ 的结点，分3种情况删除 $K$ ：
  - ①  $K$ 有0个孩子：则直接删除。
  - ②  $K$ 有1个孩子：则子承父业（让其子结点替换 $K$ ）。
  - ③  $K$ 有2个孩子：则在其右子树找关键词最小的结点 $s$ （即右子树中根序列的第一个结点，亦即 $K$ 的中根后继）替换 $K$ ，然后删除原结点 $s$ （ $s$ 只有一个右孩子或无孩子）。

## 二叉查找树—删除算法的实现

```
void remove(BSTnode* &root, int K) {
    if(root==NULL) return;
    if(K<root->key) remove(root->left, K);           //在左子树删K
    else if(K>root->key) remove(root->right, K);      //在右子树删K
    else if(root->left!=NULL && root->right!=NULL){
        BSTnode *s=root->right;
        while(s->left!=NULL) s=s->left;
        root->key=s->key; //s为t右子树中根序列第一个结点
        remove(root->right, s->key);
    }
}
```

K==root->key

情况3: K有两个孩子



## 二叉查找树—删除算法的实现

```

void remove(BSTnode* &root, int K) {
    if(root==NULL) return;
    if(K<root->key) remove(root->left, K);           //在左子树删K
    else if(K>root->key) remove(root->right, K);      //在右子树删K
    else if(root->left!=NULL && root->right!=NULL){
        BSTnode *s=root->right;
        while(s->left!=NULL) s=s->left;
        root->key=s->key; //s为右子树中根序列第一个结点
        remove(root->right, s->key);
    }
    else{
        BSTnode* oldroot=root;
        root=(root->left!=NULL)? root->left:root->right;
        delete oldroot;
    }
}

```

**情况2: K有1个孩子**



## 二叉查找树—删除算法的实现

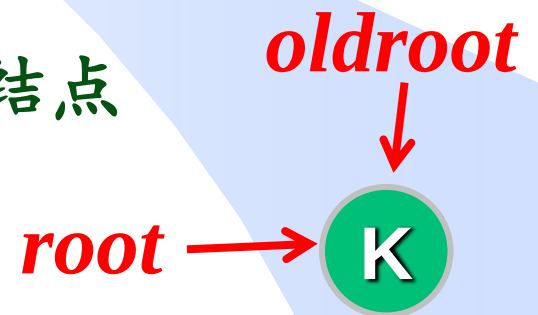
```

void remove(BSTnode* &root, int K) {
    if(root==NULL) return;
    if(K<root->key) remove(root->left, K);           //在左子树删K
    else if(K>root->key) remove(root->right, K);      //在右子树删K
    else if(root->left!=NULL && root->right!=NULL){
        BSTnode *s=root->right;
        while(s->left!=NULL) s=s->left;
        root->key=s->key; //s为右子树中根序列第一个结点
        remove(root->right, s->key);
    }
    else{
        BSTnode* oldroot=root;
        root=(root->left!=NULL)? root->left:root->right;
        delete oldroot;
    }
}

```

**情况1: K没有孩子**

**时间复杂度 $O(h)$**   
 **$h$ 为树高**



## 练习

给定一个整数  $n$ ，编程求出由  $n$  个结点组成且结点关键词为 1 到  $n$  的二叉查找树有多少种。【大厂面试题 [LeetCode96](#)】

回顾二叉树计数问题：

$$Catalan(n) = \frac{1}{n+1} C_{2n}^n$$

递推关系：

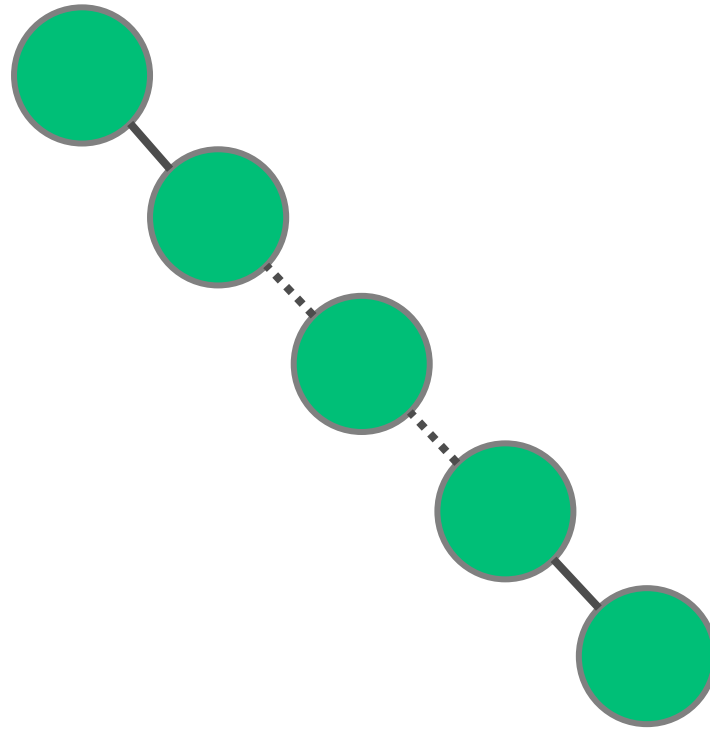
$$Catalan_n = \frac{2(2n-1)}{n+1} Catalan_{n-1}$$

$$Catalan_0 = 1$$

```
int numofBST(int n) {  
    int C=1;  
    for(int i=2; i<=n; i++)  
        C=C*2*(2*i-1)/(i+1);  
    return C;  
}
```

## 二叉查找树总结

- 则由  $n$  个互异关键词随机生成的二叉查找树，平均高度为  $O(\log n)$ ;
- 查找、插入、删除平均时间复杂度  $O(\log n)$ ，但最坏情况时间复杂度为  $O(n)$



Bruce Reed. The Height of A Random Binary Search Tree. *Journal of the ACM*. 50(3):306–332,2003.

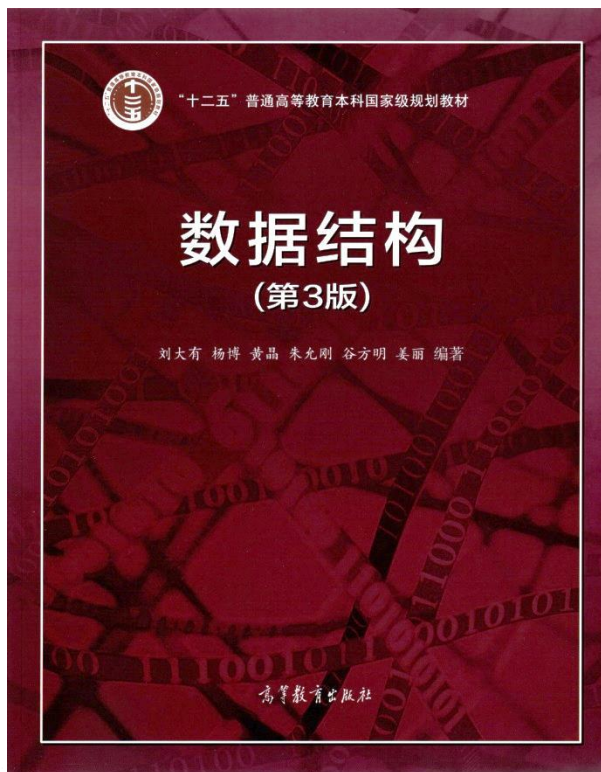
## 课下思考

二叉查找树里查询一个关键词的最坏时间复杂度是\_\_\_\_\_.

【阿里笔试题】

A.  $O(n)$     B.  $O(n\log n)$     C.  $O(n^2)$     D.  $O(n^3)$     E.  $O(\log n)$

给一个二叉查找树的先序序列，可以唯一确定该二叉查找树么？



## 二叉查找树

- 二叉查找树
- **高度平衡树**
- 红黑树简介
- 伸展树

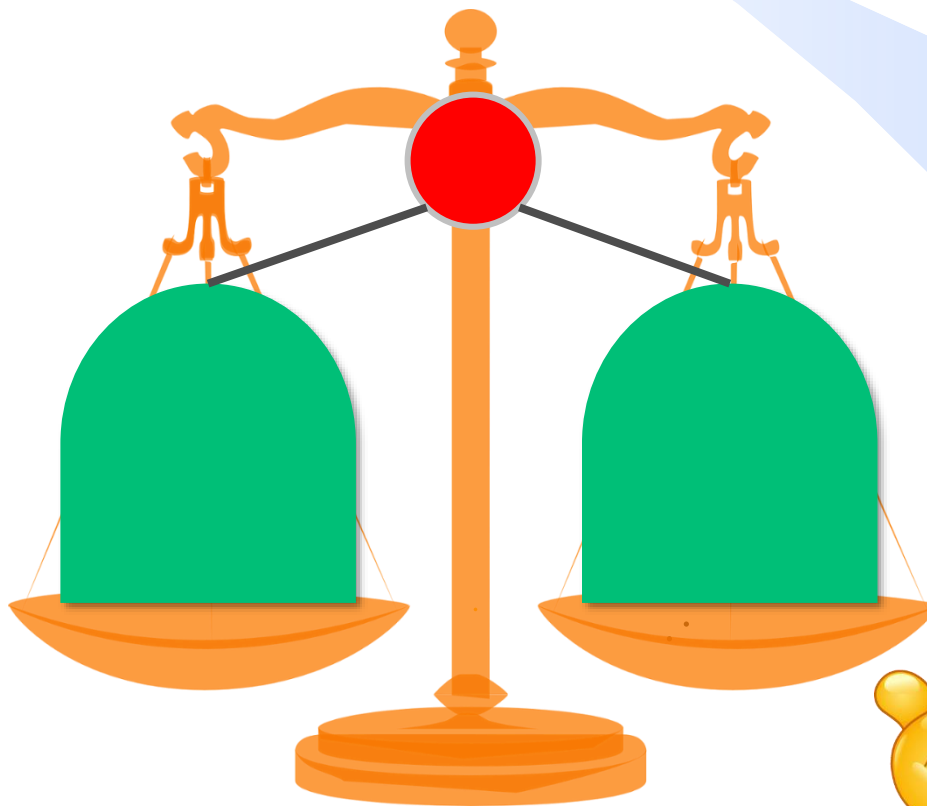
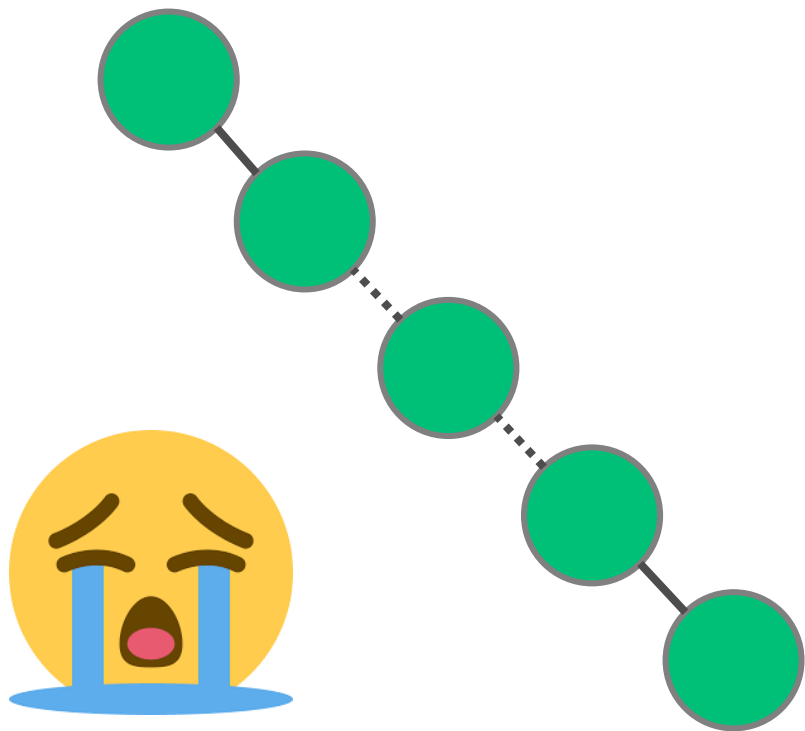
数据之法  
结构之美  
算法之道

zhuyungang@jlu.edu.cn

# 平衡二叉查找树——动机

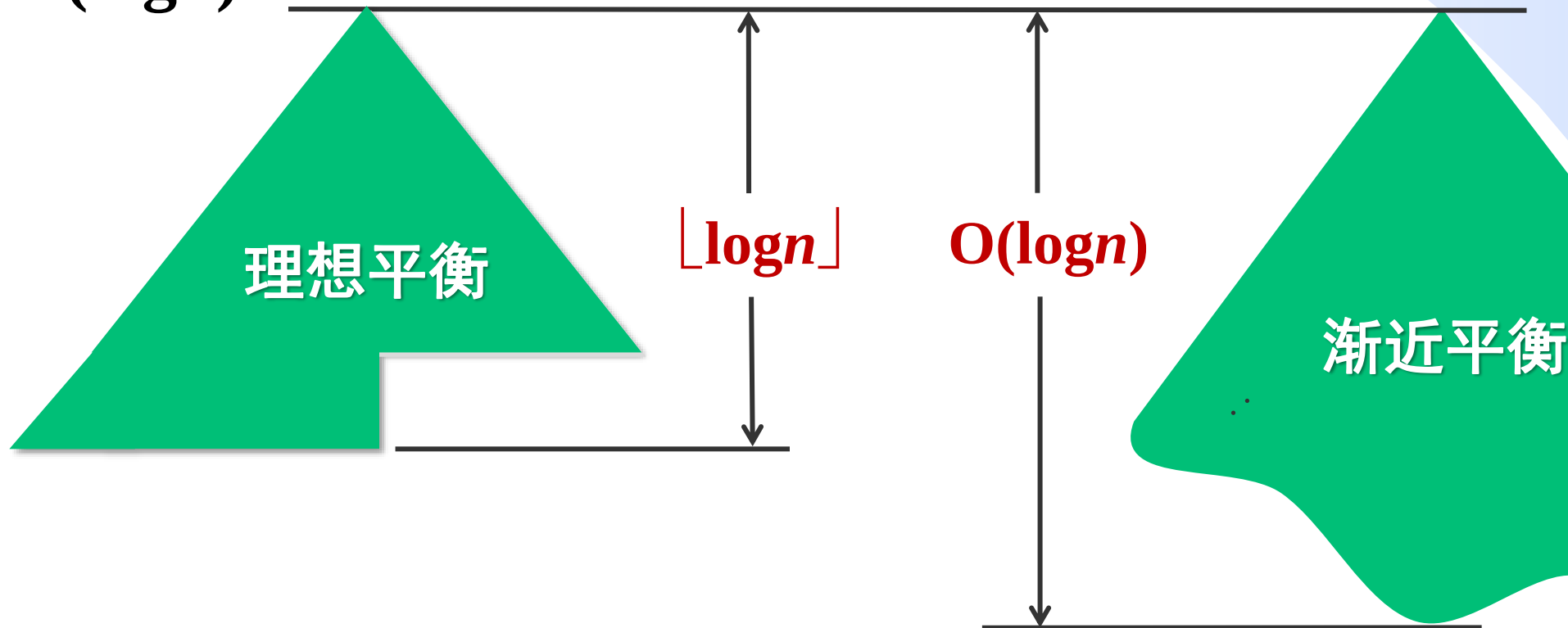
A

- 二叉查找树各核心操作的时间复杂度由树高度决定，最坏情况下树高可能为 $n$ ，所以希望高度尽可能小。
- 即树形尽可能“矮胖”，每个结点的左右子树的高度尽可能相等（称为平衡）。



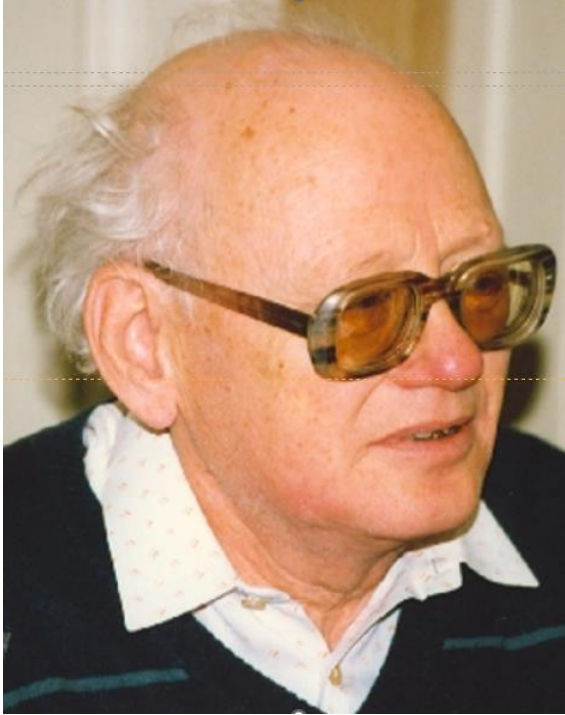
# 理想平衡 vs 渐近平衡

- **理想平衡**：满二叉树或完全二叉树，树高 $h=\lfloor \log n \rfloor$ ，条件过于苛刻且维护成本高。
- **渐近平衡**：适当放松标准，树高 $h=O(\log n)$ 即可接受， $O(h)=O(\log n)$ 。





# 高度平衡树（亦称AVL树）



**Georgy Adelson-Velsky**  
(1922-2014)  
前苏联数学家

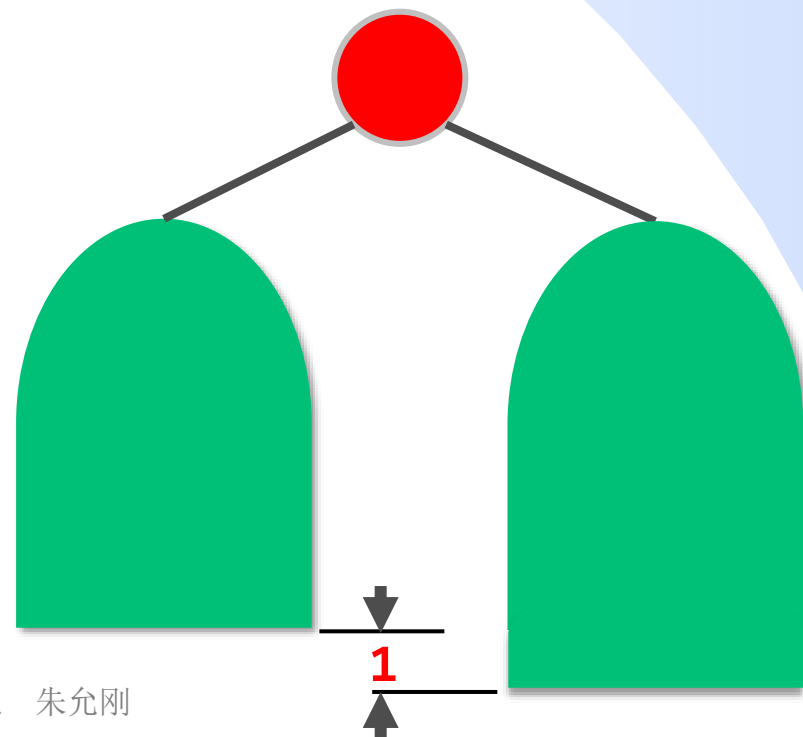
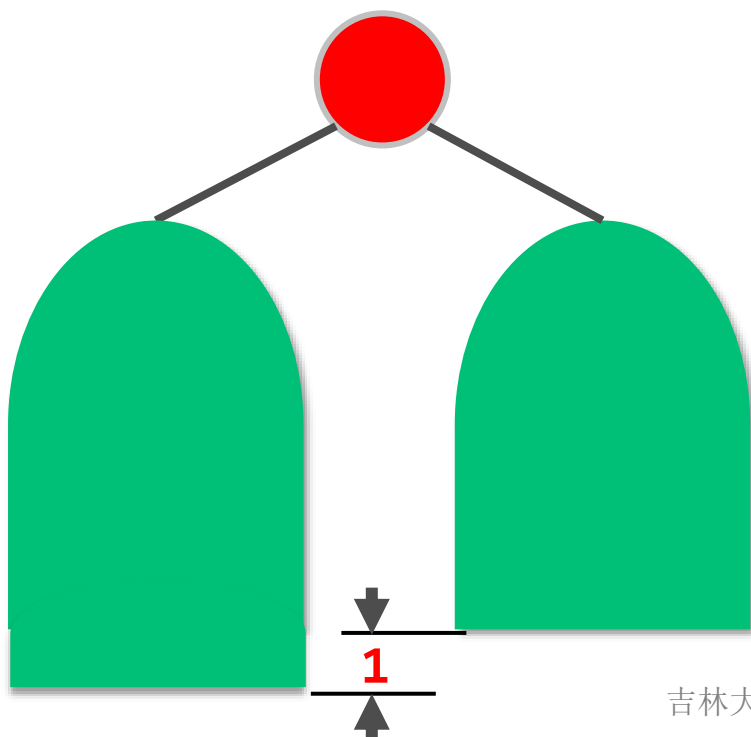


**Evgenii Landis**  
(1921-1997)  
前苏联数学家  
莫斯科州立大学教授



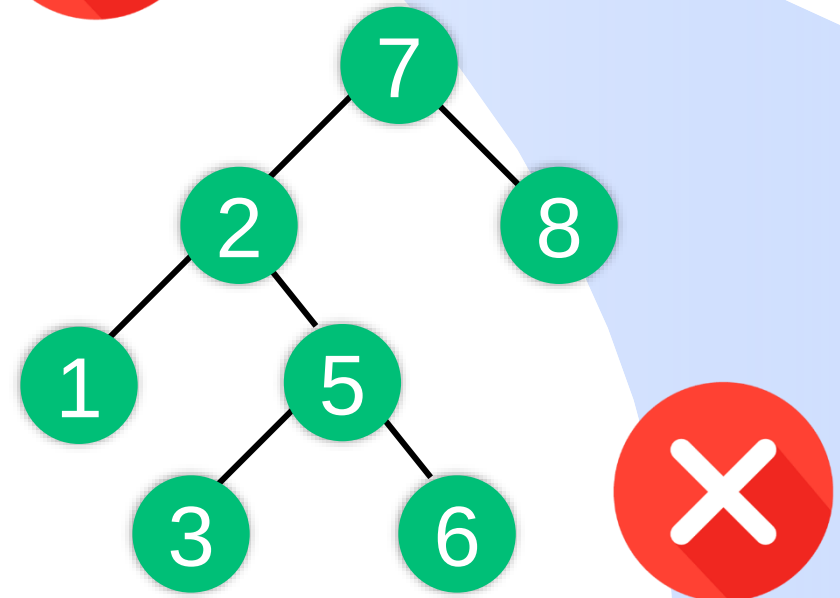
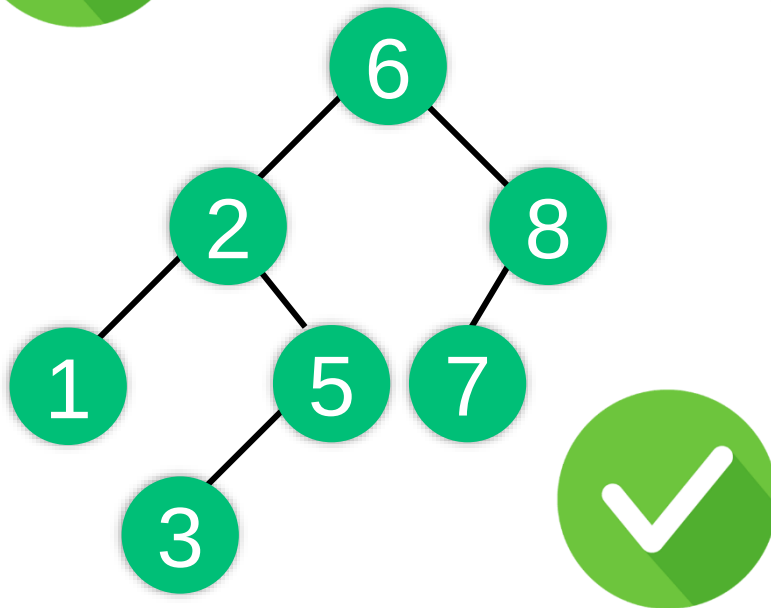
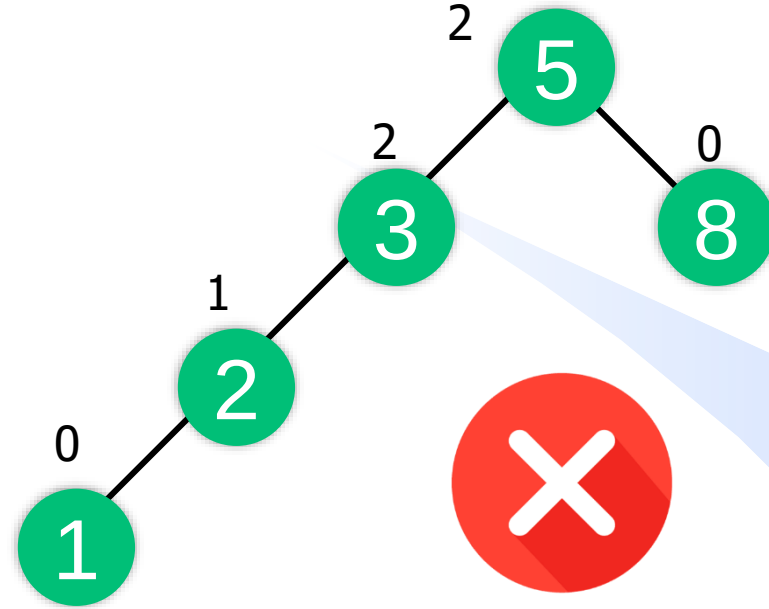
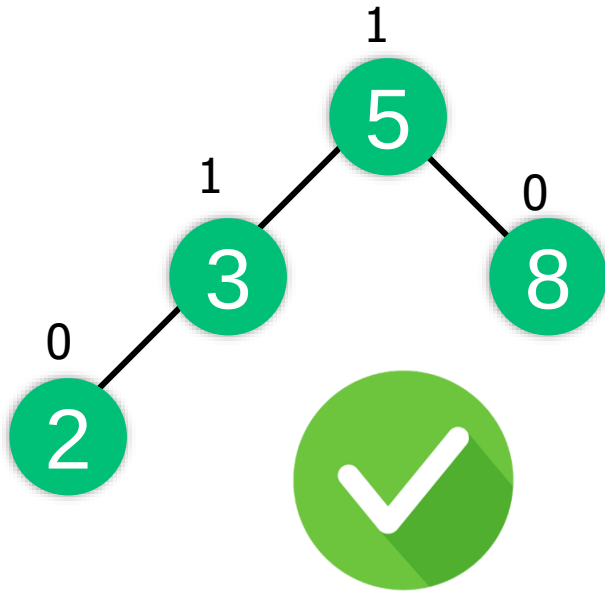
# 高度平衡树（亦称AVL树、平衡二叉树）

- 高度平衡树（亦称AVL树）是一棵满足如下条件的二叉查找树：任意结点的左子树和右子树的高度最多相差1。
- 即对于任意结点 $P$ ， $|P\text{的左子树高度} - P\text{的右子树高度}| \leq 1$
- AVL树中任意结点 $P$ 的平衡系数定义为： $P$ 的左子树高度减去右子树高度。从定义可知平衡系数只可能为：-1, 0, 1。



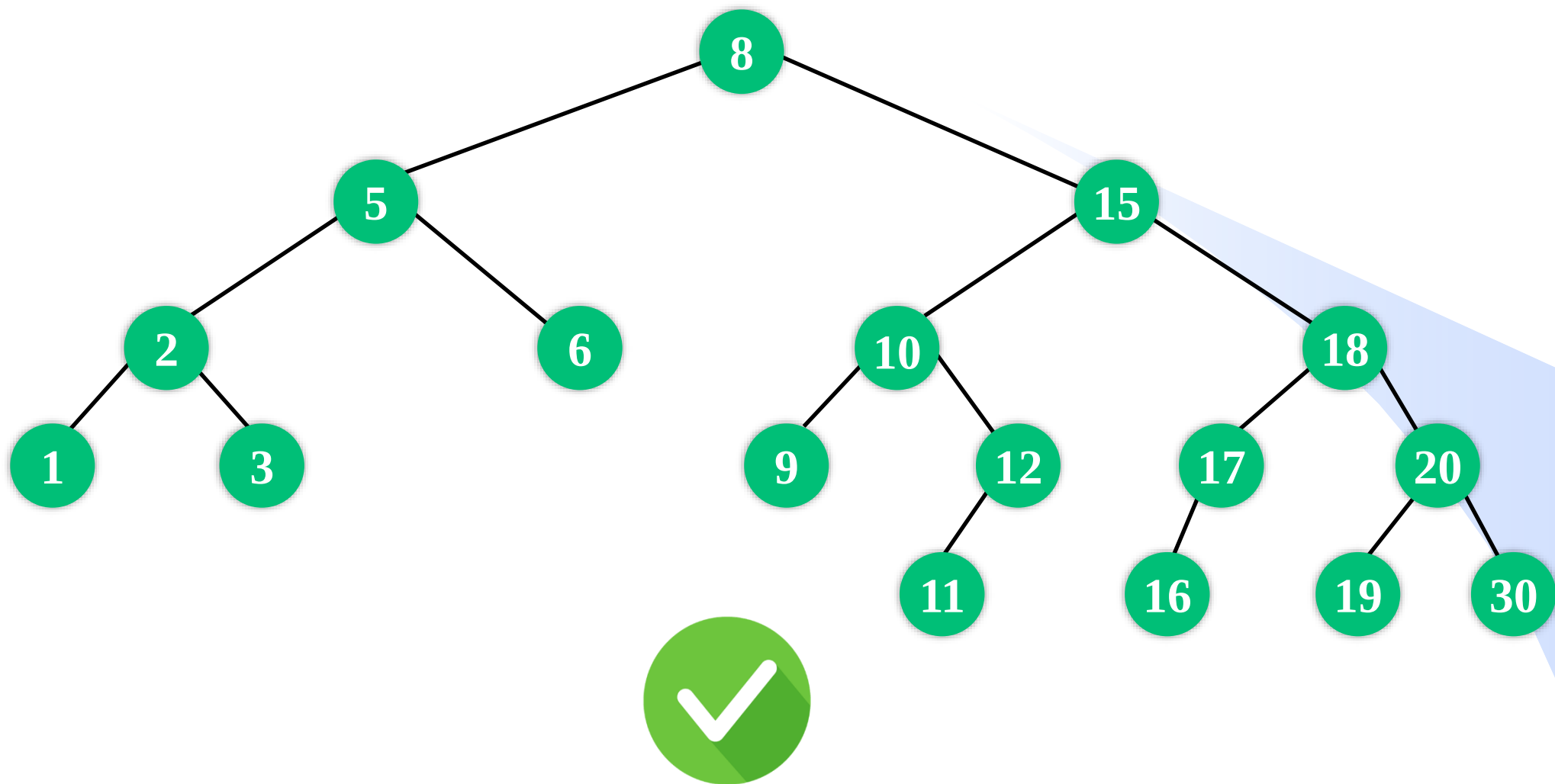
# 高度平衡树（AVL树）示例

A



# 高度平衡树（AVL树）示例

A



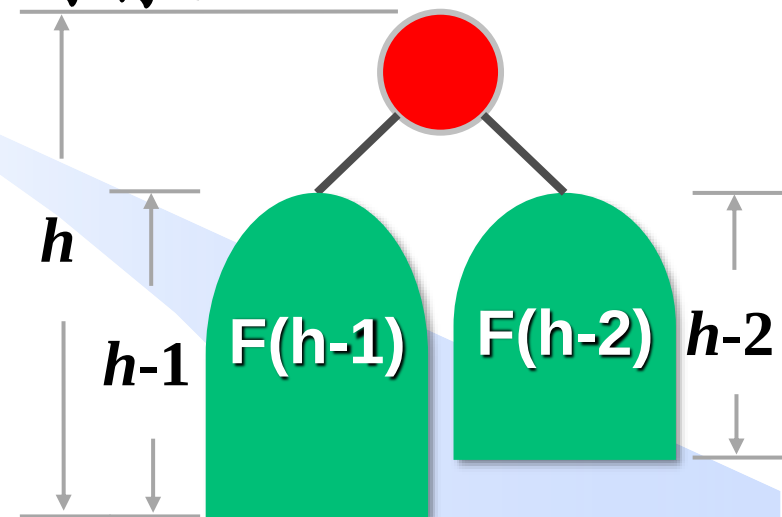
# 高度平衡树的高度——粗略估算

C

高度为 $h$ 的高度平衡树至少包含 $F(h)$ 个结点，则有：

$$F(h) = \begin{cases} 1 & h = 0 \\ 2 & h = 1 \\ F(h-1) + F(h-2) + 1 & h \geq 2 \end{cases}$$

$$\begin{aligned} F(h) &\geq 2F(h-2) \\ &\geq 2^2 F(h-4) \\ &\geq 2^3 F(h-6) \\ &\dots\dots \\ &\geq 2^{h/2} F(0) \\ &\geq 2^{h/2} \end{aligned}$$



若高度为 $h$ 的高度平衡树包含 $n$ 个结点，

则有：  $n \geq 2^{h/2}$

故：  $h \leq 2\log n$

# 高度平衡树的高度——更精确的上界

**定理** 具有 $n$ 个结点的高度平衡树的高度 $h$ 满足：

$$h \leq 1.4404 \log_2(n + 2) - 0.3277$$

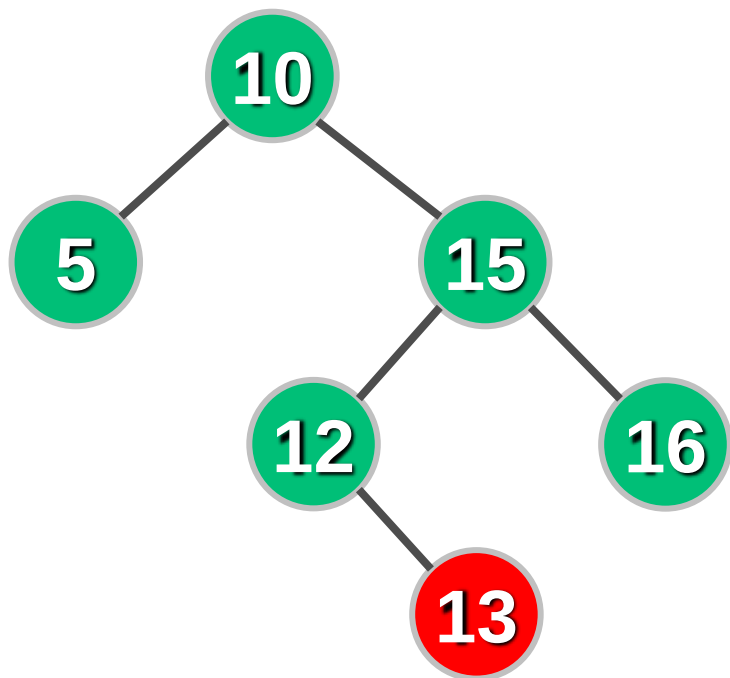
**AVL树的高度最多比完全二叉树高约44%**

# 高度平衡树的实现

```
struct AVLnode {  
    int key;           //关键词  
    int height;        //以该结点为根的子树高度  
    AVLnode *left, *right;  
    AVLnode(int K){ key=K; height=0; left=right=NULL; }  
};  
  
int Height(AVLnode *t){ return(t==NULL)?-1:t->height; }  
int max(int a, int b){ return (a>b)? a:b; }  
void UpdateHeight(AVLnode *t){  
    t->height = max(Height(t->left),Height(t->right))+1;  
}
```

# 高度平衡树的核心操作

- **查找**：与普通二叉查找树一致。
- **插入**和**删除**：先使用二叉查找树的插入/删除方法，但插入/删除一个结点后，有可能破坏AVL树的平衡性。



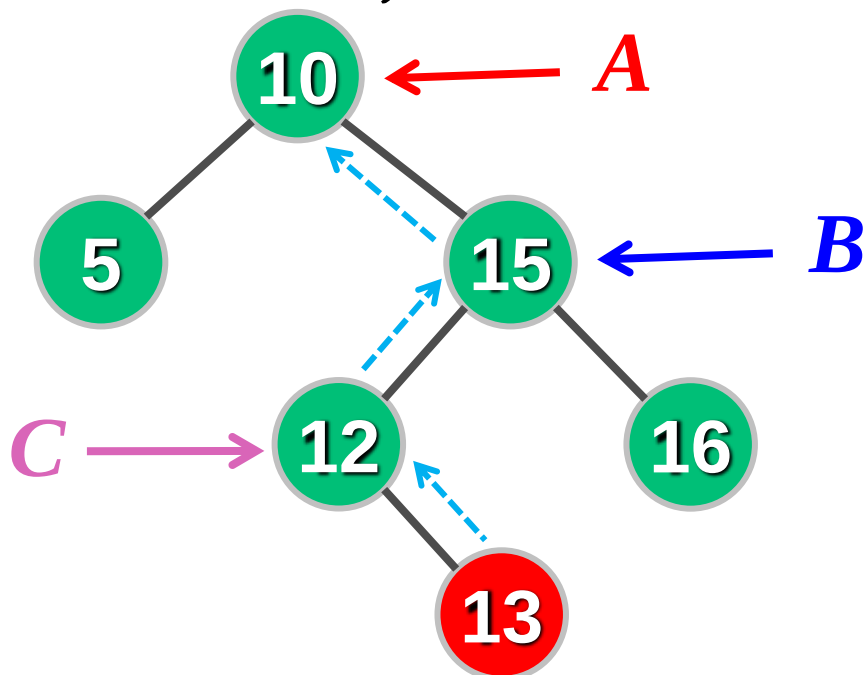
例如：插入关键词为**13**的结点将会破坏结点⑩处的平衡。

需对平衡树进行调整，恢复其平衡性，实现这种调整的操作称为“**旋转 (Rotation)**”。

# 高度平衡树的插入——准备工作

A

- 插入一个新结点X后，可能会破坏某些结点的平衡性。
- ① 应调整失去平衡的最小子树，即沿**插入点**到**根结点**的路径**向上**找第一个不平衡的结点A。
- ② 从发生不平衡的结点起，沿刚才的路径向下找**祖孙三代**结点A、B、C。即从结点A起，沿刚才的路径再找结点A下两层的结点B和C。

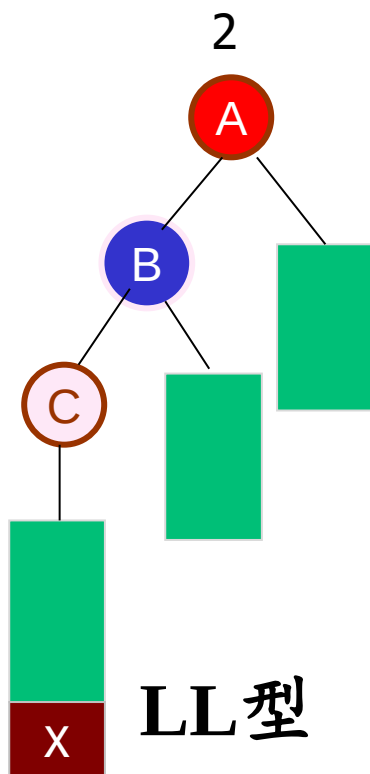




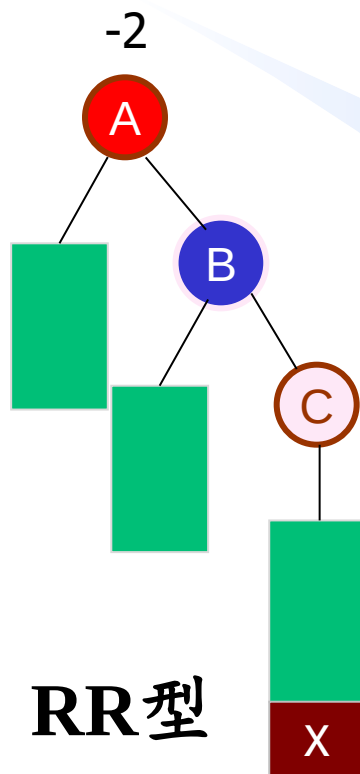
# 高度平衡树的插入

造成结点A不平衡的原因有下面几种情况：

1. 对A的左孩子的左子树进行插入，导致不平衡



2. 对A的右孩子的右子树进行插入，导致不平衡

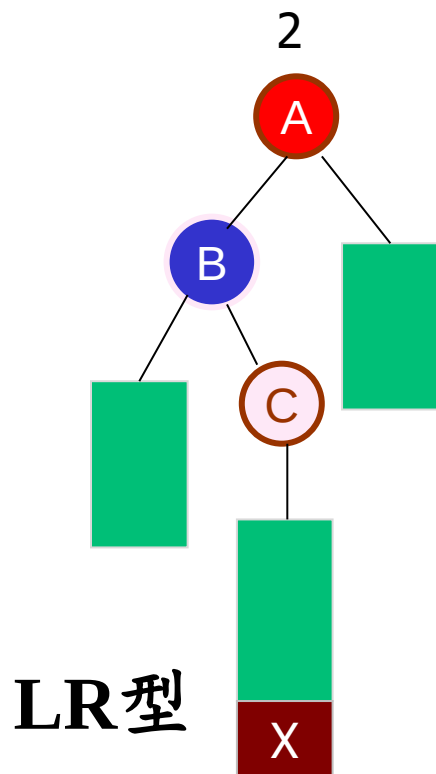


A、B、C处于一条直线上

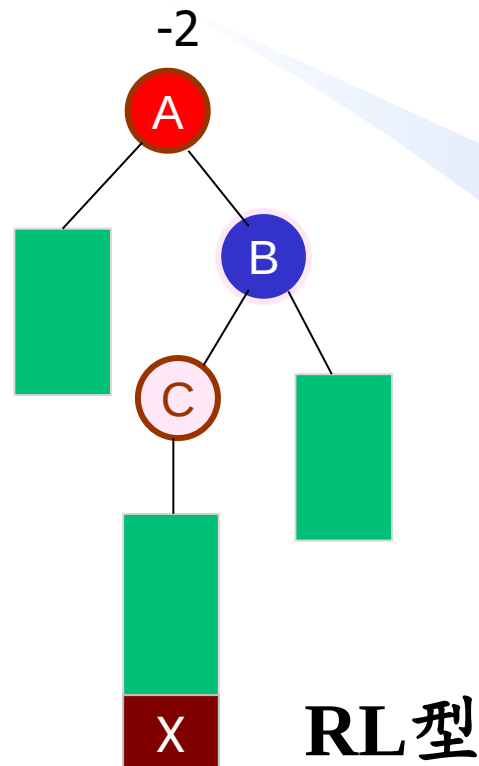
# 高度平衡树的插入

造成结点A不平衡的原因有下面几种情况：

3. 对A的左孩子的右子树进行插入, 导致不平衡。



4. 对A的右孩子的左子树进行插入, 导致不平衡。



A、B、C处于一条折线上

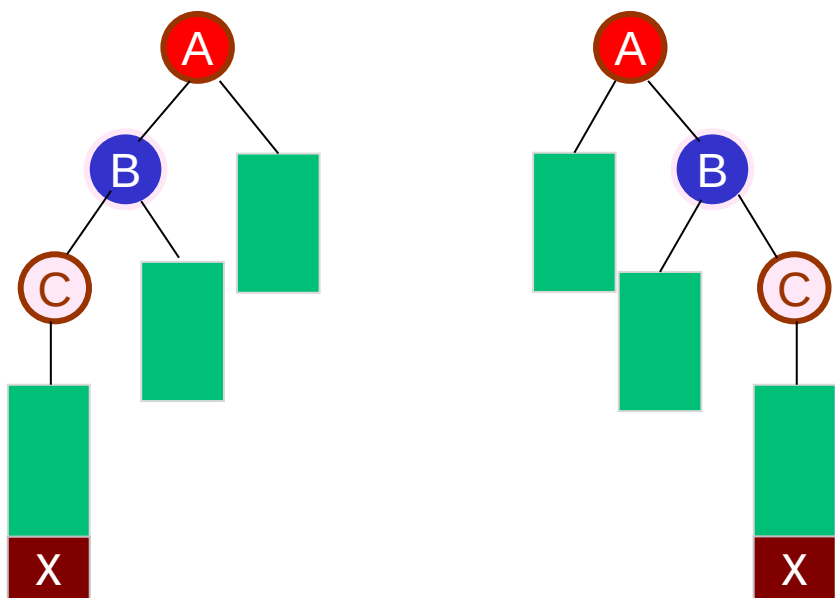
# 高度平衡树的插入——平衡方案

情况1、2:

即LL/RR型:

A、B、C处于一条直线上

平衡方案: 单旋转

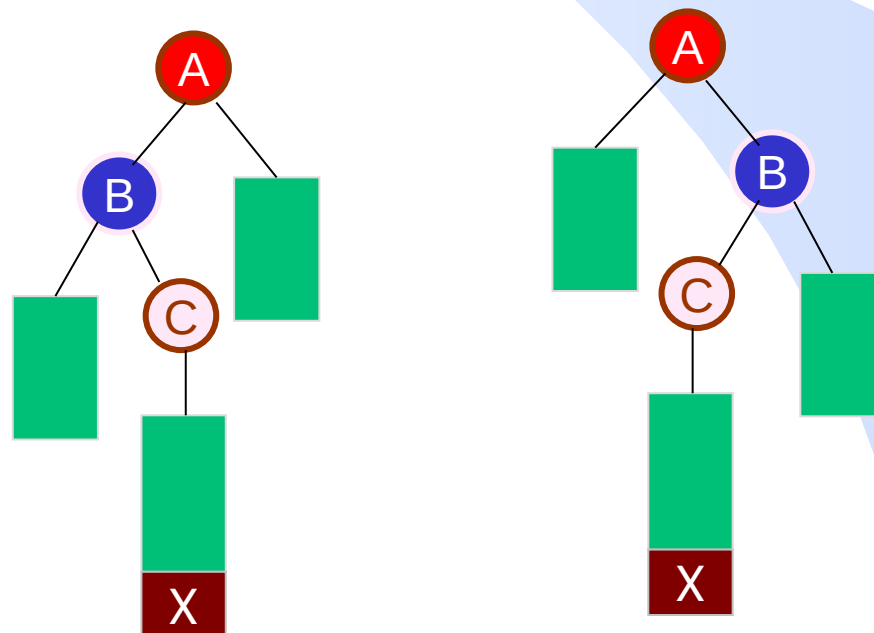


情况3、4:

即LR/RL型:

A、B、C处于一条折线上

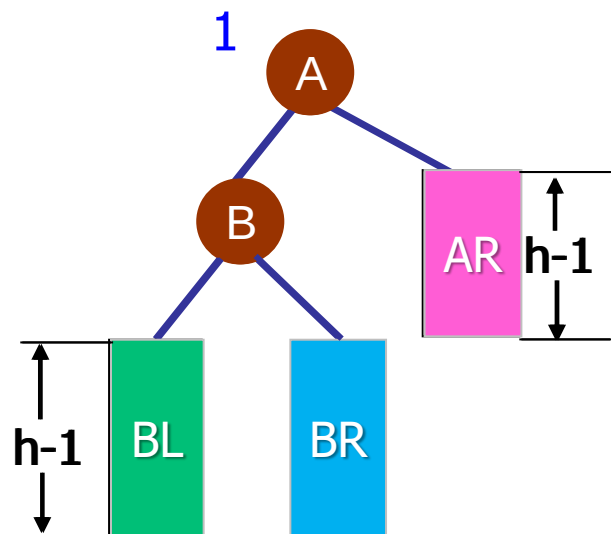
平衡方案: 双旋转



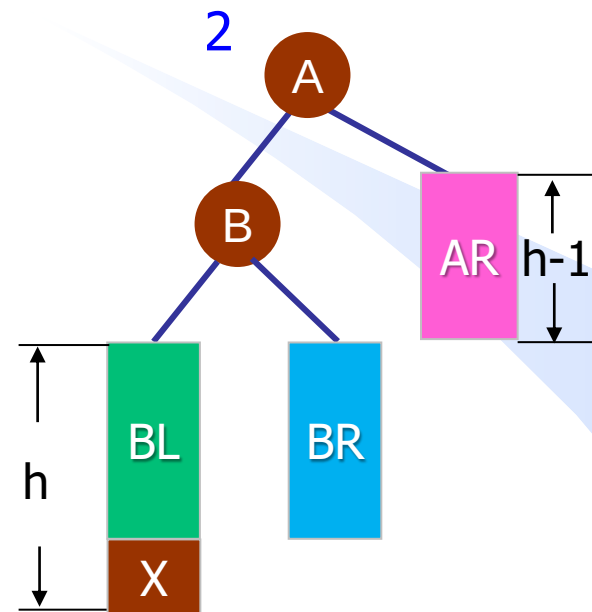
# 高度平衡树的插入—单旋转

A

## 一、单旋转-LL型



插入前  
高度 $h+1$

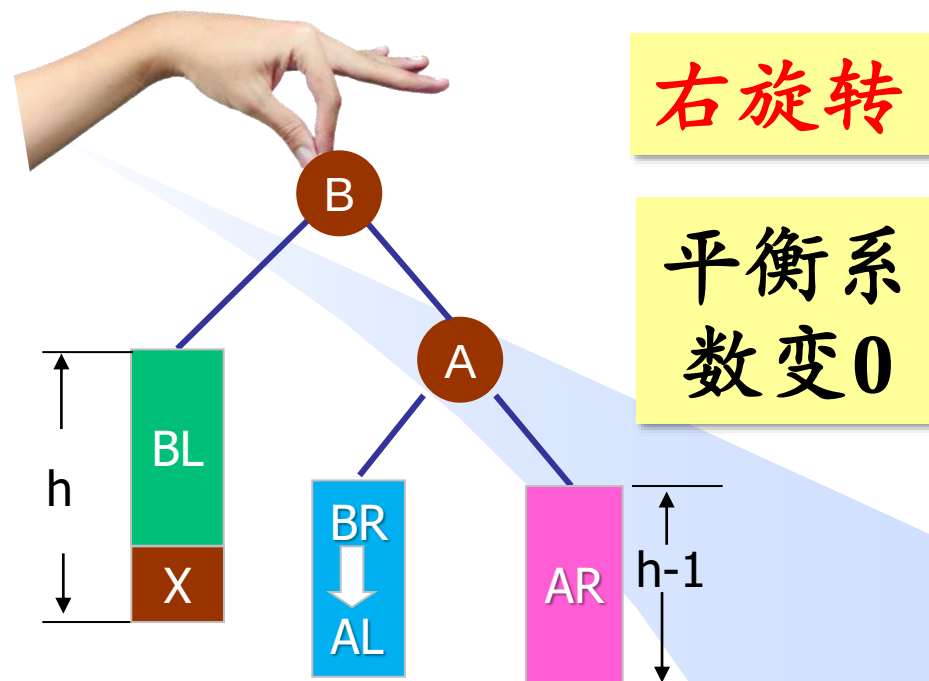
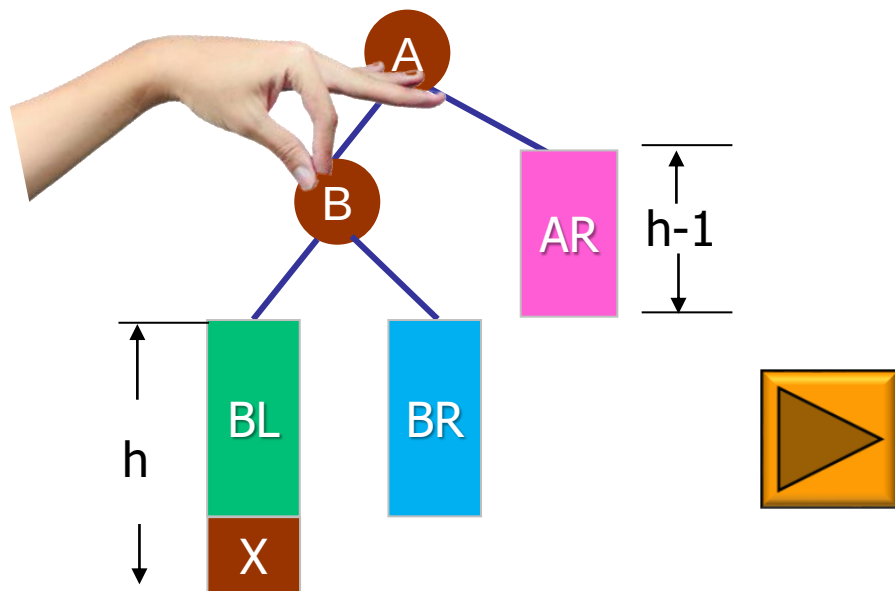


插入X后不再平衡

# 高度平衡树的插入—单旋转

A

## 一、单旋转-LL型的调整过程

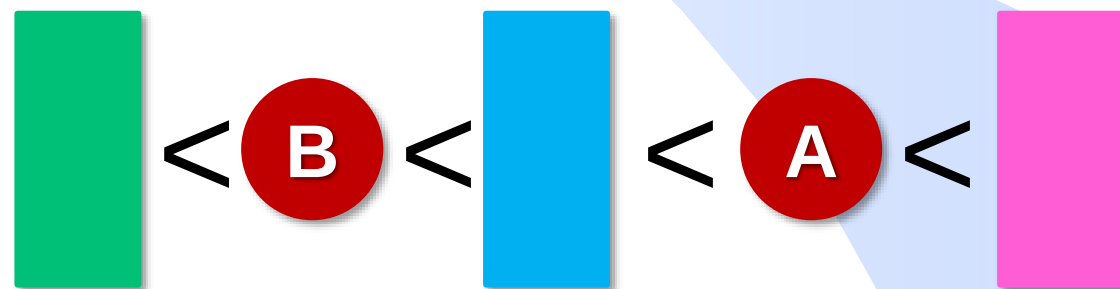
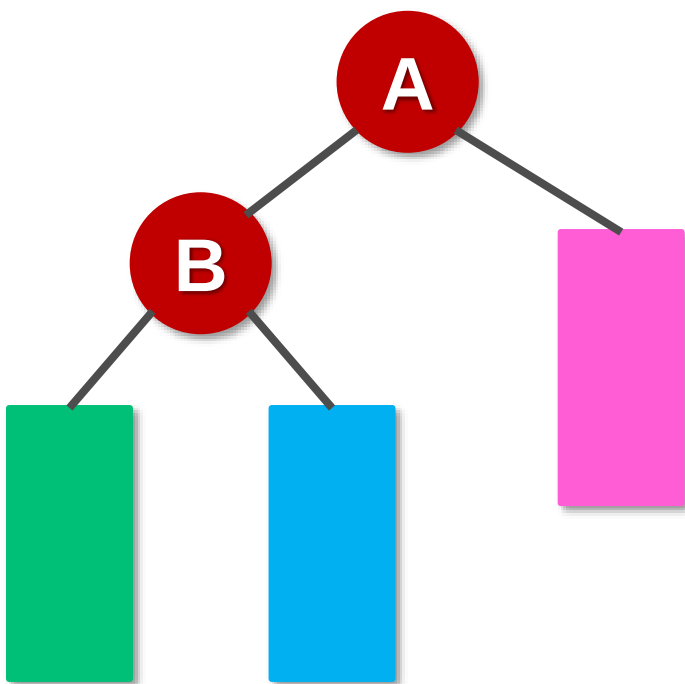


抓住结点B往上拽使之成为根结点（以B为轴将A右转/顺时针旋转），使A成为B的右孩子，BL、AR不变，BR成为A的左子树。

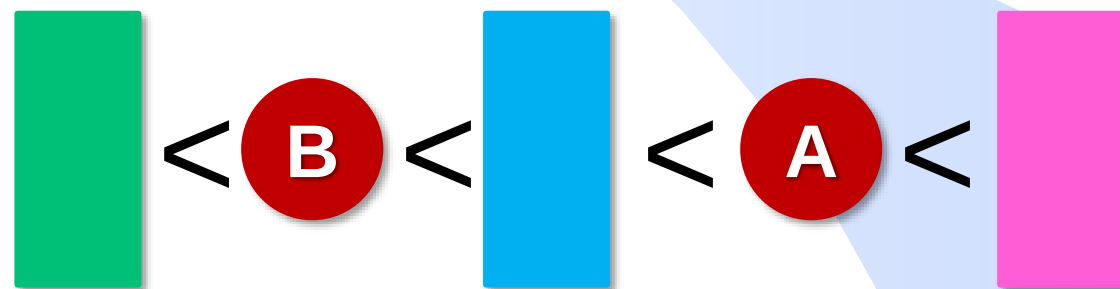
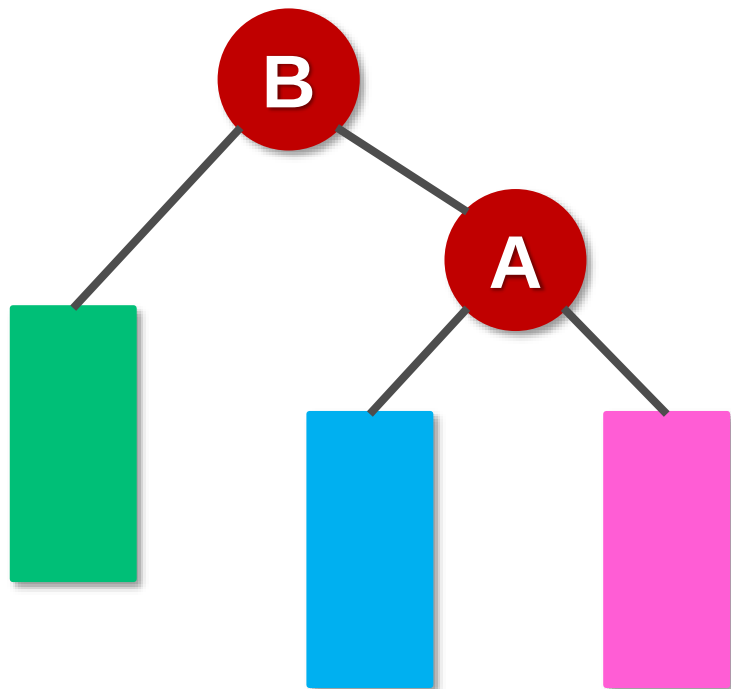
左子树高，则右转

插入前：高度 $h+1$   
插入后：高度 $h+1$

## 演示：以B为轴将A右转



## 演示：以B为轴将A右转

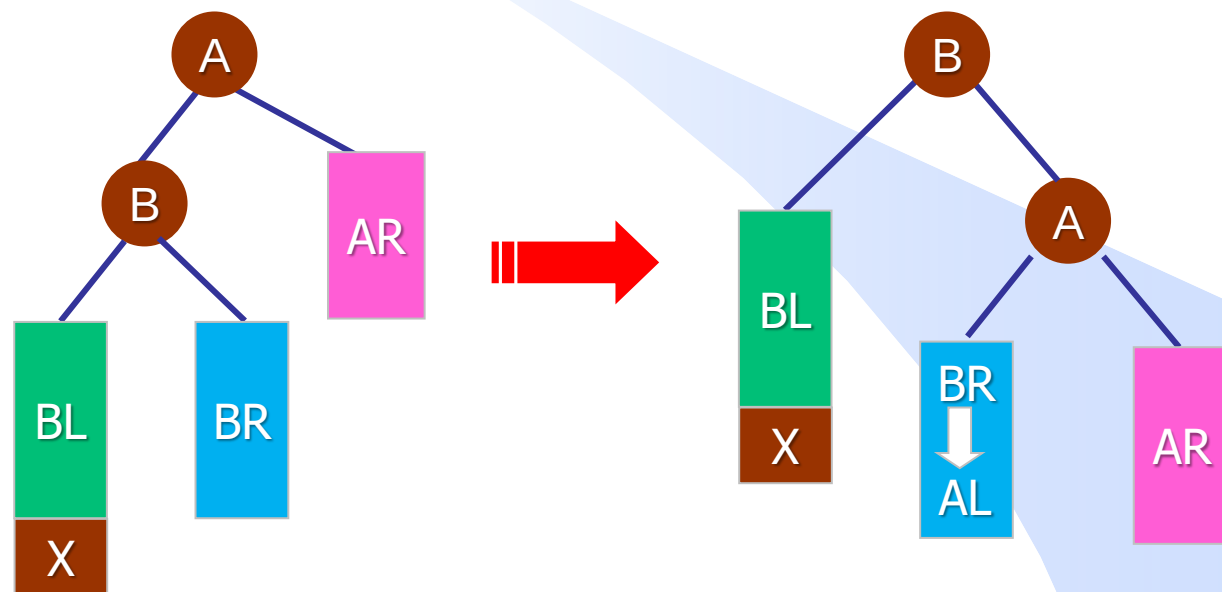


# 高度平衡树的插入—单旋转实现

C

## LL型的调整算法的实现

```
void LL(AVLnode* &A) {  
    AVLnode *B = A->left;  
    A->left = B->right;  
    B->right = A;  
    UpdateHeight(A);  
    UpdateHeight(B);  
    A = B;  
}
```

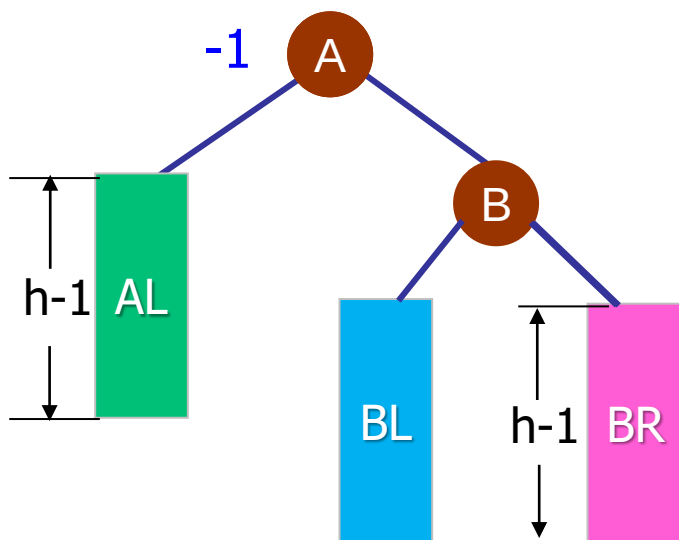




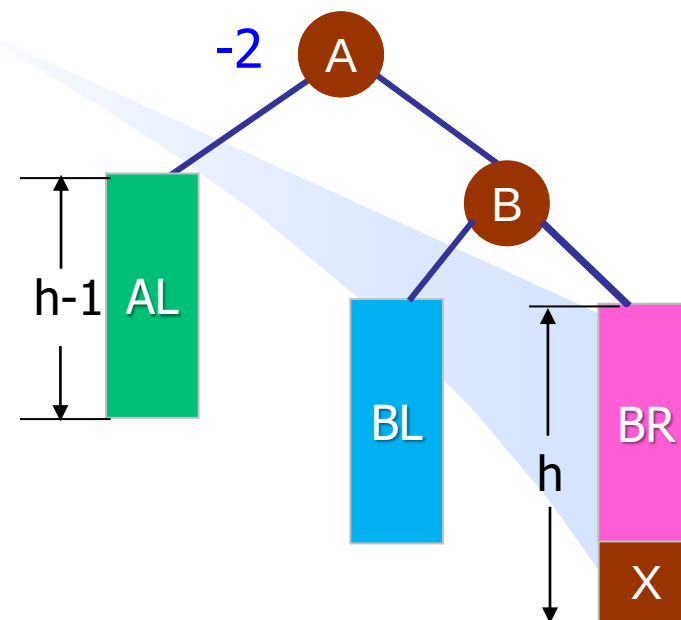
# 高度平衡树的插入—单旋转

A

## 二、单旋转-RR型



插入前  
高度 $h+1$

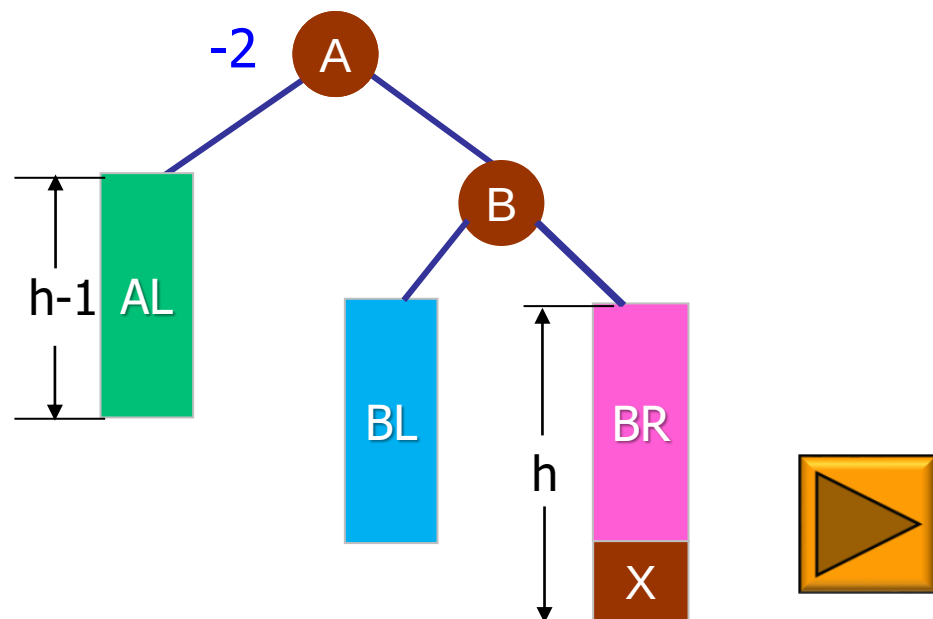


插入X后不再平衡

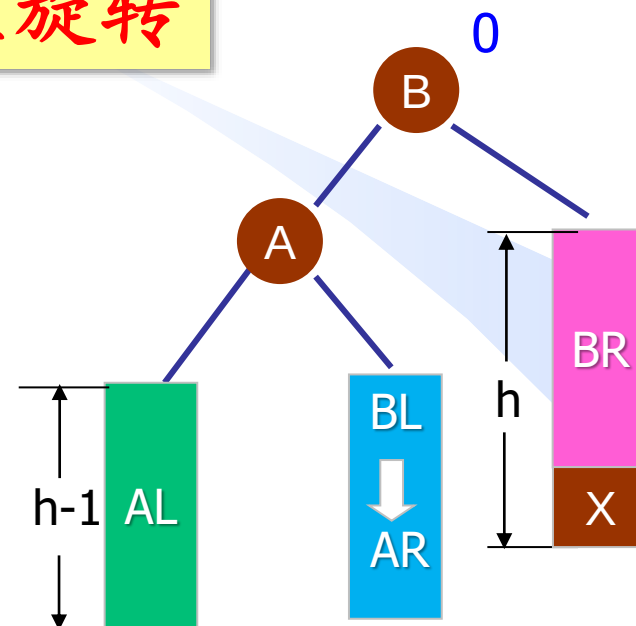
# 高度平衡树的插入—单旋转

A

## 二、单旋转-RR型的调整过程



左旋转

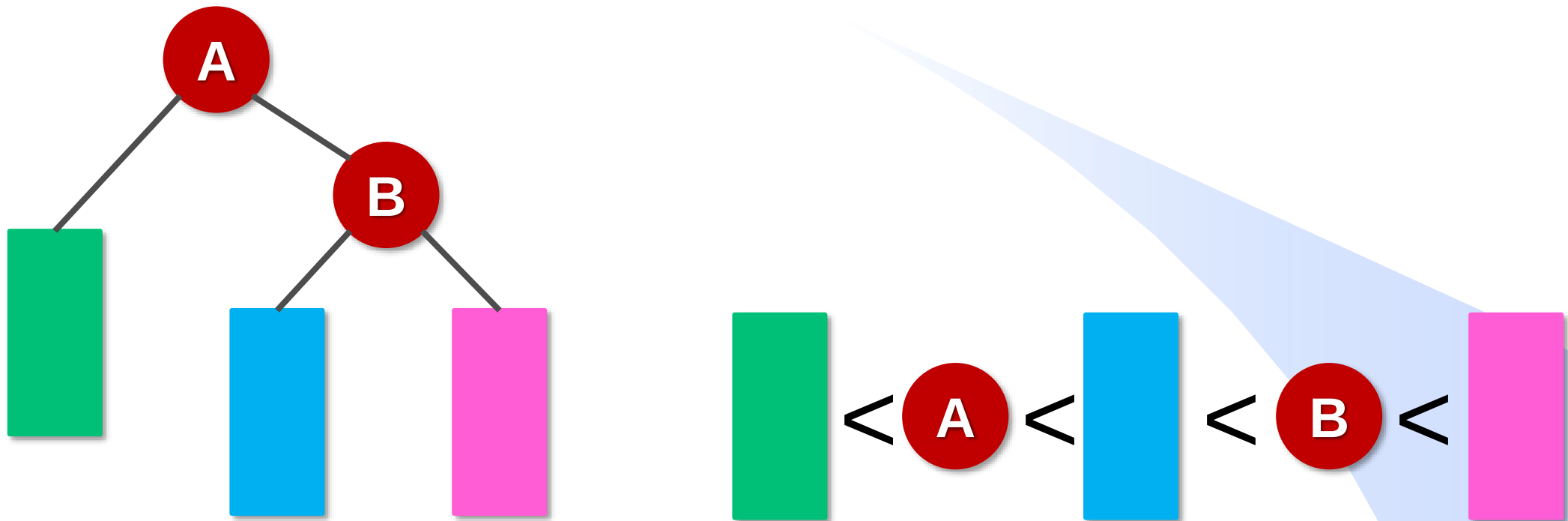


抓住结点B往上拽使之成为根结点（以B为轴将A左转/逆时针旋转），使得，A成为B的左孩子，BR、AL不变，BL成为A的右子树。

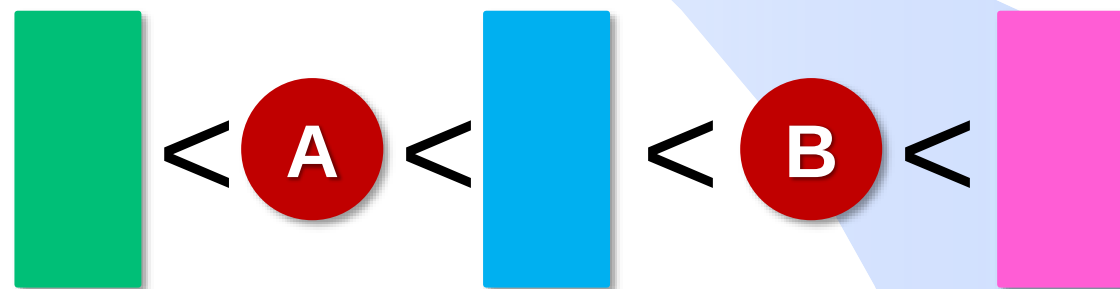
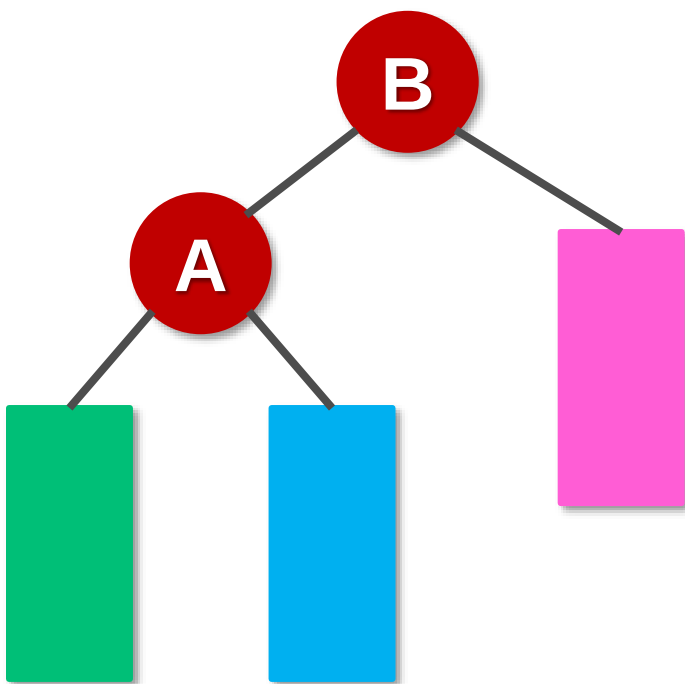
右子树高，则左转

插入前：高度 $h+1$   
插入后：高度 $h+1$

## 演示：以B为轴将A左转



# 演示：以B为轴将A左转

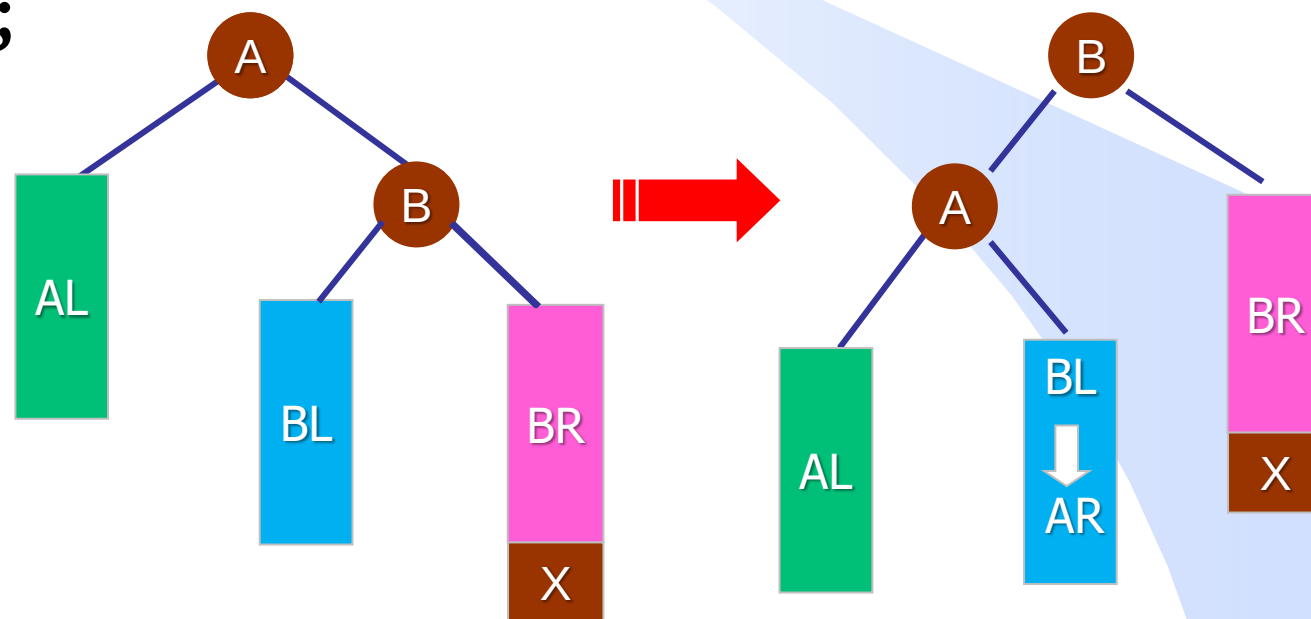


# 高度平衡树的插入—单旋转实现

C

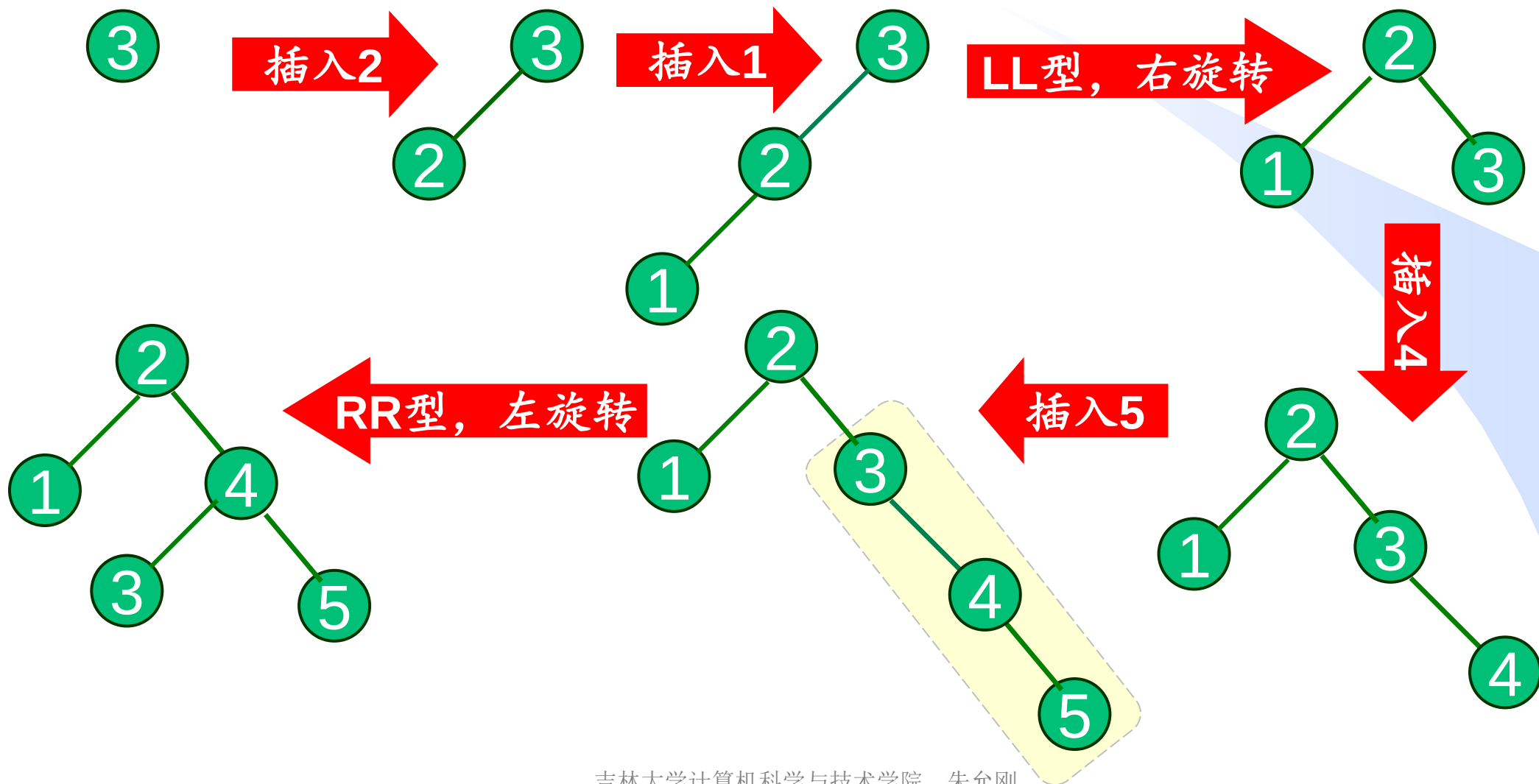
## RR型的调整算法的实现

```
void RR(AVLnode* &A) {  
    AVLnode *B = A->right;  
    A->right = B->left;  
    B->left = A;  
    UpdateHeight(A);  
    UpdateHeight(B);  
    A = B;  
}
```



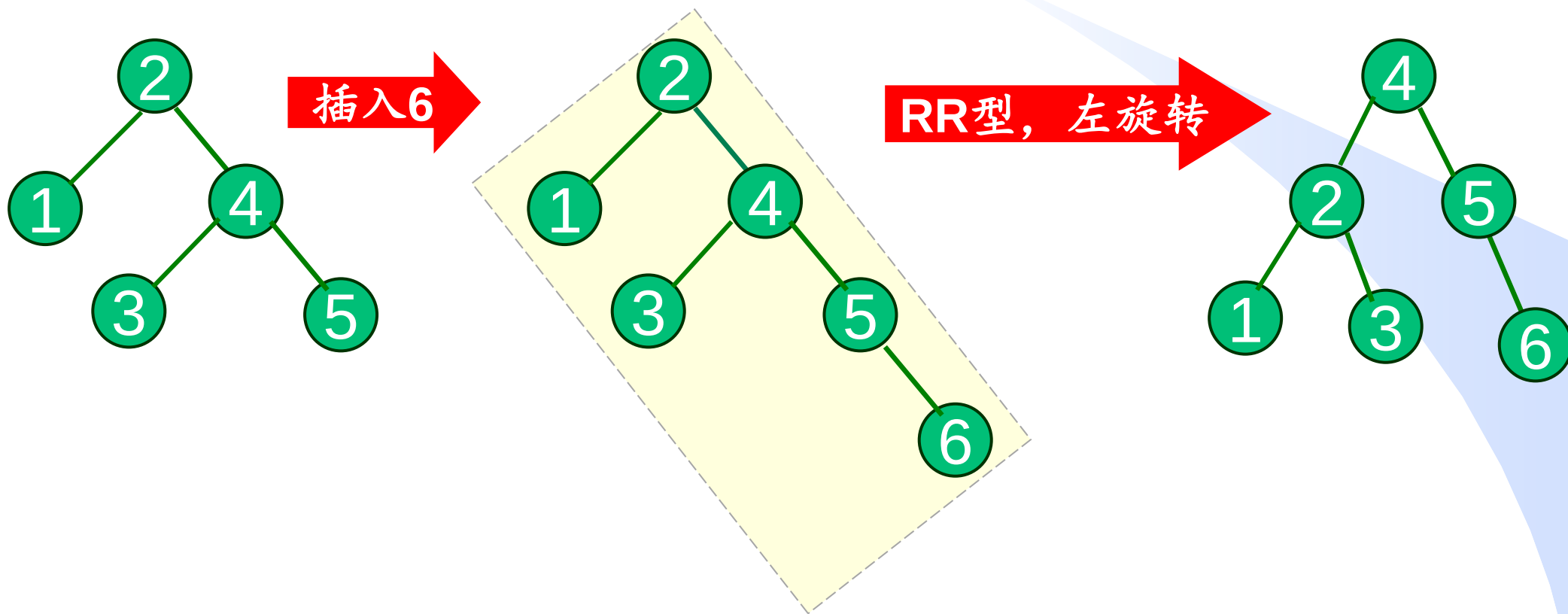
# 单旋转举例

从空AVL树开始依次插入关键字3, 2, 1, 4, 5, 6, 7



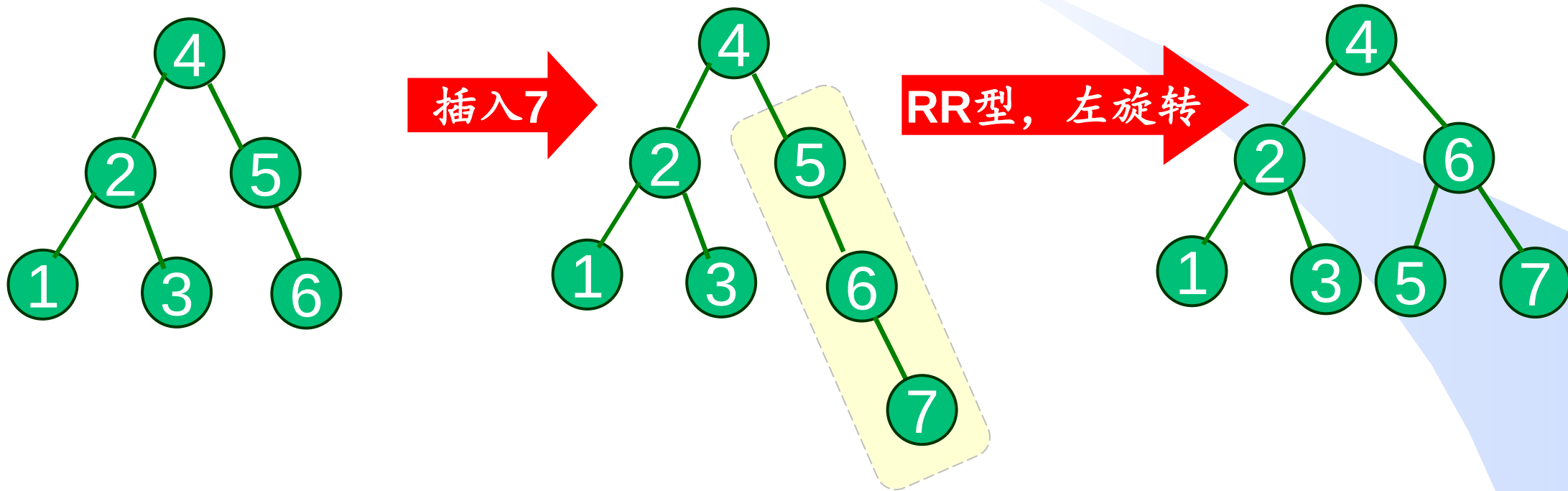
# 单旋转举例

从空AVL树开始依次插入关键字3, 2, 1, 4, 5, 6, 7



# 单旋转举例

从空AVL树开始依次插入关键字3, 2, 1, 4, 5, 6, 7

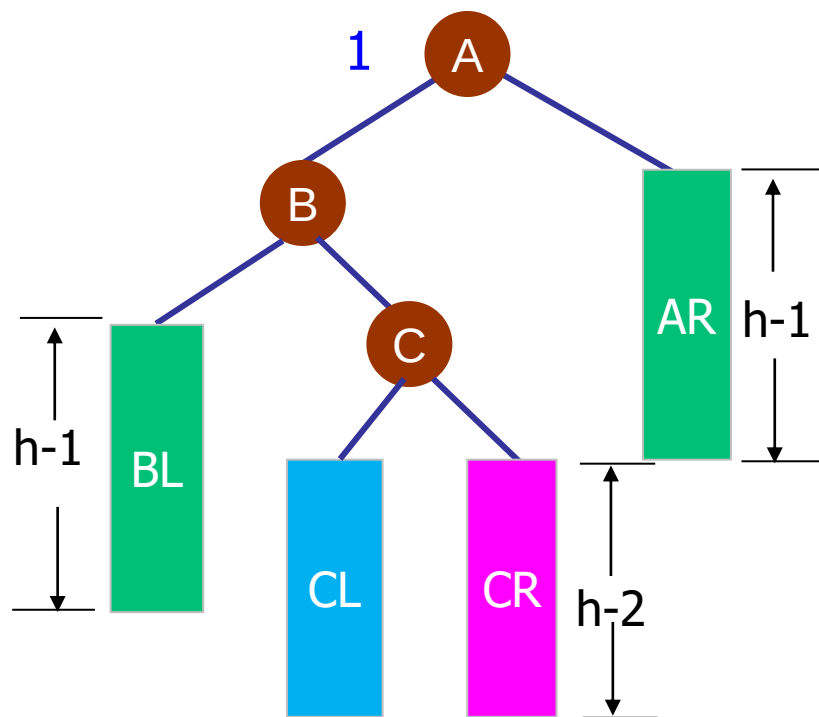




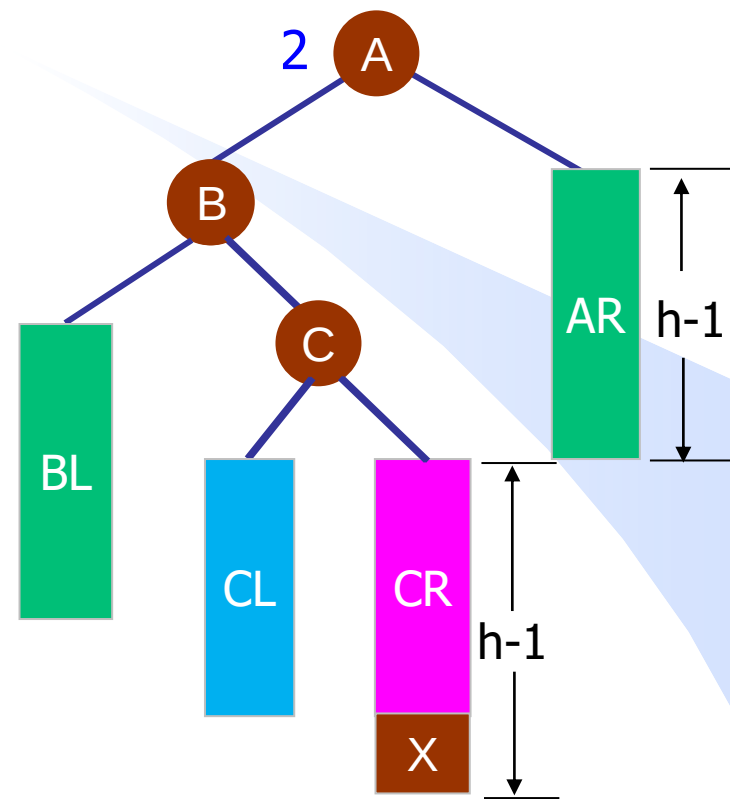
# 高度平衡树的插入—双旋转

A

## 三、双旋转-LR型



插入前：高度 $h+1$



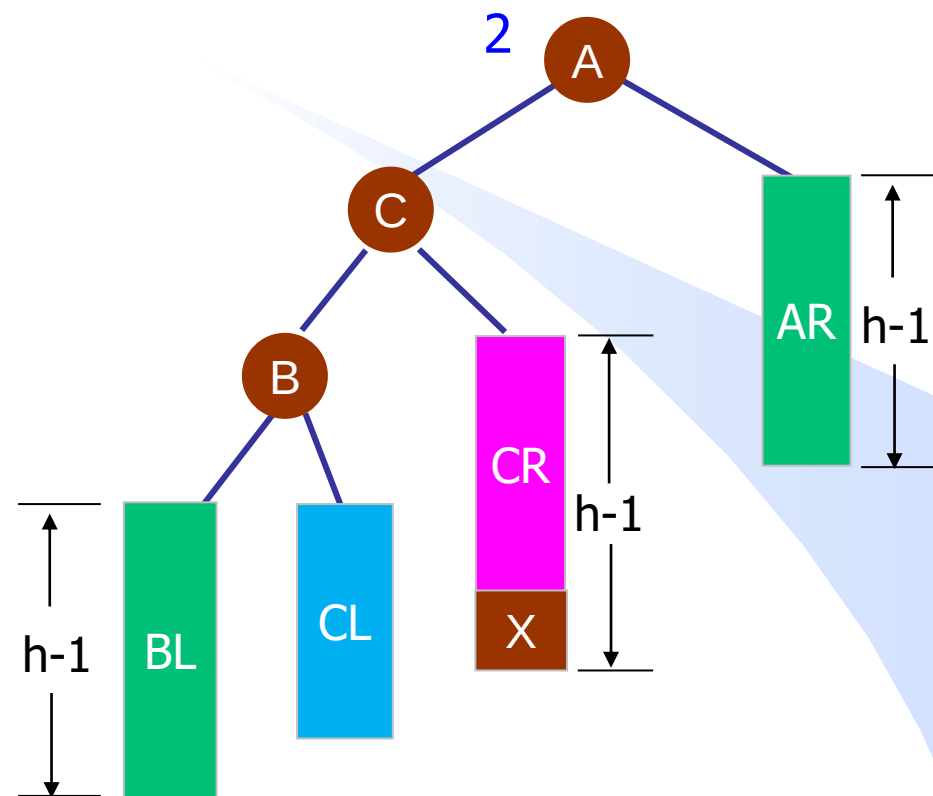
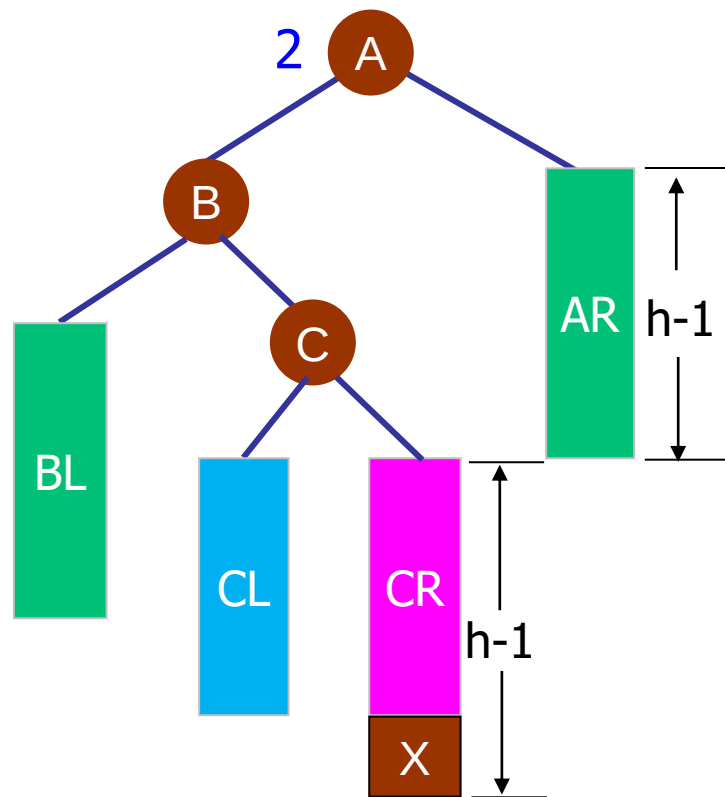
插入X后不再平衡

现在ABC不在一条直线，故我们可以先把ABC变为在一条直线

# 高度平衡树的插入—双旋转

A

## 三、双旋转-LR型的调整过程

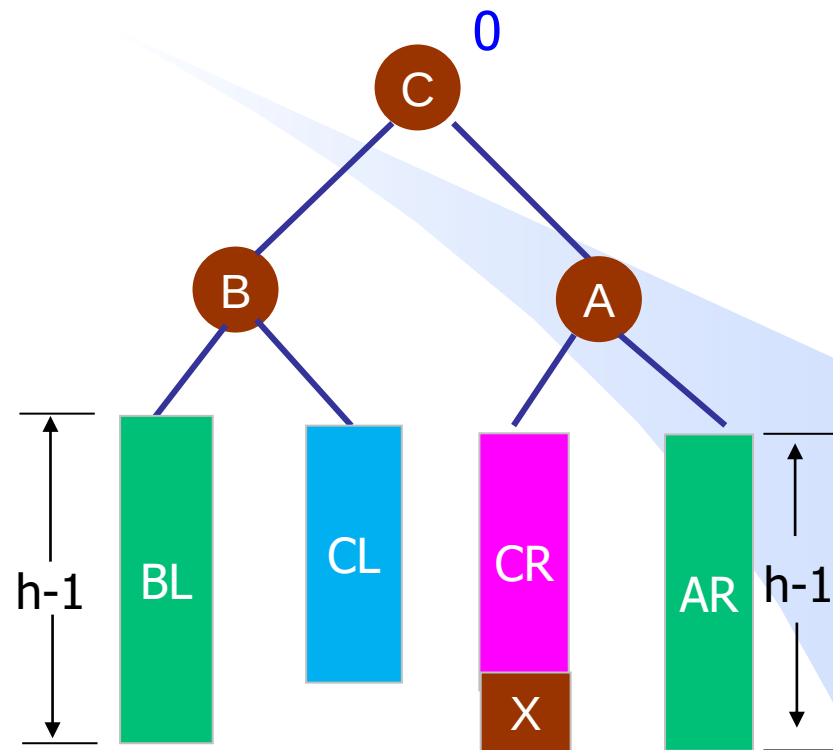
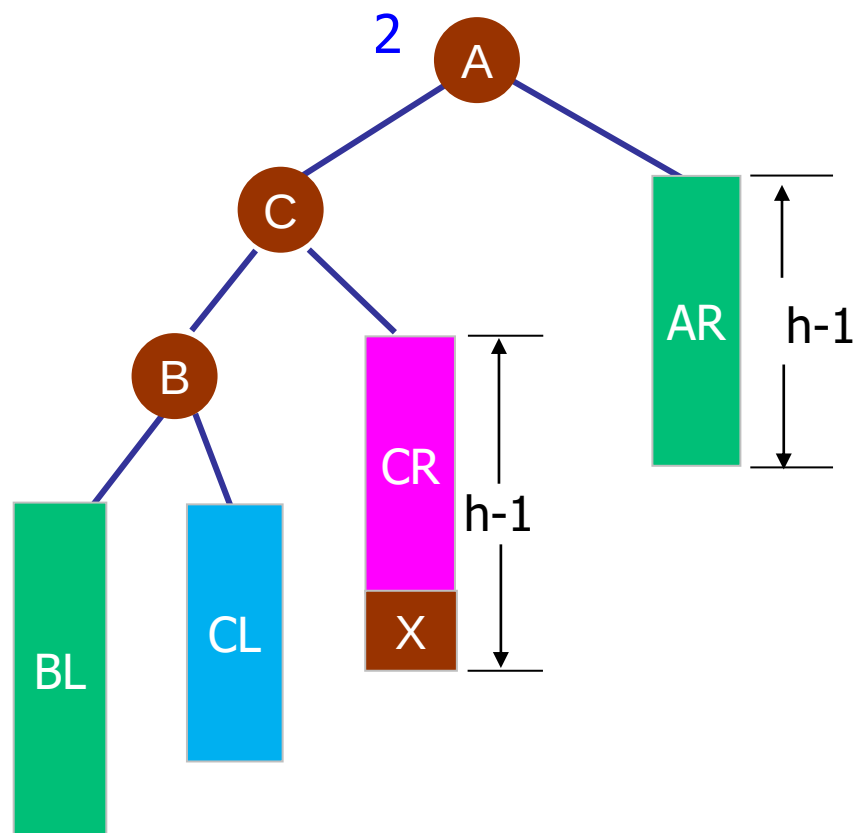


看以B为根的子树，其右子树高，那就**以C为轴**，将B左转。  
旋转后ABC在一条直线上。

# 高度平衡树的插入—双旋转

A

## 三、双旋转-LR型的调整过程



看以A为根的子树，其左子树高，那就**以C为轴**，将A右转。  
旋转后平衡系数为0，高度仍为 $h+1$ 。

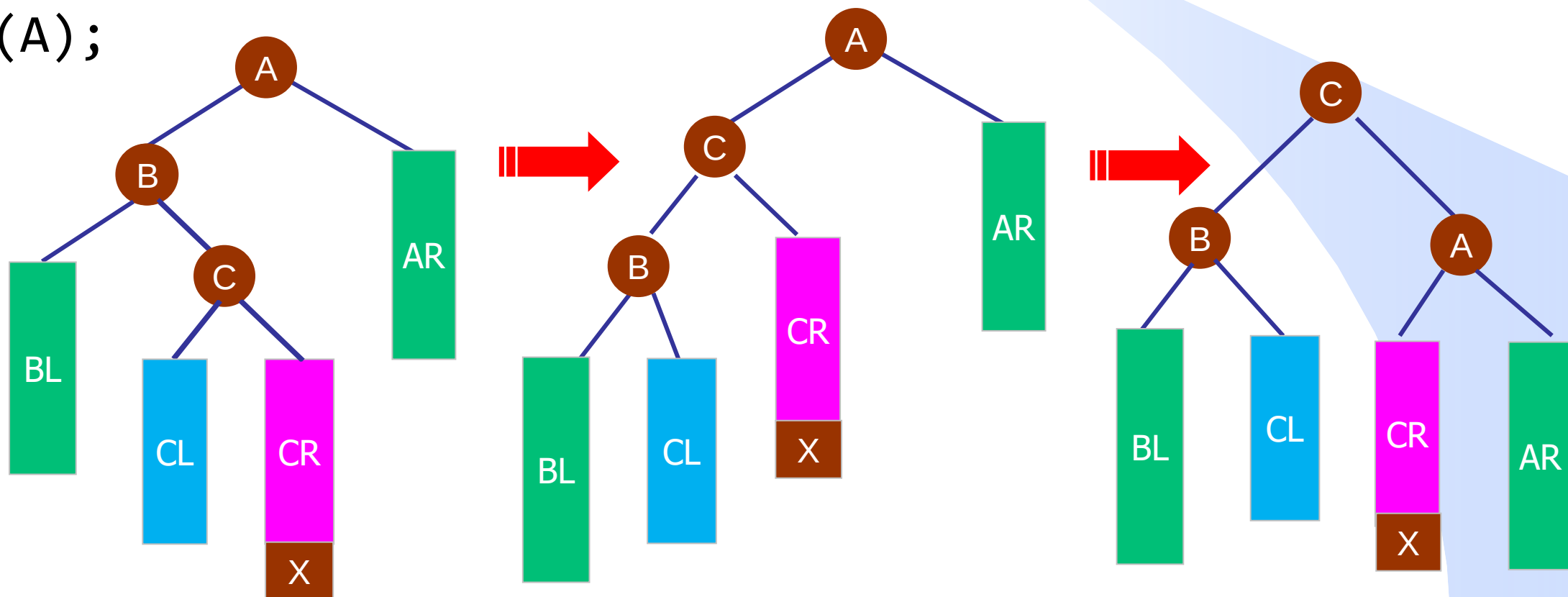
# 高度平衡树的插入—双旋转实现

C

## LR型的调整算法的实现

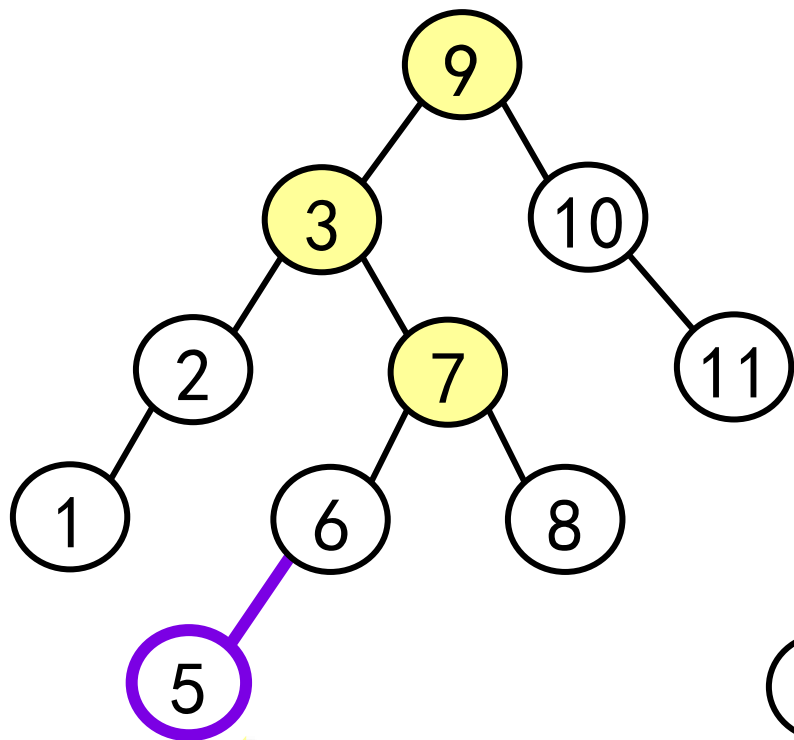
```
void LR(AVLnode* &A) {  
    RR(A->left);  
    LL(A);  
}
```

以C为轴先左转后右转

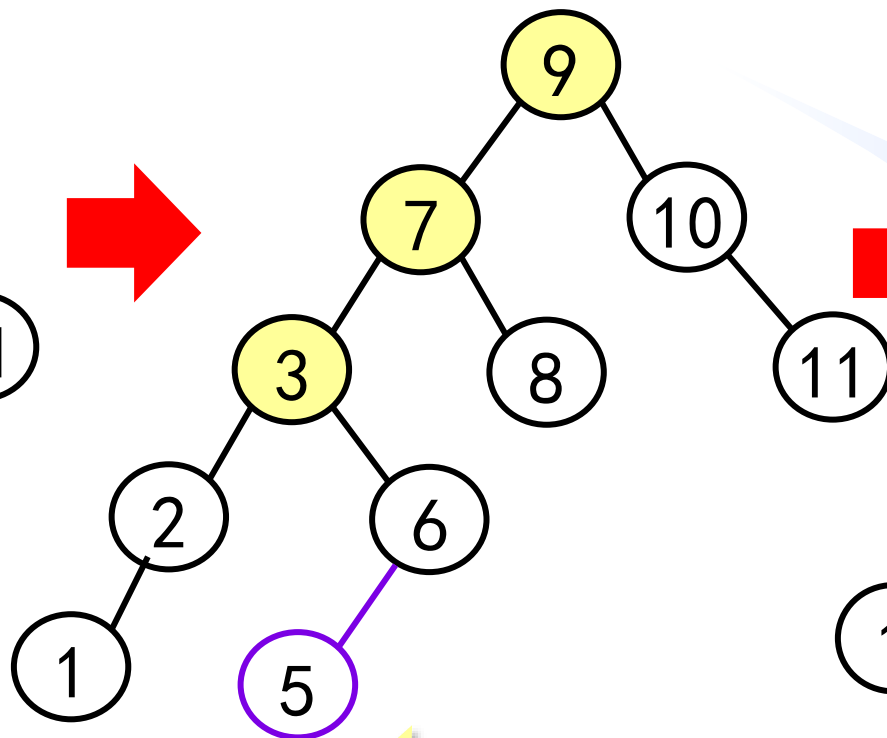


# 练习—LR型举例

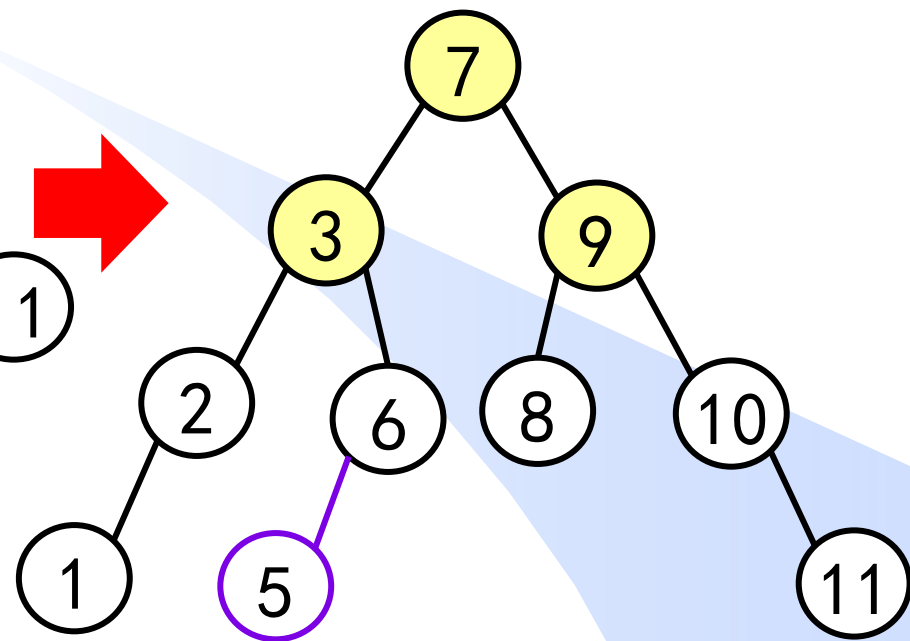
A



插入5



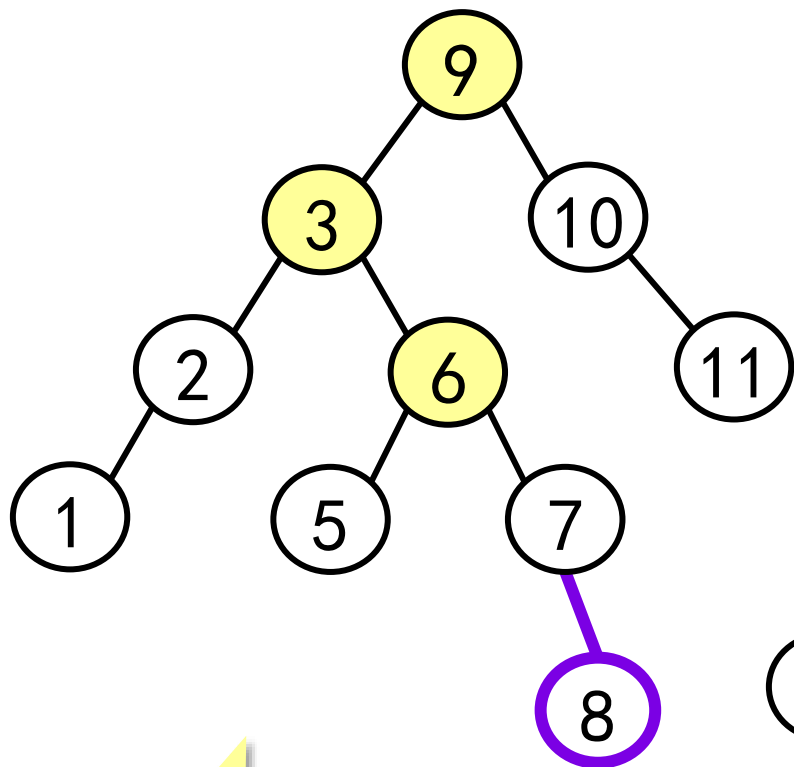
以7为轴将3左转



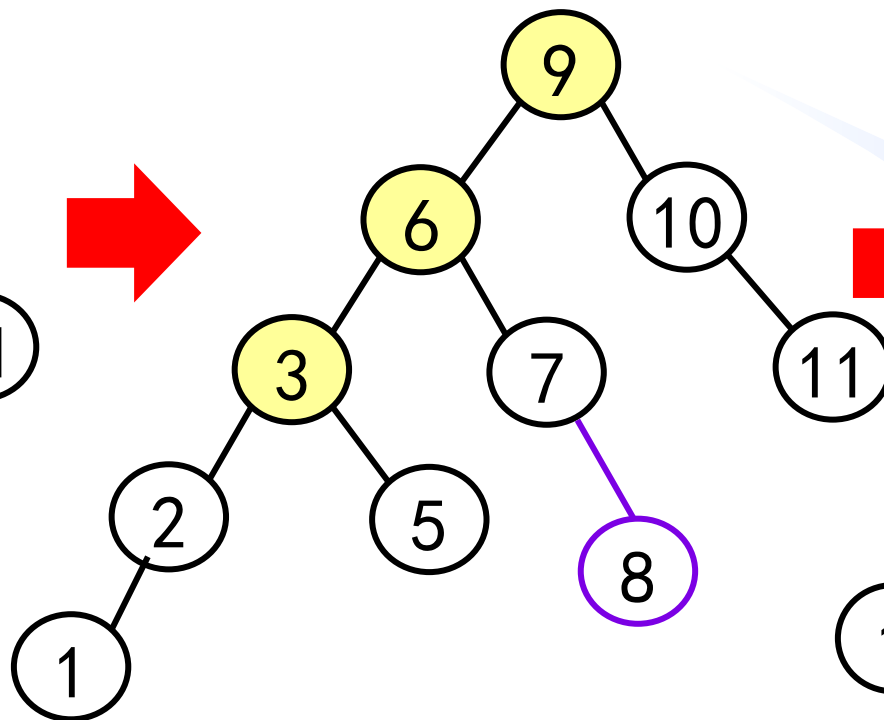
以7为轴将9右转

# 课下思考—LR型举例

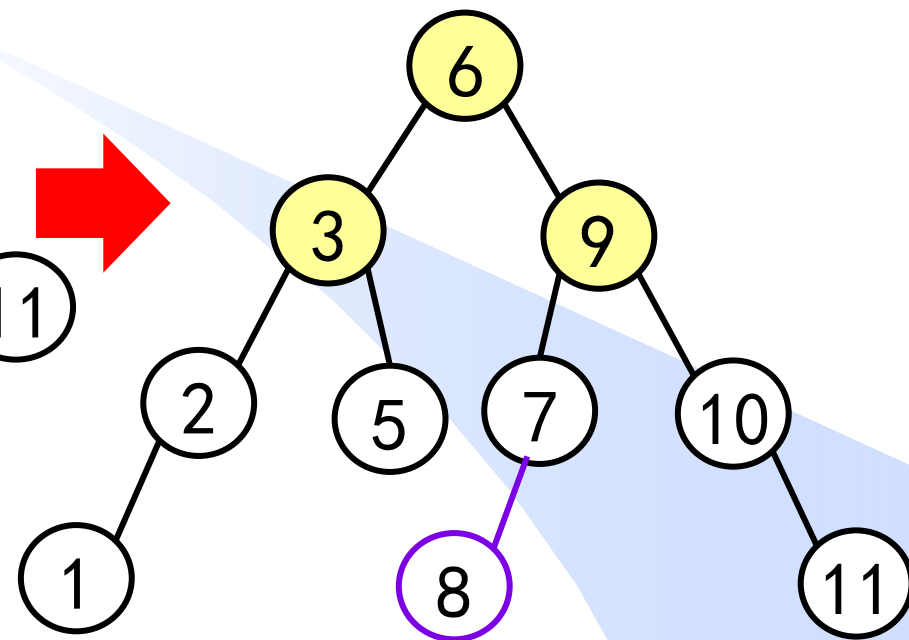
A



插入8



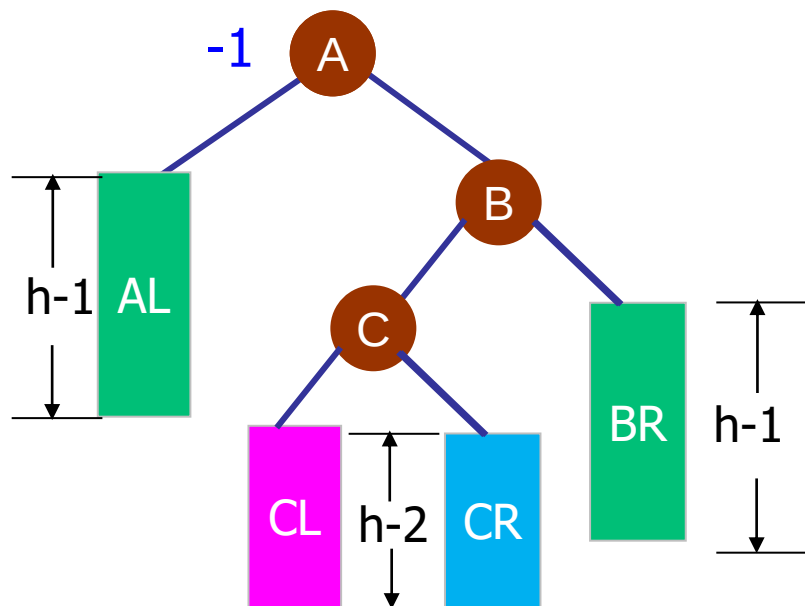
以6为轴将3左转



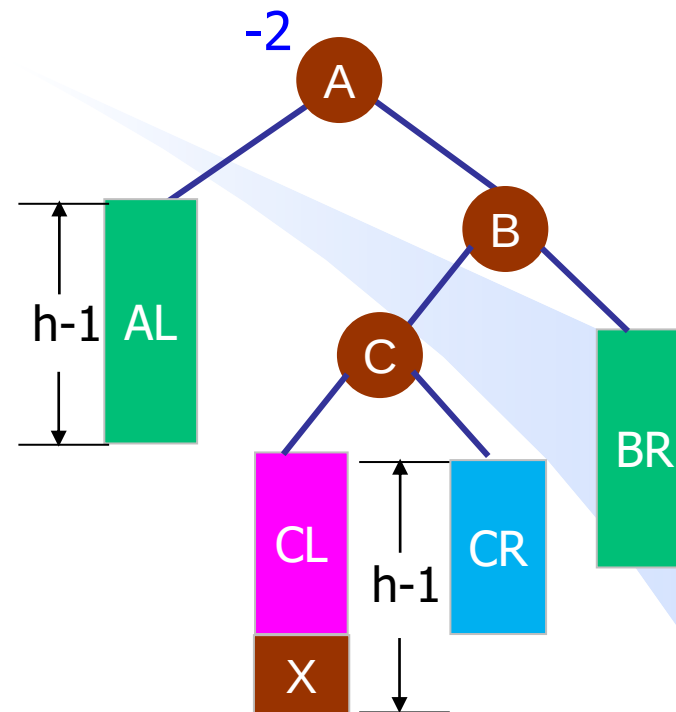
以6为轴将9右转

# 高度平衡树的插入—双旋转

## 四、双旋转-RL型



插入前：高度 $h+1$



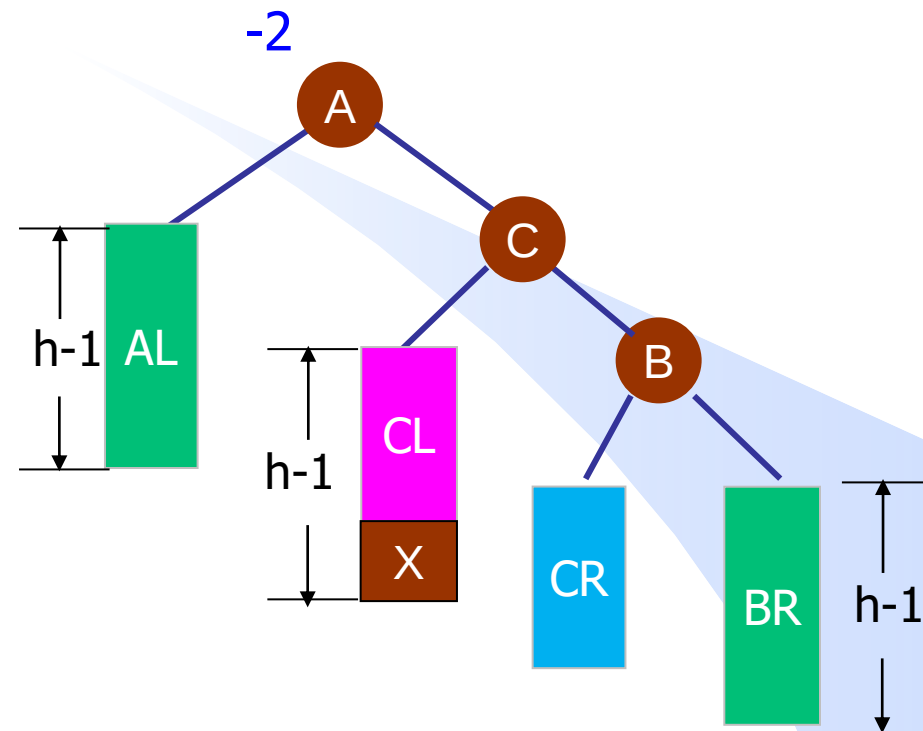
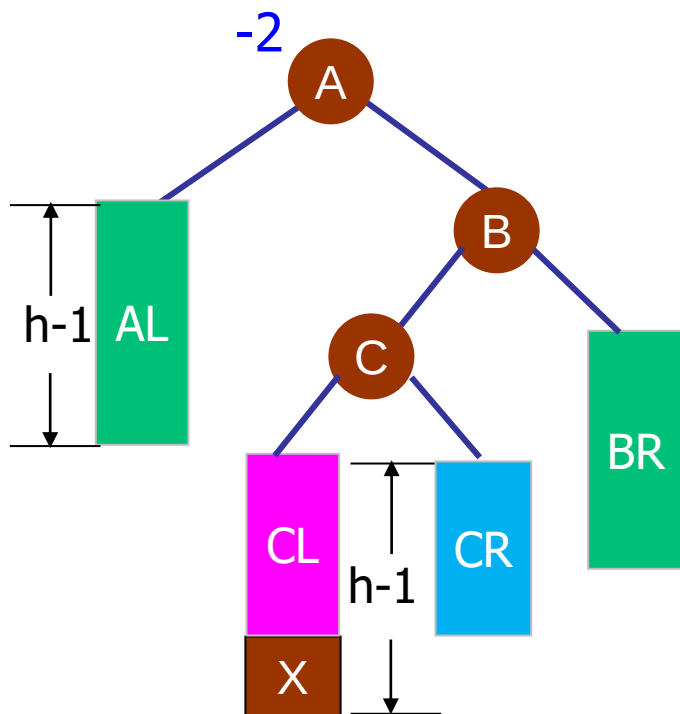
插入 $X$ 后不再平衡

**$ABC$ 不在一条直线，先把 $ABC$ 变为在一条直线**

# 高度平衡树的插入—双旋转

A

## 四、双旋转-RL型的调整过程



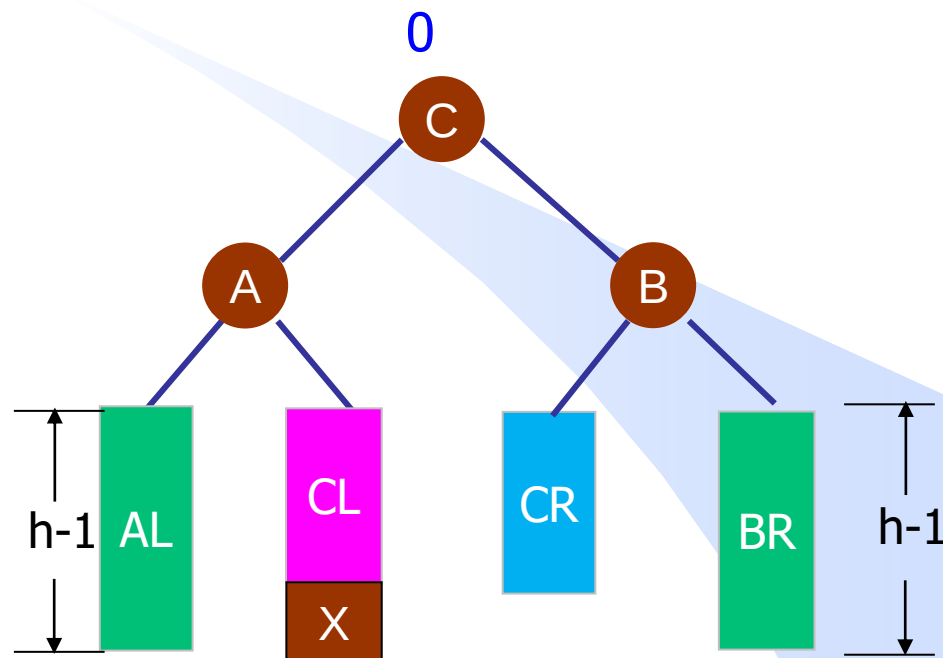
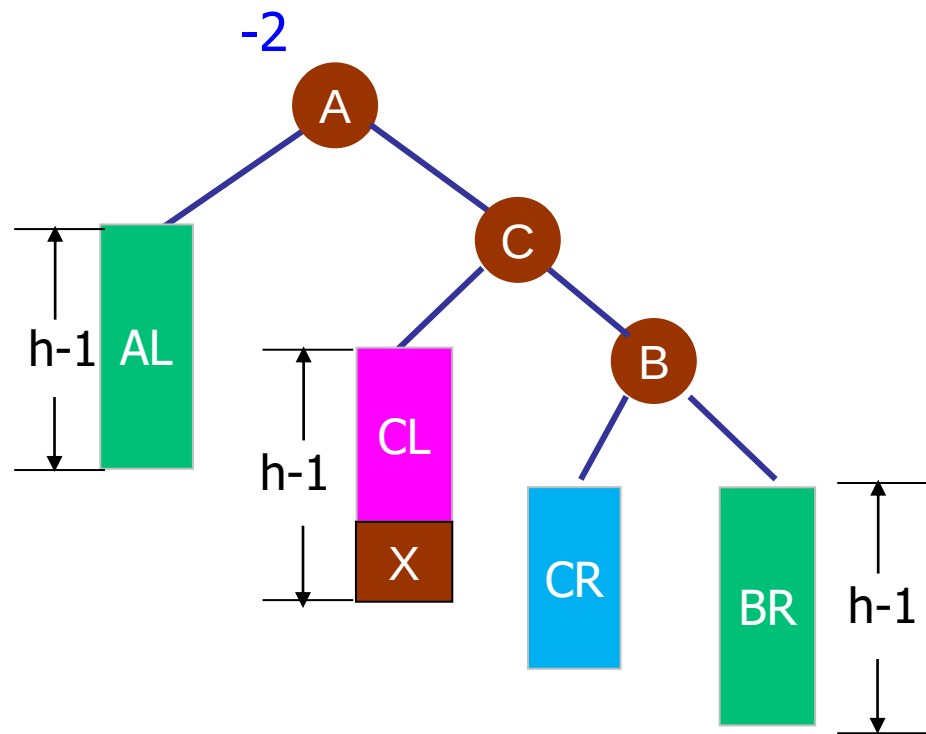
看以B为根的子树，其左子树高，那就以C为轴，将B右转。  
旋转后ABC在一条直线上。



# 高度平衡树的插入—双旋转

A

## 四、双旋转-RL型的调整过程



看以A为根的子树，其右子树高，那就**以C为轴**，将A左转。  
旋转后平衡系数为0，高度为 $h+1$ 。

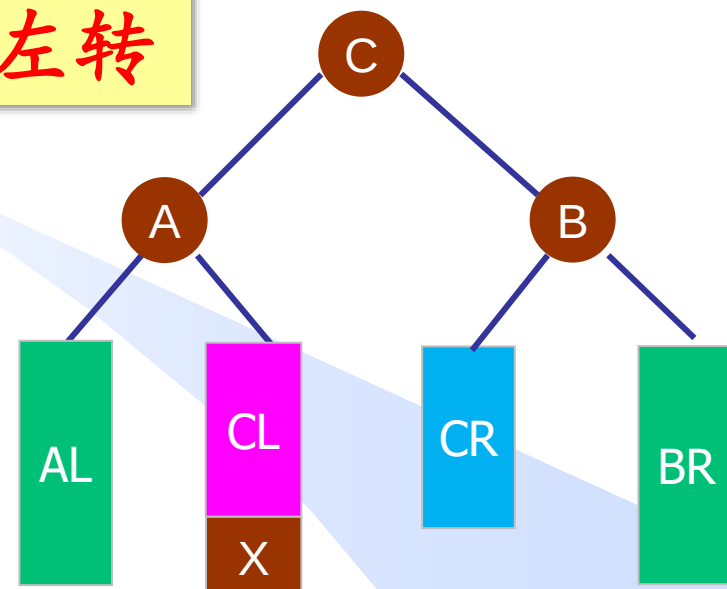
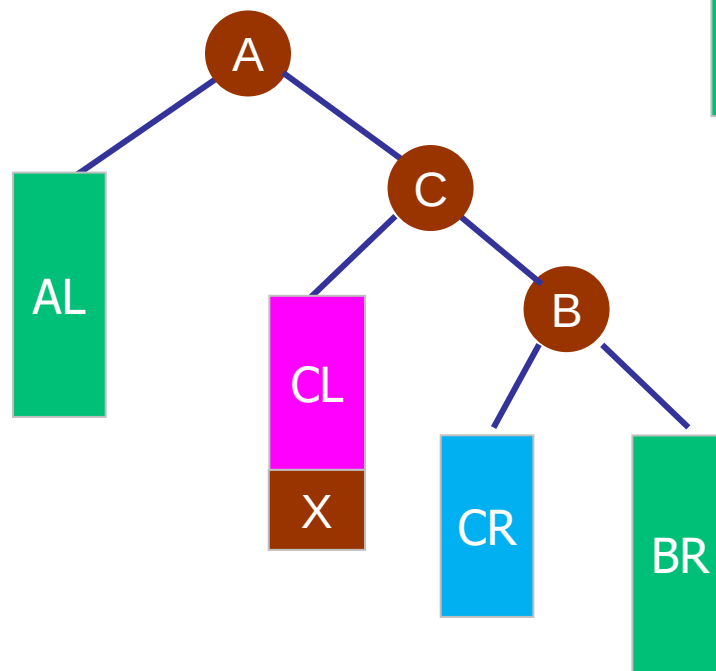
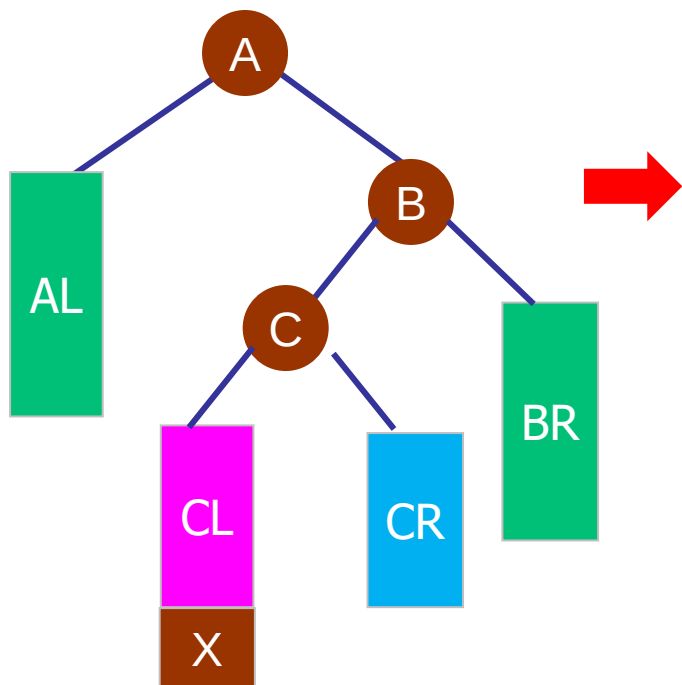
# 高度平衡树的插入—双旋转实现

C

## RL型的调整算法的实现

以C为轴先**右转**后**左转**

```
void RL(AVLnode* &A){  
    LL(A->right);  
    RR(A);  
}
```



## 练习

给定如下AVL树，插入关键字23后，根结点的关键字是\_\_\_\_\_

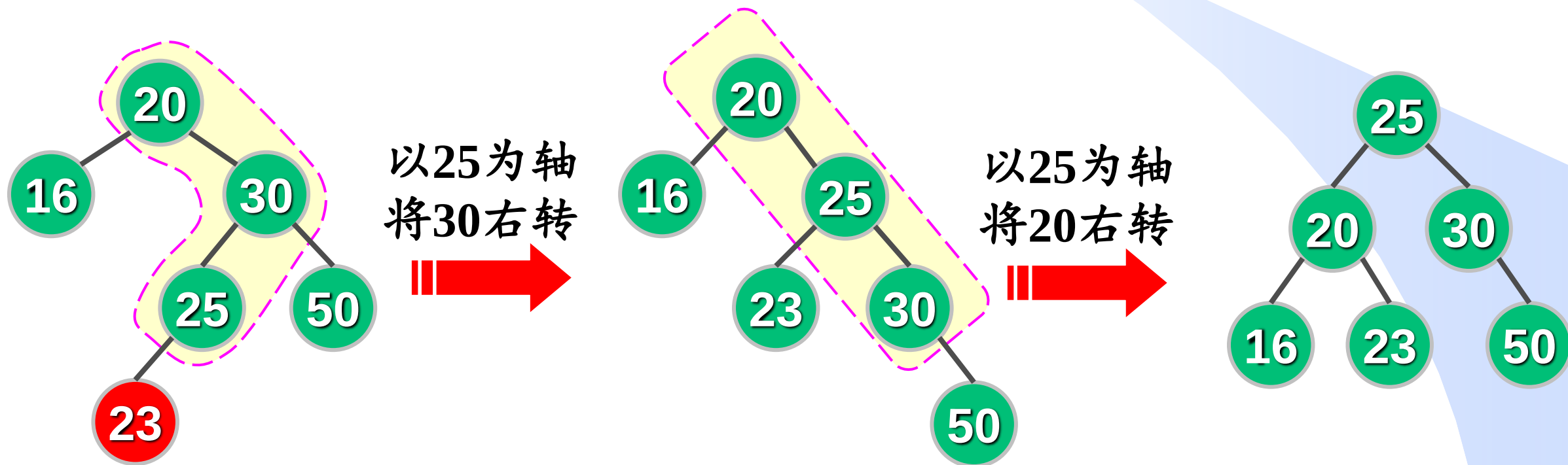
【2021年考研题全国卷，吉林大学21级期末考试题】

A. 16

B. 20

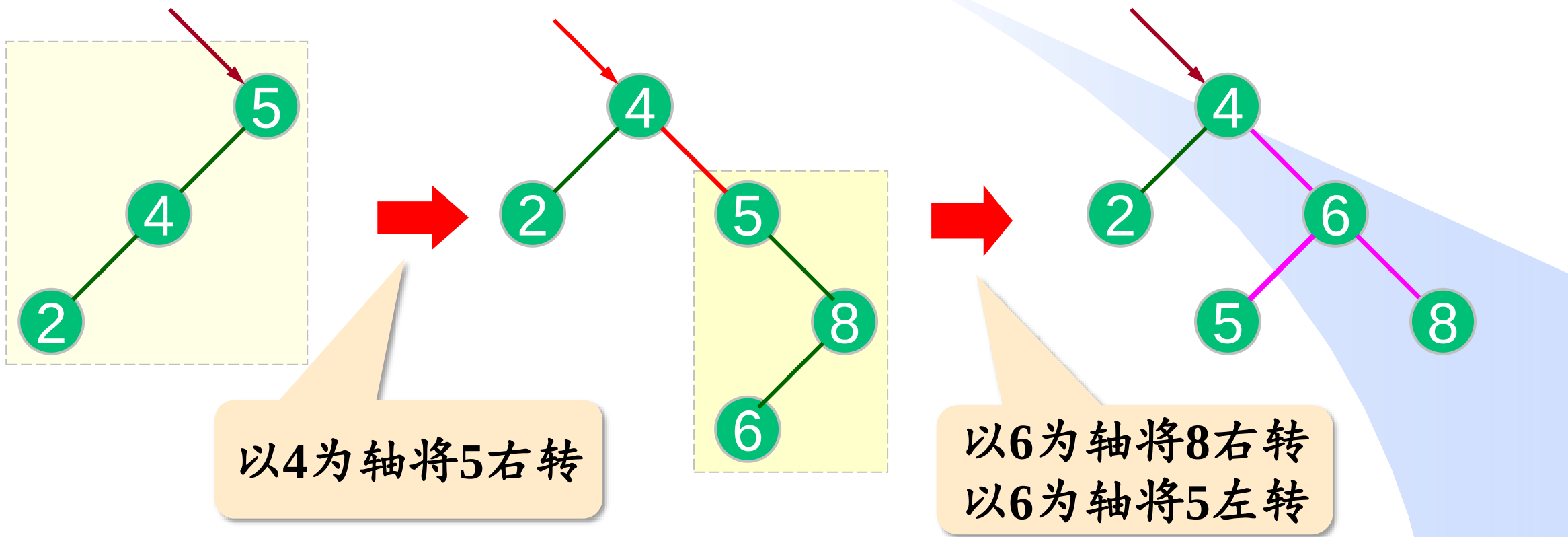
C. 23

D. 25

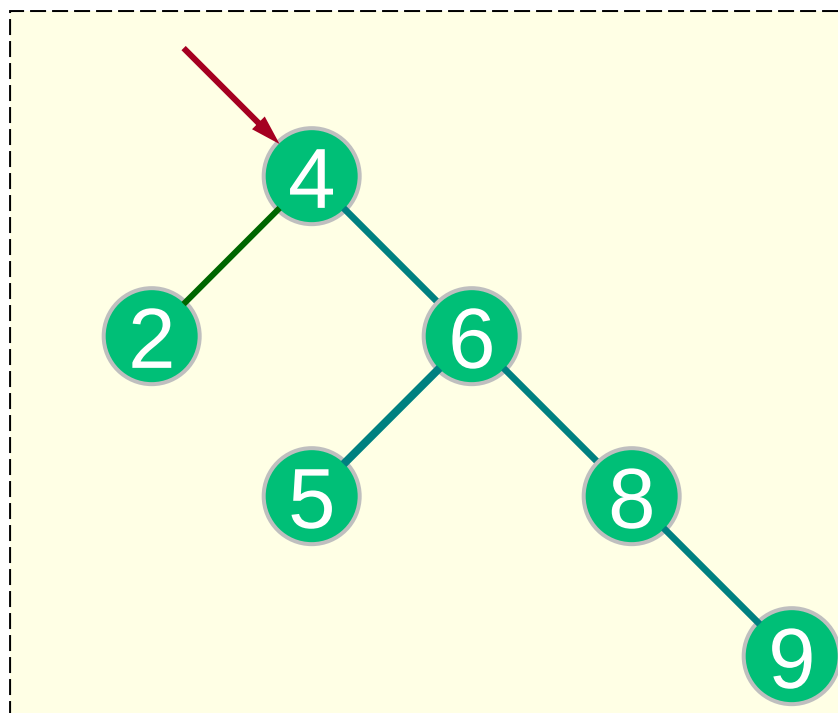


# 课下思考

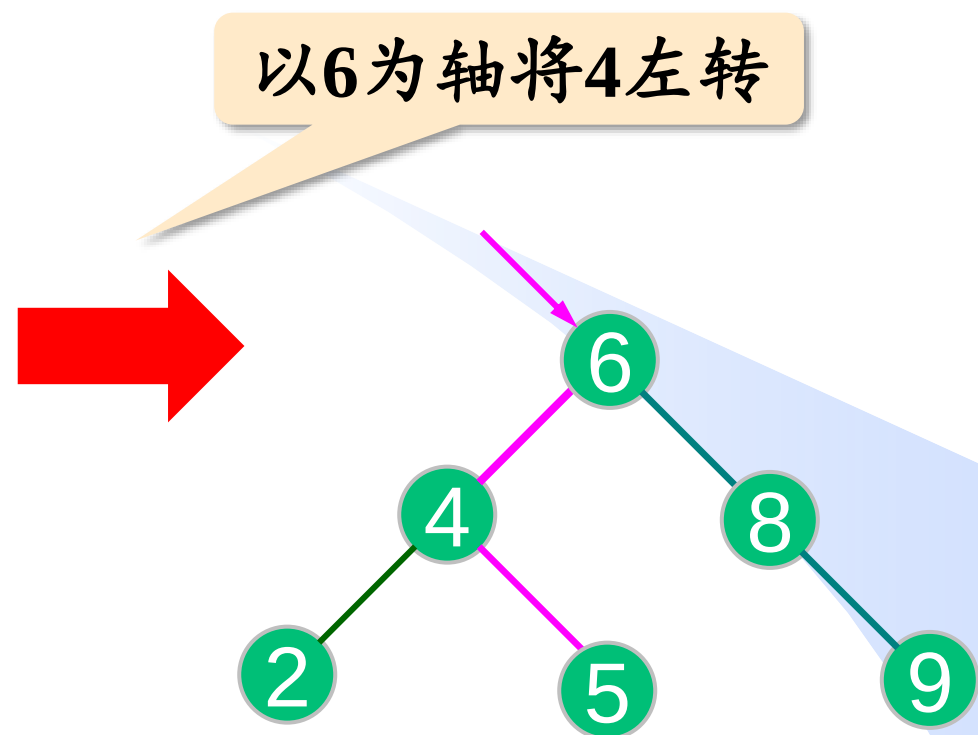
例：依次插入关键字5, 4, 2, 8, 6, 9



## 课下思考

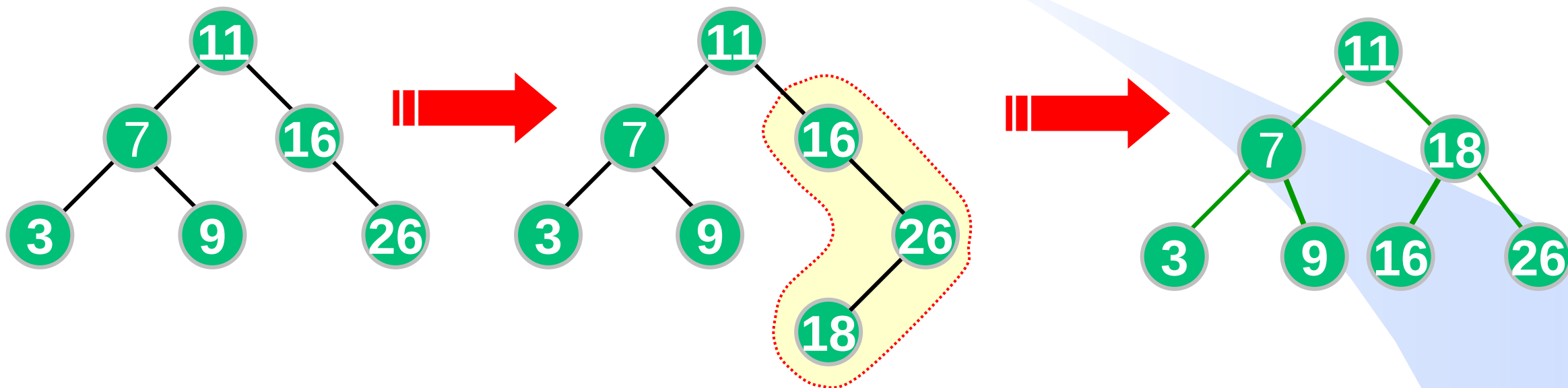


继续插入关键字9



# 课下思考

画出下面高度平衡树插入18后的树形



# AVL树的插入—小结

A

➤ 插入新结点X后，若AVL树失去平衡，应调整失去平衡的最小子树，即找从X到根结点的路径上的第一个失衡结点A，**平衡以A为根的子树。**

➤ 调整策略：

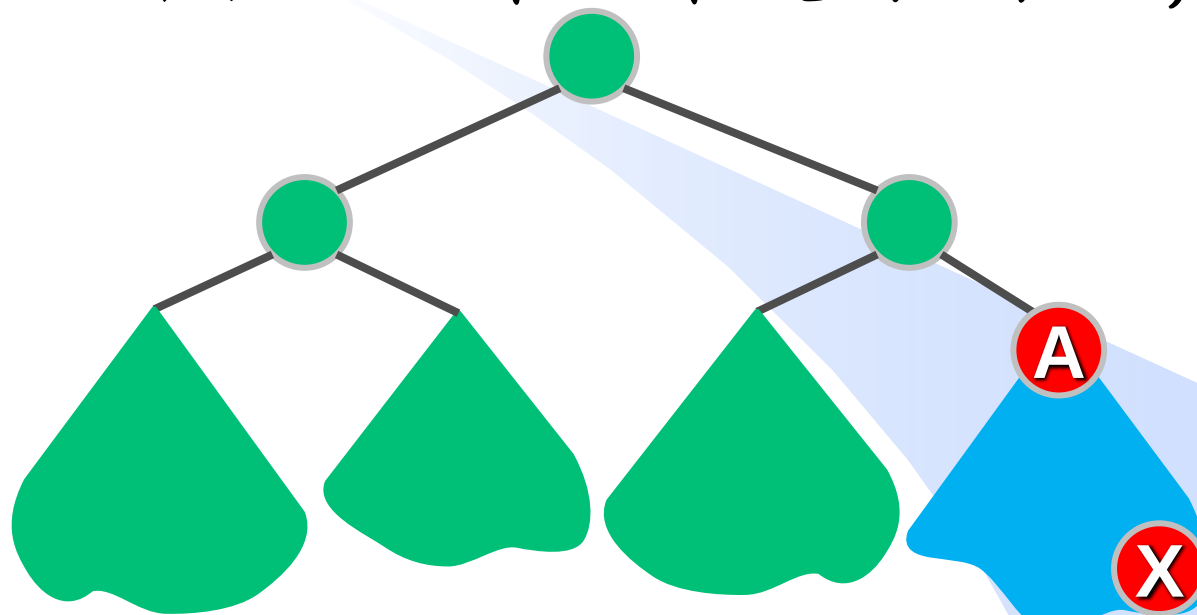
① LL型：右转

② RR型：左转

③ LR型：左转+右转

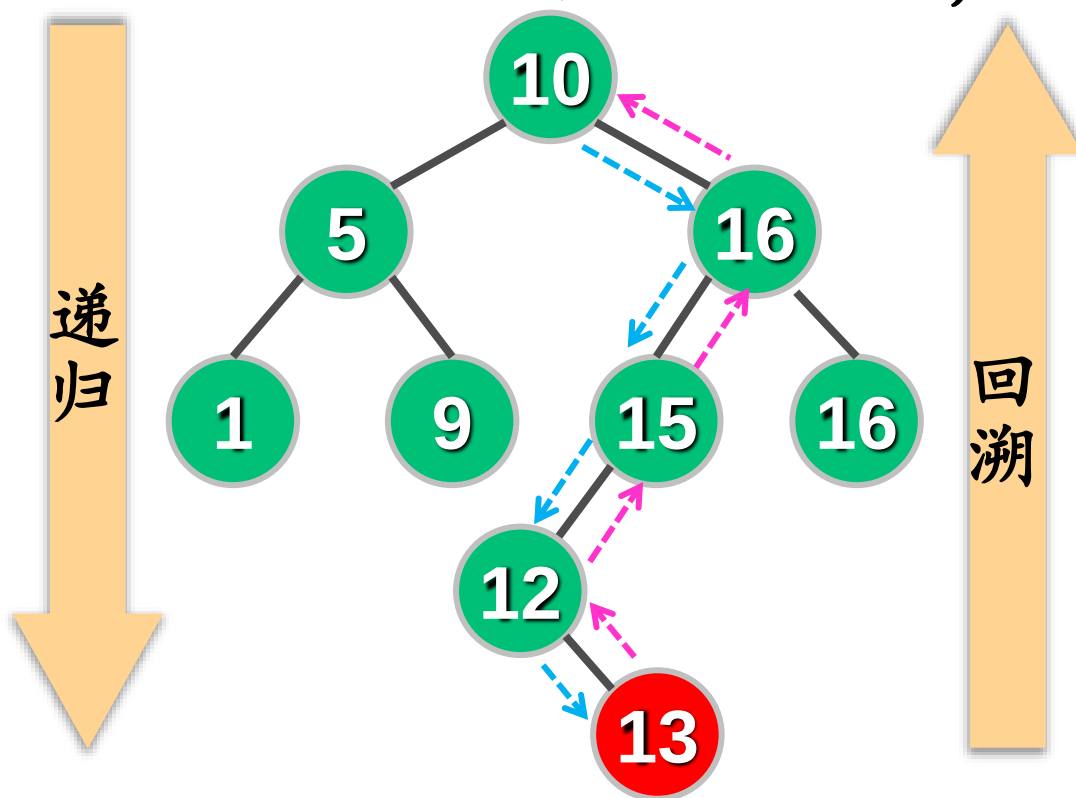
④ RL型：右转+左转

➤ 旋转后，该子树根结点平衡系数变为0，且该子树高度与插入前相同。从而使A子树以外的结点的平衡性不受影响，故插入最多涉及2次旋转。



# AVL树插入算法的实现

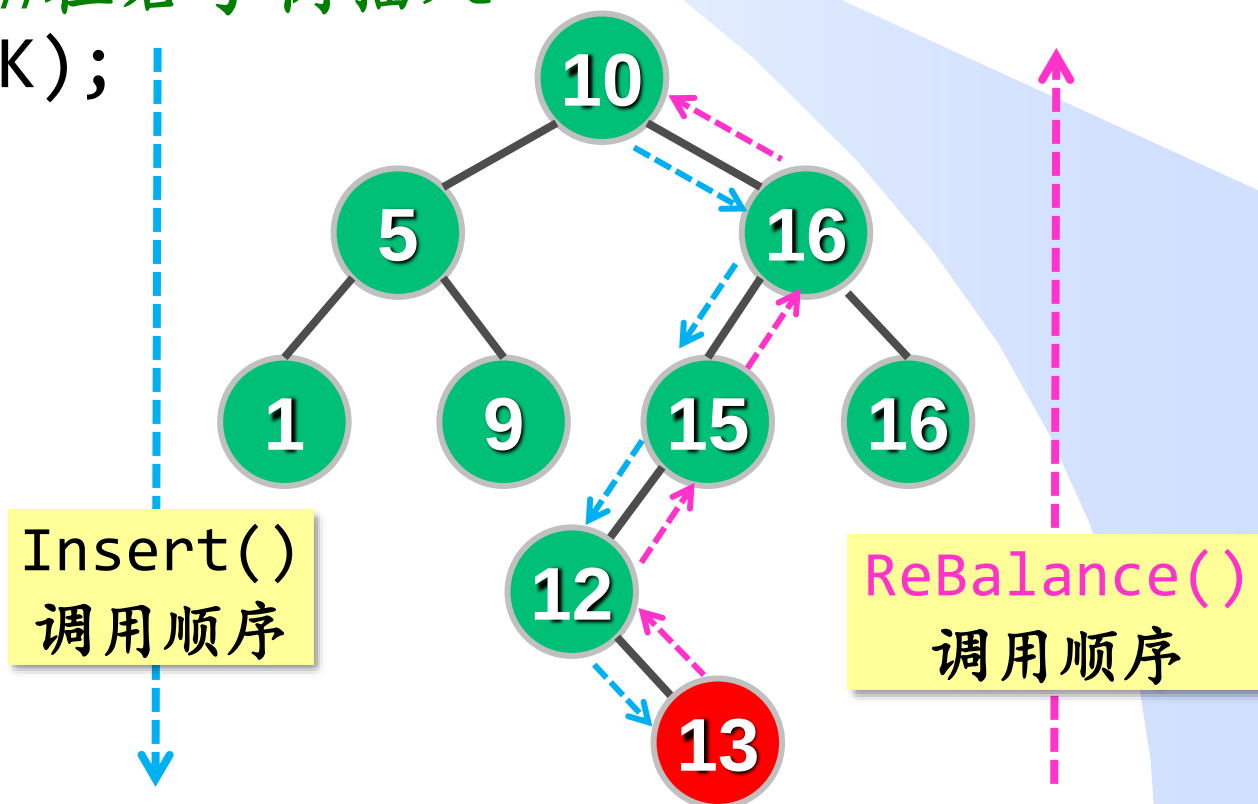
- 如何找从插入点到根结点的路径中第一个失衡结点A?
- 从根到插入点（自顶向下）：通过递归过程实现；
- 从插入点到根（自底向上）：通过递归函数的返回过程（回溯）实现。在上一层递归函数返回后，检查当前结点的平衡性。





# AVL树插入算法的实现

```
void Insert(AVLnode* &root, int K) {  
    if(root==NULL) root=new AVLnode(K);  
    else if(K < root->key) //在左子树插入  
        Insert(root->left, K);  
    else if(K > root->key) //在右子树插入  
        Insert(root->right, K);  
    ReBalance(root);  
}
```



# AVL树插入算法的实现

C

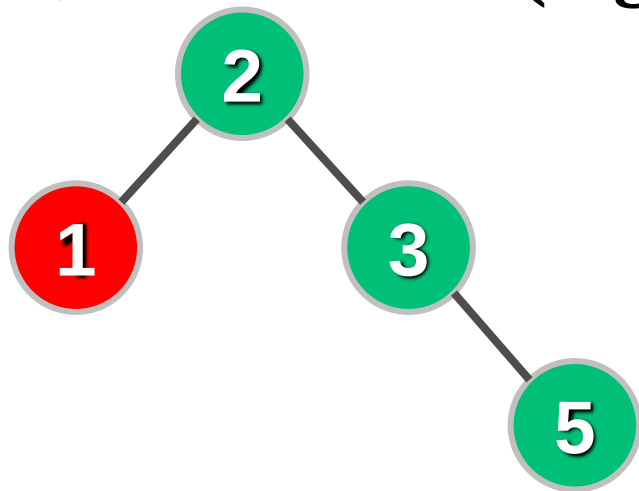
```
void ReBalance(AVLnode* &t) {  
    if(t==NULL) return;  
    if(Height(t->left) - Height(t->right)==2){  
        if(Height(t->left->left) >= Height(t->left->right))  
            LL(t);  
        else  
            LR(t);  
    }else if(Height(t->right) - Height(t->left)==2){  
        if(Height(t->right->right) >= Height(t->right->left))  
            RR(t);  
        else  
            RL(t);  
    }  
    UpdateHeight(t);  
}
```

The diagrams illustrate four balance cases for an AVL tree:

- LL (Left-Left):** Root  $t$  has balance factor  $L$ . Its left child has balance factor  $L$ . The node  $K$  is the left child of the left child.
- LR (Left-Right):** Root  $t$  has balance factor  $L$ . Its left child has balance factor  $R$ . The node  $K$  is the right child of the left child.
- RR (Right-Right):** Root  $t$  has balance factor  $R$ . Its right child has balance factor  $R$ . The node  $K$  is the right child of the right child.
- RL (Right-Left):** Root  $t$  has balance factor  $R$ . Its right child has balance factor  $L$ . The node  $K$  is the left child of the right child.

# AVL树的删除

- 先采用二叉查找树的删除算法进行删除。
- 删除某结点X后，沿**实际删除点**到**根结点**的路径，向上找第一个不平衡点为A，平衡以A为根的子树。
- 平衡后，可能使子树A高度变小。这样可能导致A的父结点不满足平衡性。
- 所以要继续向上考察结点的平衡性，最远可能至根结点，即最多需要做 $O(\log n)$ 次旋转。



对比“插入”：平衡A后，子树**高度不变**，A子树以外的结点不受影响，故插入最多涉及 $O(1)$ 次旋转。

# AVL树删除算法的实现

```

void remove(AVLnode* &root, int K) {
    if(root==NULL) return;
    if(K<root->key) remove(root->left, K);           //在左子树删K
    else if(K>root->key) remove(root->right, K);      //在右子树删K
    else if(root->left!=NULL && root->right!=NULL){
        AVLnode *s=root->right;
        while(s->left!=NULL) s=s->left;
        root->key=s->key;           //s为右子树中根序列第一个结点
        remove(root->right, s->key);
    }else{
        AVLnode* oldroot=root;
        root=(root->left!=NULL)? root->left:root->right;
        delete oldroot;
    }
    ReBalance(root);
}

```

*K==root->key*

# 高度平衡树总结

- AVL树的高度为 $O(\log n)$ ，因此使插入、删除、查找的最坏时间复杂度均为 $O(\log n)$ 。
- 删除操作**最坏**情况下需要做 $O(\log n)$ 次旋转。
- 对任意连续多次删除操作，每次删除所需的均摊旋转次数为 $O(1)^{[1]}$ 。对于任意多次插入和删除的混合序列，存在精心构造出的特定操作序列，使每次删除所需的均摊旋转次数为 $O(\log n)^{[2]}$ 。

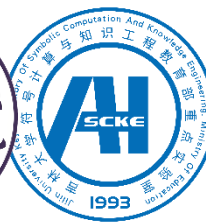
[1] Tsakalidis. Rebalancing operations for deletions in AVL-trees. *RAIRO Informatique Théorique*. 19(4): 323-329. 1985.

[2] Mahdi Amani, Kevin Lai, Robert Tarjan. Amortized rotation cost in AVL trees. *Information Processing Letters*. 116(5):327-330. 2016.





# AC MILAN



## 二叉查找树

---

- 二叉查找树
- 高度平衡树
- **红黑树简介**
- 伸展树

# 红黑树 (Red Black Tree)



**Rudolf Bayer**  
慕尼黑工业大学教授



**Leonidas J. Guibas**  
斯坦福大学教授  
美国科学院院士  
美国工程院院士



**Robert Sedgwick**  
普林斯顿大学教授

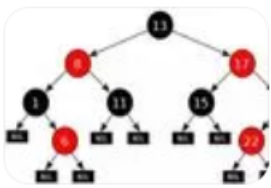
- [1] Rudolf Bayer. Symmetric Binary B-Trees: Data Structures and Maintenance Algorithms. *Acta Informatica*, 1:290-306, 1972.
- [2] Leo J. Guibas, Robert Sedgwick. A Dichromatic Framework for Balanced Trees. *Proceedings of the 19th Annual Symposium on Foundations of Computer Science*, pp.8-21, IEEE Computer Society, 1978.



# 红黑树 (Red Black Tree)

- 2022年起，全国考研大纲新增“红黑树”。
- 腾讯、阿里、华为、字节跳动、京东、百度、美团等各大公司面试常考题目。

面试阿里,字节跳动,美团必被问到的红黑树



2020年5月21日 因为要满足节点,那就违反了性点,那就只有在要插, 博客园 百度快照

美团面试被问“红黑树”,我一脸懵逼  
2020年7月22日 美团面  
ee)是一种自平衡的二叉  
前也叫做平衡二叉B树  
CSDN技术社区

京东内部面试题:Mysql索引优化,索引数据结构红黑树.



2021年10月10日 京东内部面试题:Mysql索引优化,ash,B+树详解腾讯内容

红黑树面试题目---华为C++工程师

美团面试:熟悉红黑树?能不能手写一下?

知乎

首发于  
帅地玩编程



2021年6月7日 美团面试:熟悉红  
nbianshu.github.io 图片 手写红  
正常,只要理解就行 红黑树是数  
搜狐网 百度快照

记一次腾讯面试: 有了二叉查找树、平衡树 (AVL) 为啥还需要红黑树?

阿里员工吐槽: 手撕AVL和红黑树, 字节跳动面试真的太无聊!



# 红黑树 (Red Black Tree)

C

➤ 在AVL树基础上进一步放宽平衡条件。

➤ 红黑树是具有如下特点的**二叉查找树**：

① 每个结点是**红色**或**黑色**的；非红即黑

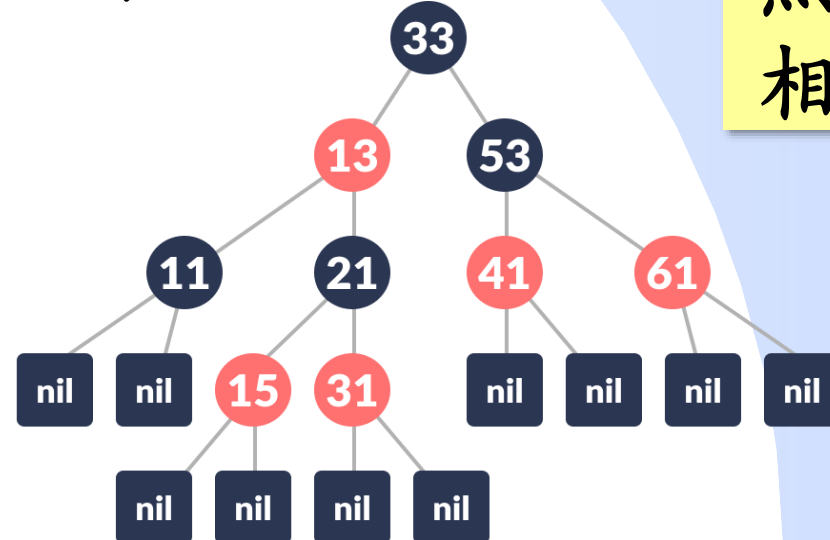
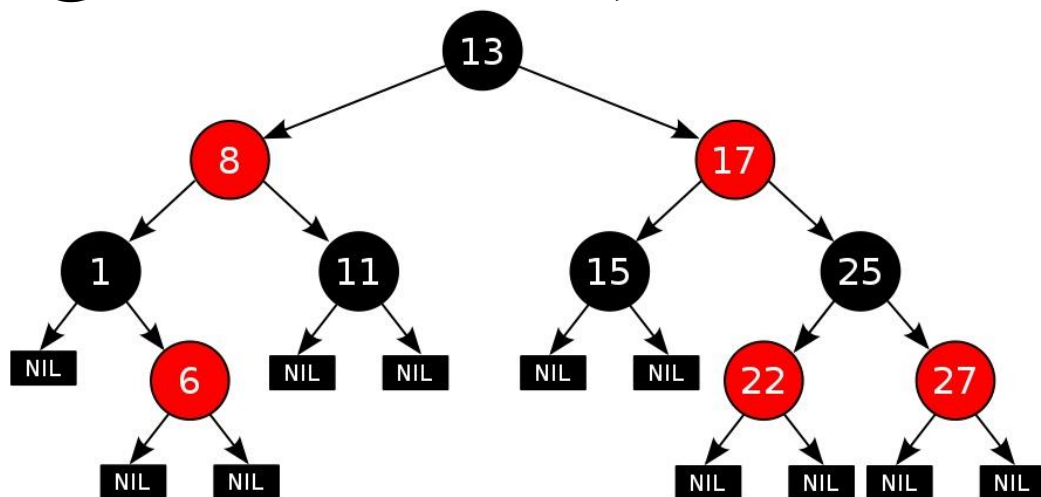
② 根结点为黑色；黑根

③ 外结点为黑色；黑外

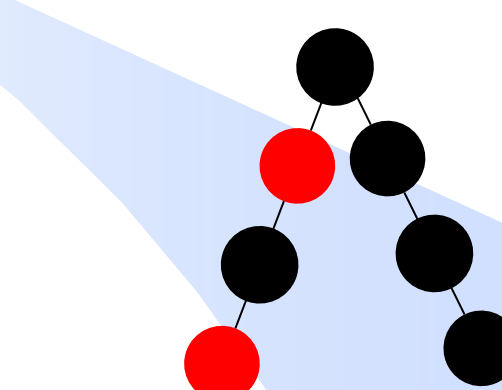
④ 如果一个结点是红色，那么它的孩子必须是黑色。红父黑子

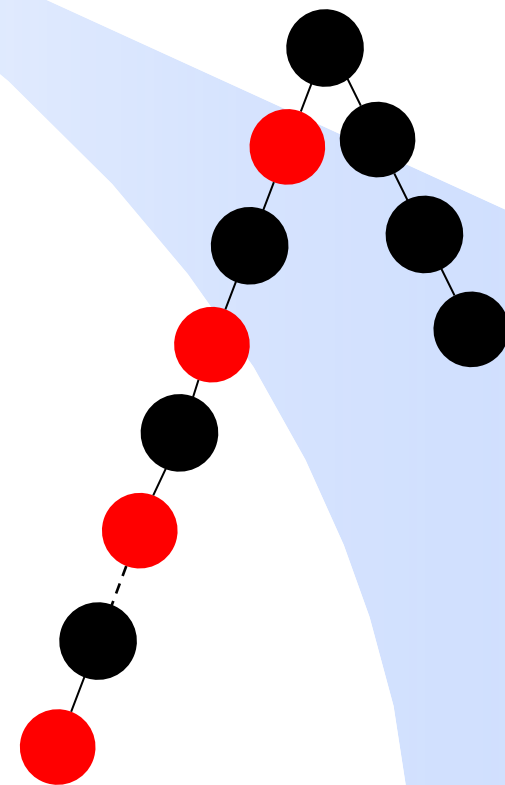
⑤ 任一结点到外结点的路径上，包含相同数目的黑结点。黑高相等

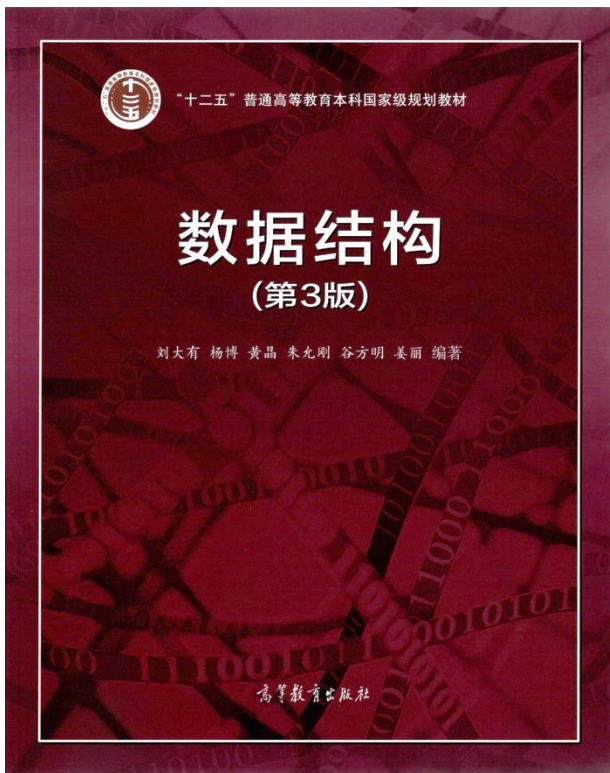
结点的黑高度：该结点到外结点的路径上包含的黑结点数目



# 红黑树 (Red Black Tree)

- 若忽略红结点而只考虑黑结点，则这棵树是平衡的；
  - 任何一条路径上不能有两个连续的红结点。从任意结点出发的最长路径（红黑结点间隔组成）的长度是最短路径（仅由黑结点组成）的2倍；
  - 任何一个结点的左右子树的高度最多相差2倍；
  - $n$ 个内结点的红黑树的高度 $h$ 满足  $h \leq 2\log n$ ；
  - 查找、插入、删除操作的最坏时间复杂度是  $O(\log n)$ ，且最多涉及3次旋转。
  - 与AVL树相比，红黑树平衡性略弱，故查找效率略低。但插入、删除引起失衡的概率也较小，故插入删除效率更高，维持平衡的成本更低。
- 





## 二叉查找树

- 二叉查找树
- 高度平衡树
- 红黑树简介
- **伸展树**

数据之法  
结构之美  
算法之道



# 伸展树 (Splay树)



**Robert Tarjan**

普林斯顿大学教授

图灵奖获得者

美国科学院/工程院院士



**Daniel Sleator**

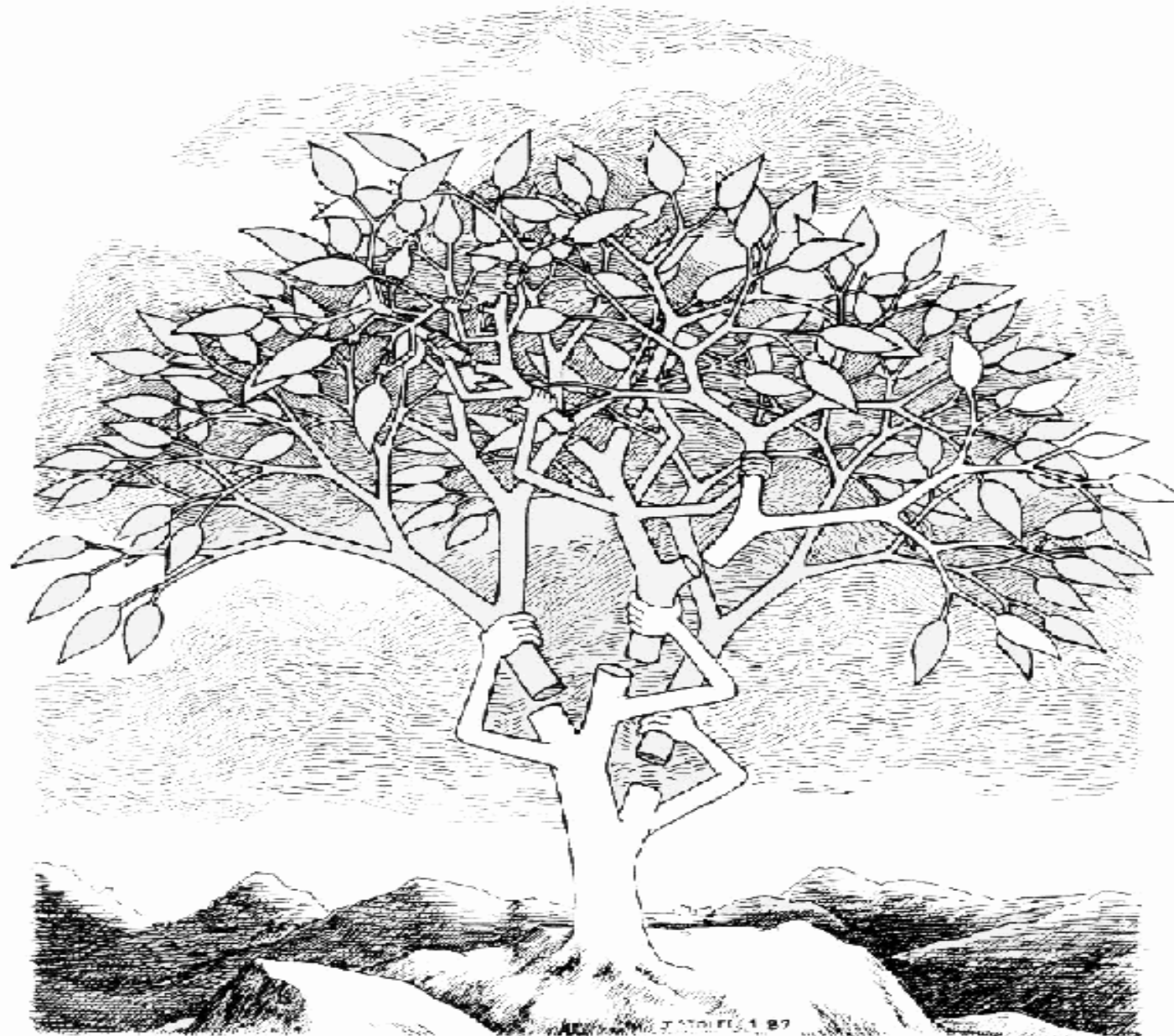
卡内基梅隆大学教授

In a splay tree, each retrieval of an item is followed by a restructuring of the tree, called a splay operation. (Splay, as a verb, means “to spread out.”)

*A splay moves the retrieved node to the root of the tree, approximately halves the depth of all nodes accessed during the retrieval.*

——Robert Tarjan

# 伸展树 (Splay树)



# 伸展树 (Splay树)

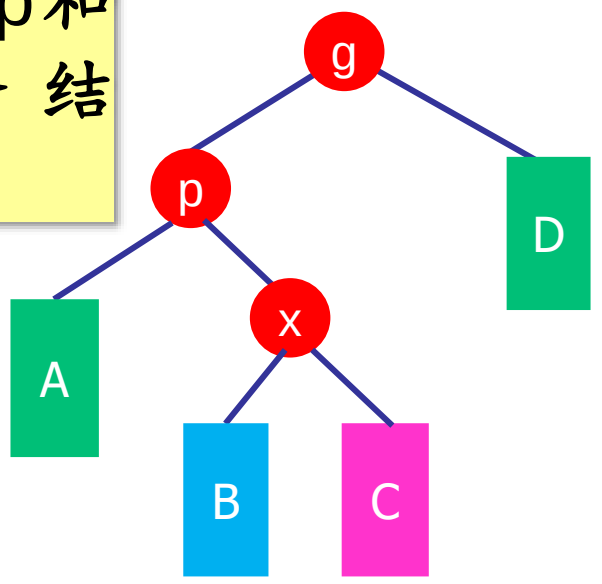
- **时间局部性假设：**刚被访问的数据极有可能很快地被再次访问。
- **动机：**不严格限制查找树的形状，而是假设数据访问具有局部性，让数据在访问后不久再次访问时变快。并不一味追求平衡，而是追求整体的访问效率。
- **目标：**经常被访问的结点放在靠近根的地方。
- **策略：**每当访问一个结点X时，在保持二叉查找树有序性的前提下，通过一系列旋转操作将X调整至树的根部，该过程称为伸展 (Splay) 操作。
- **性能：**Splay树各操作的均摊时间复杂度是 $O(\log n)$ 。



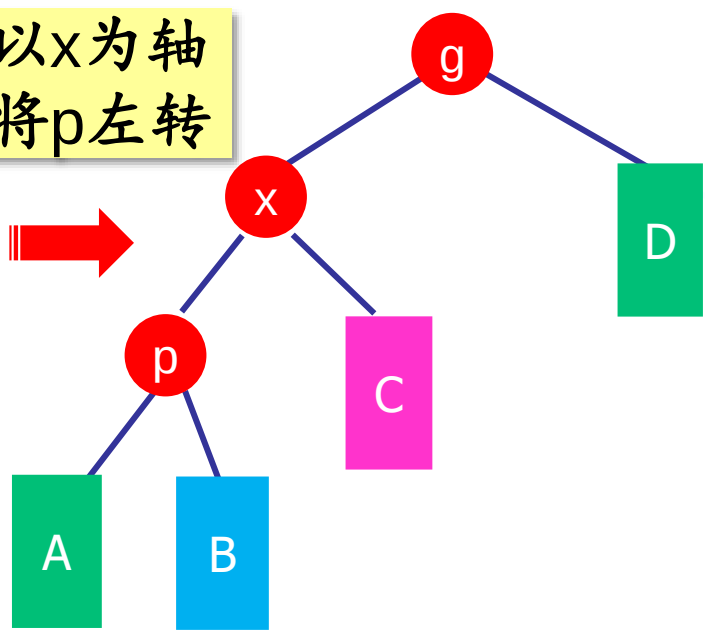
与AVL树LR和RL处理方案一致

# Splay操作将结点x旋转至根——情况1: LR/RL型

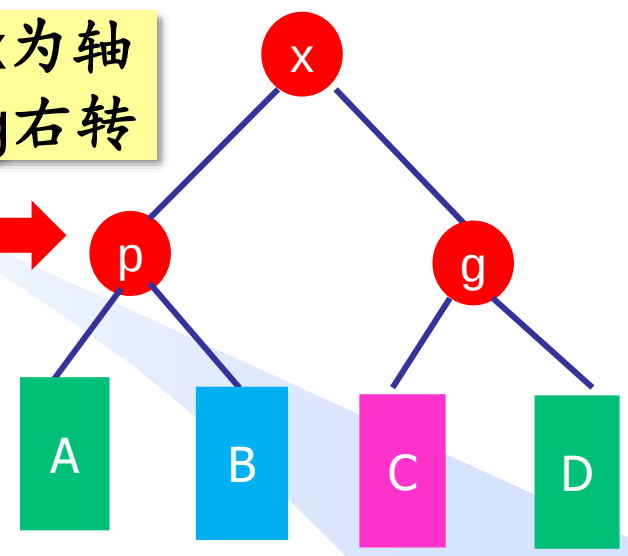
对x取父  
结点p和  
爷爷结  
点g



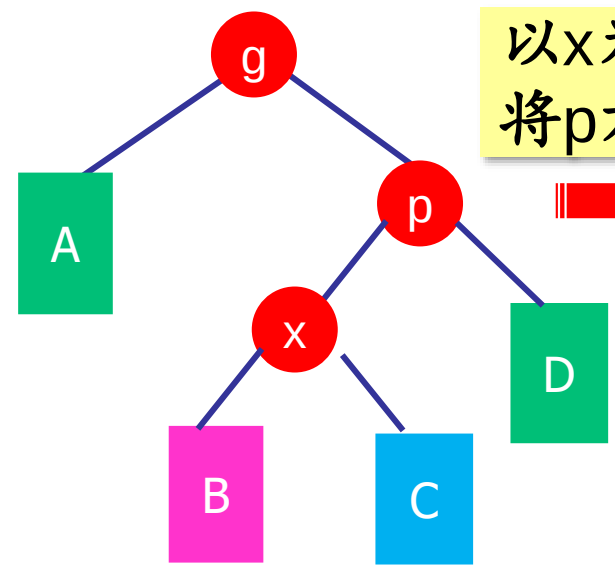
以x为轴  
将p左转



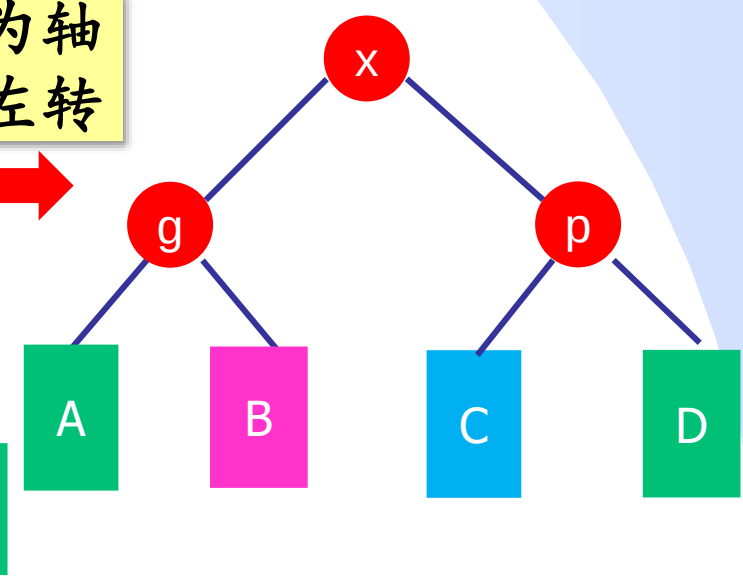
以x为轴  
将g右转



以x为轴  
将p右转

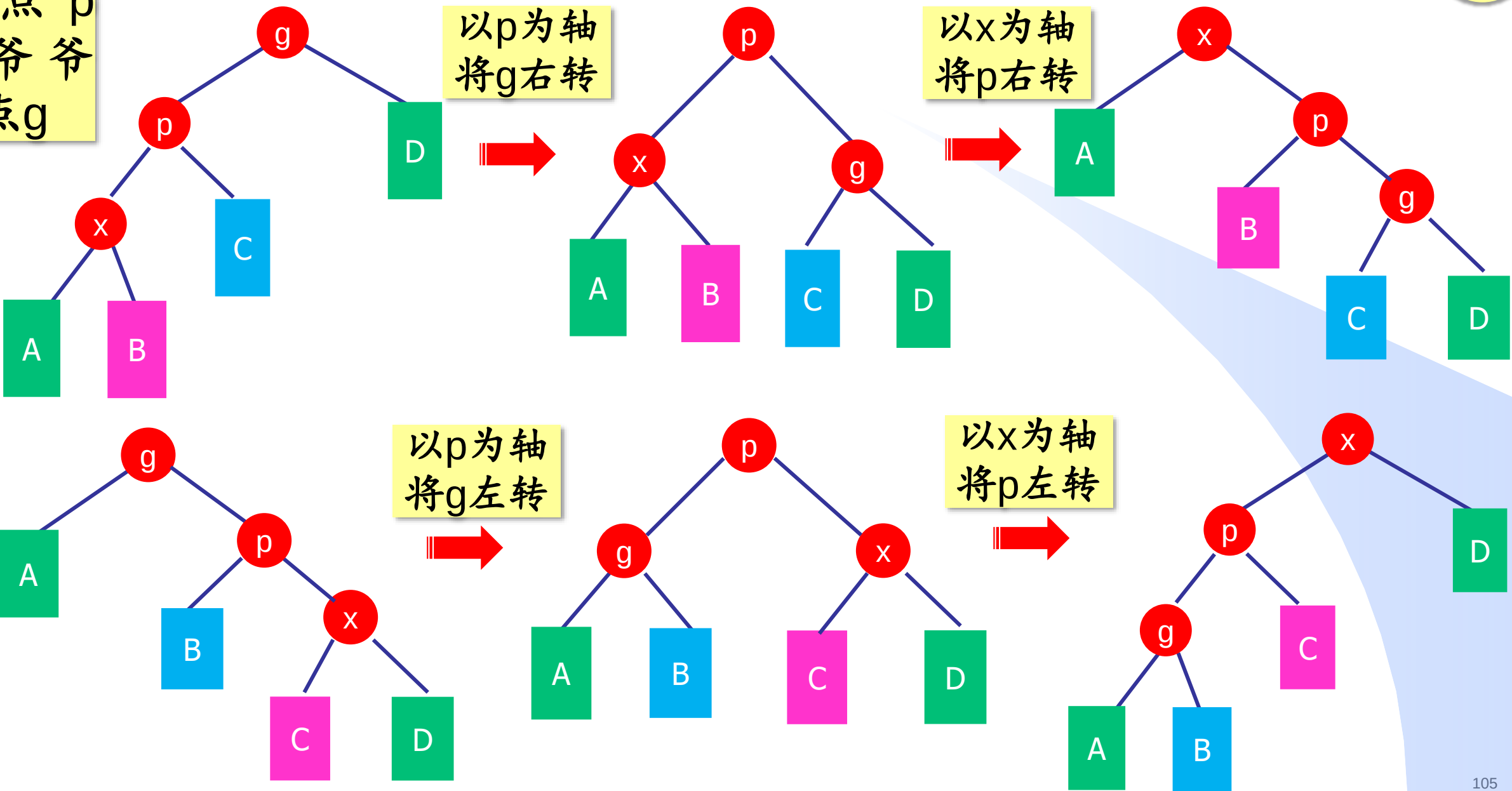


以x为轴  
将g左转



对x取父  
结点p  
和爷爷  
结点g

# Splay操作将结点x旋转至根——情况2：LL/RR型

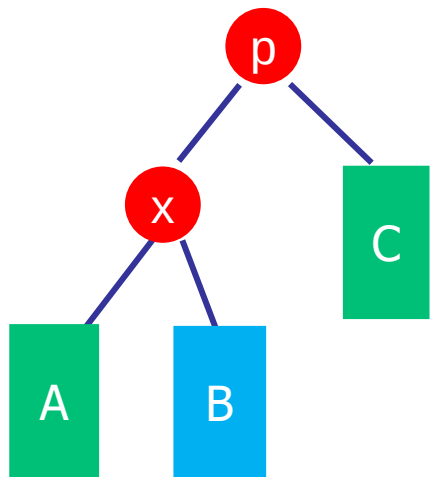




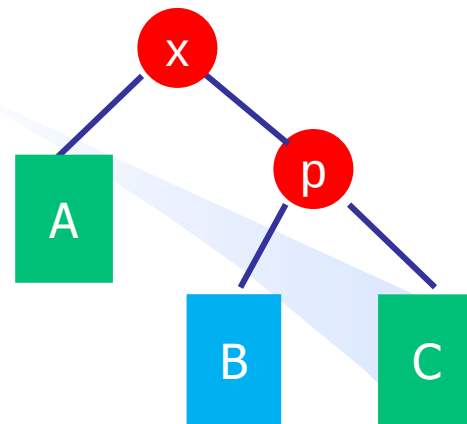
# Splay操作将结点x旋转至根——情况3：x有父无爷

C

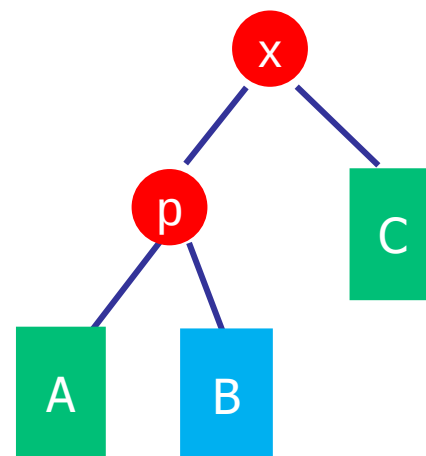
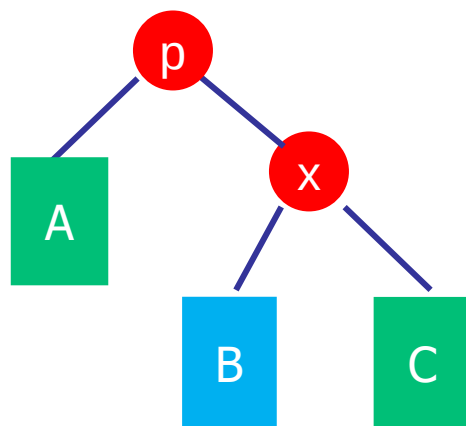
x的父结点  
p即为根



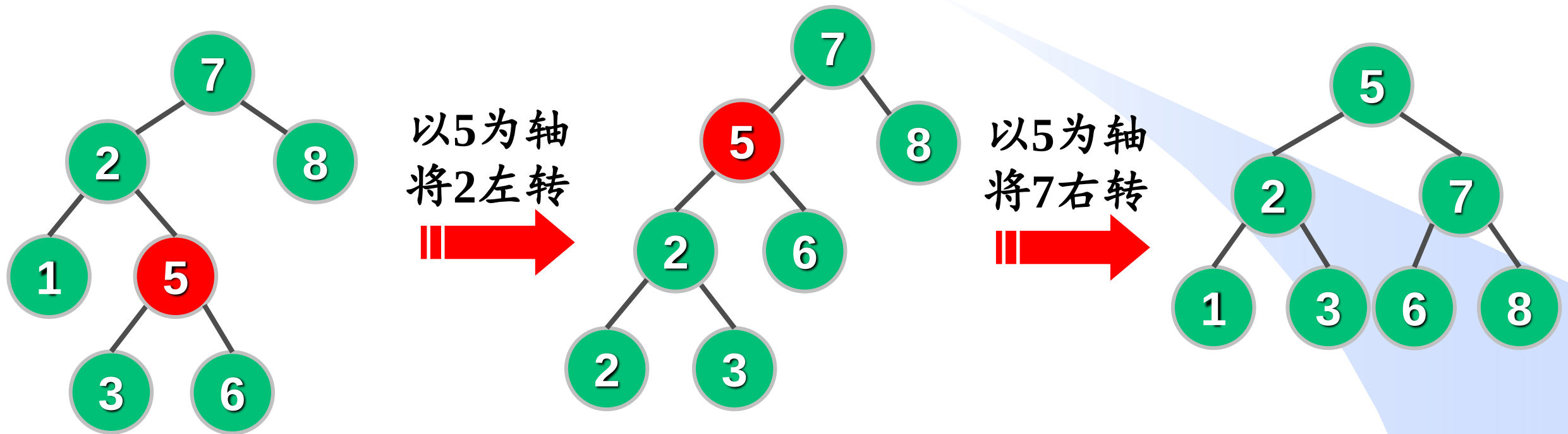
以x为轴  
将p右转



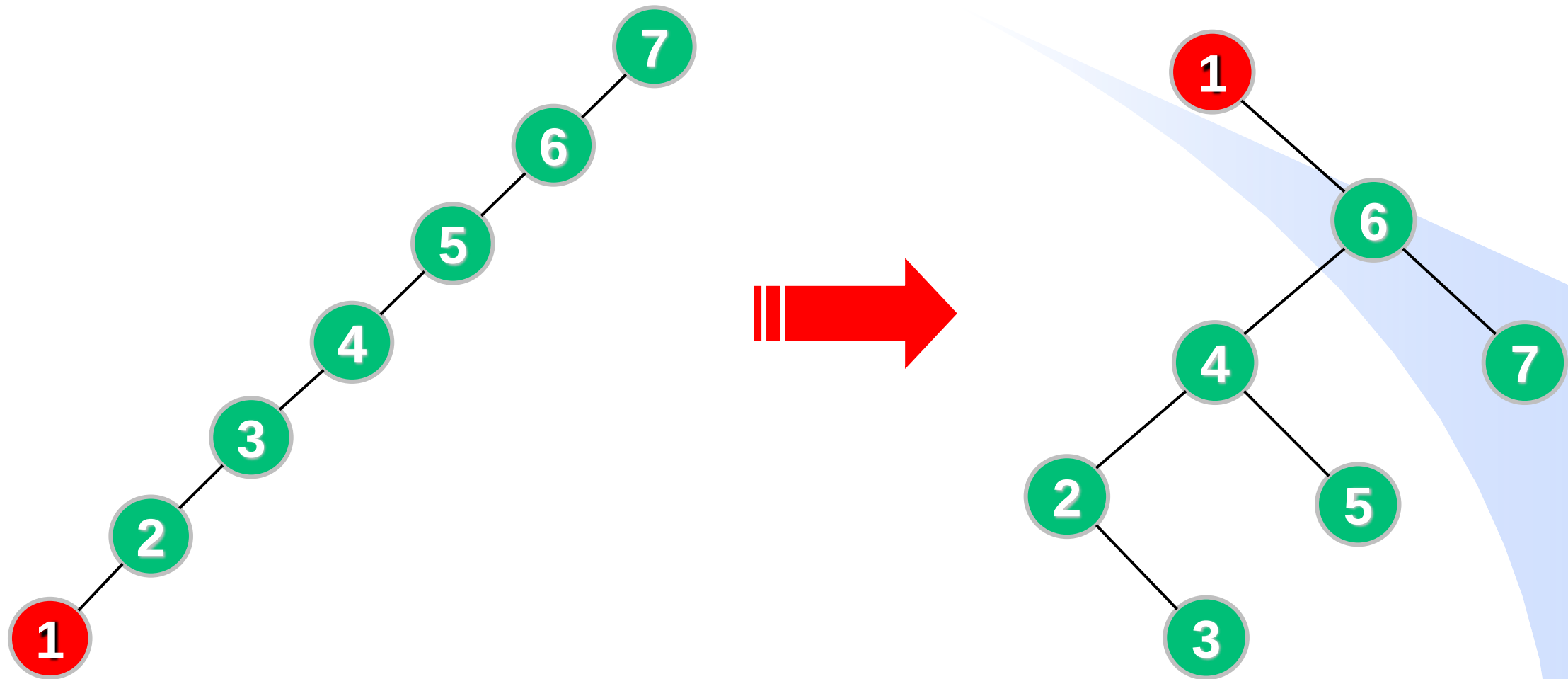
以x为轴  
将p左转



# 例子：访问5

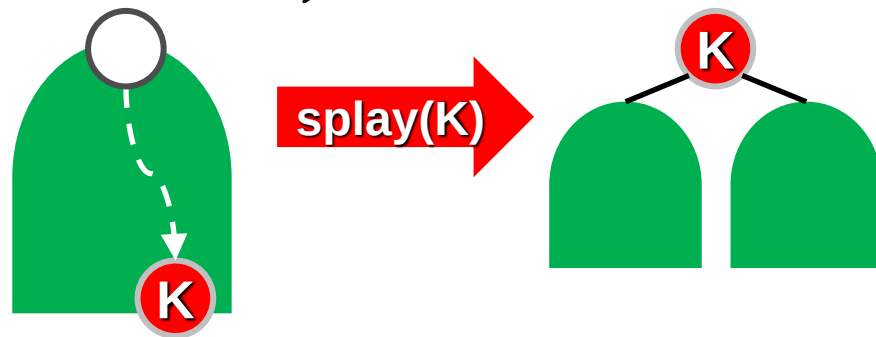


# 课下验证：访问1

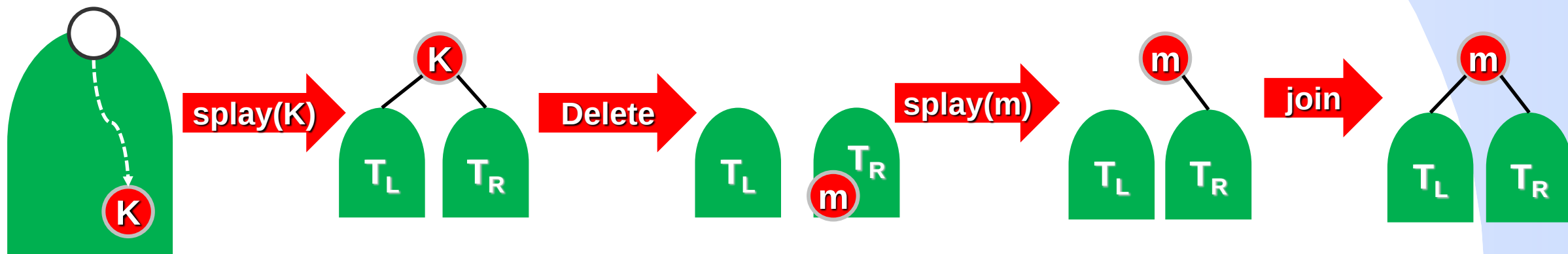


# 伸展树 (Splay树)

- **查找：** 执行BST查找算法，然后将找到的结点伸展至根。
- **插入：** 执行BST插入算法，然后将新插入的结点伸展至根。



- **删除：** 找到待删结点 $K$ ，将 $K$ 伸展至根，删去 $K$ ，在右子树 $T_R$ 中选取关键词最小的结点 $m$ ，将 $m$ 伸展至根，将 $T_L$ 作为 $m$ 的左子树。



# 自平衡二叉查找树 (*Self-Balanced Binary Search Tree*)

C

- 都是在平衡度（查找效率）与平衡维护成本（增删效率）之间做*trade-off*。
- 平衡度越好，树越矮，查找效率越高；增删操作效率越低，维持平衡所需的成本越高。
- 平衡度越差，树越高，查找效率越差；增删操作效率越高，维持平衡所需的成本越低。

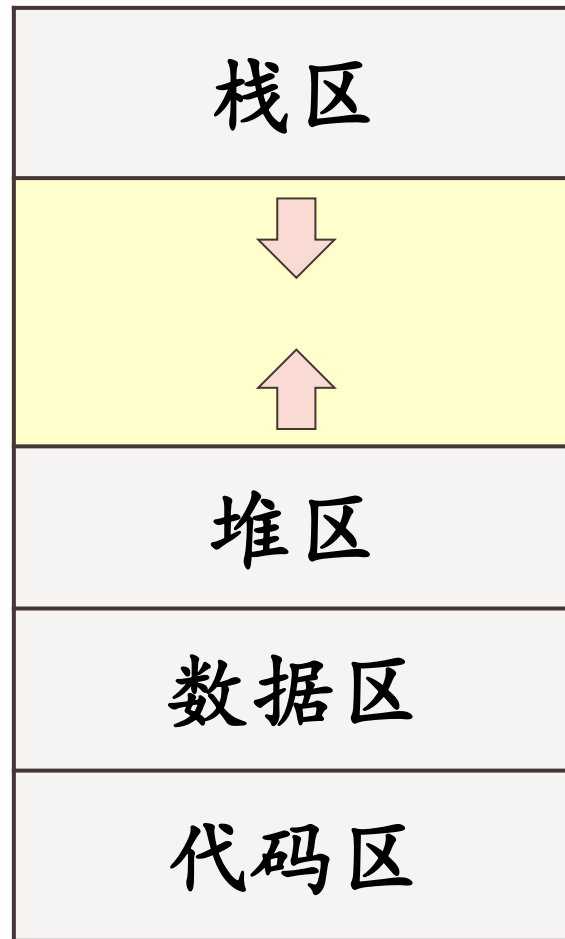


# 自平衡二叉查找树的应用——堆内存管理

进程的虚拟内存地址空间

- 代码区：存放程序的可执行代码
- 数据区：存放全局变量、常量等
- **堆区：存放动态分配的空间**
- 栈区：存放函数的局部变量、参数、返回地址等

高地址

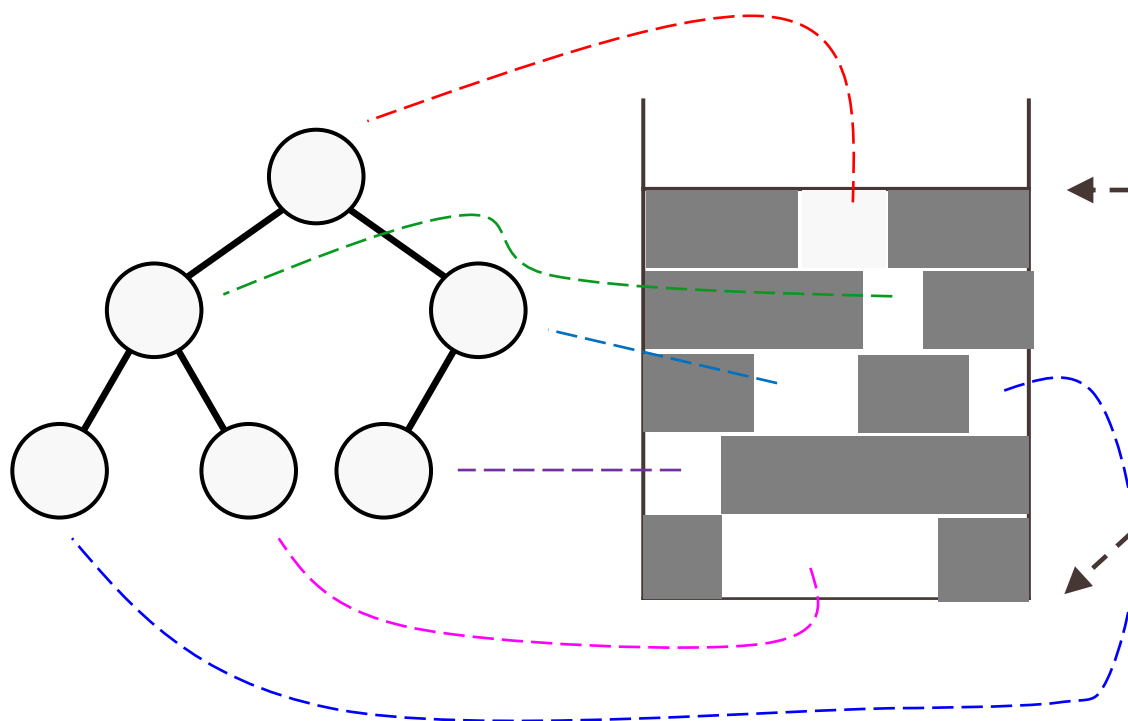


低地址

# 堆内存管理

如何管理  
空闲块？

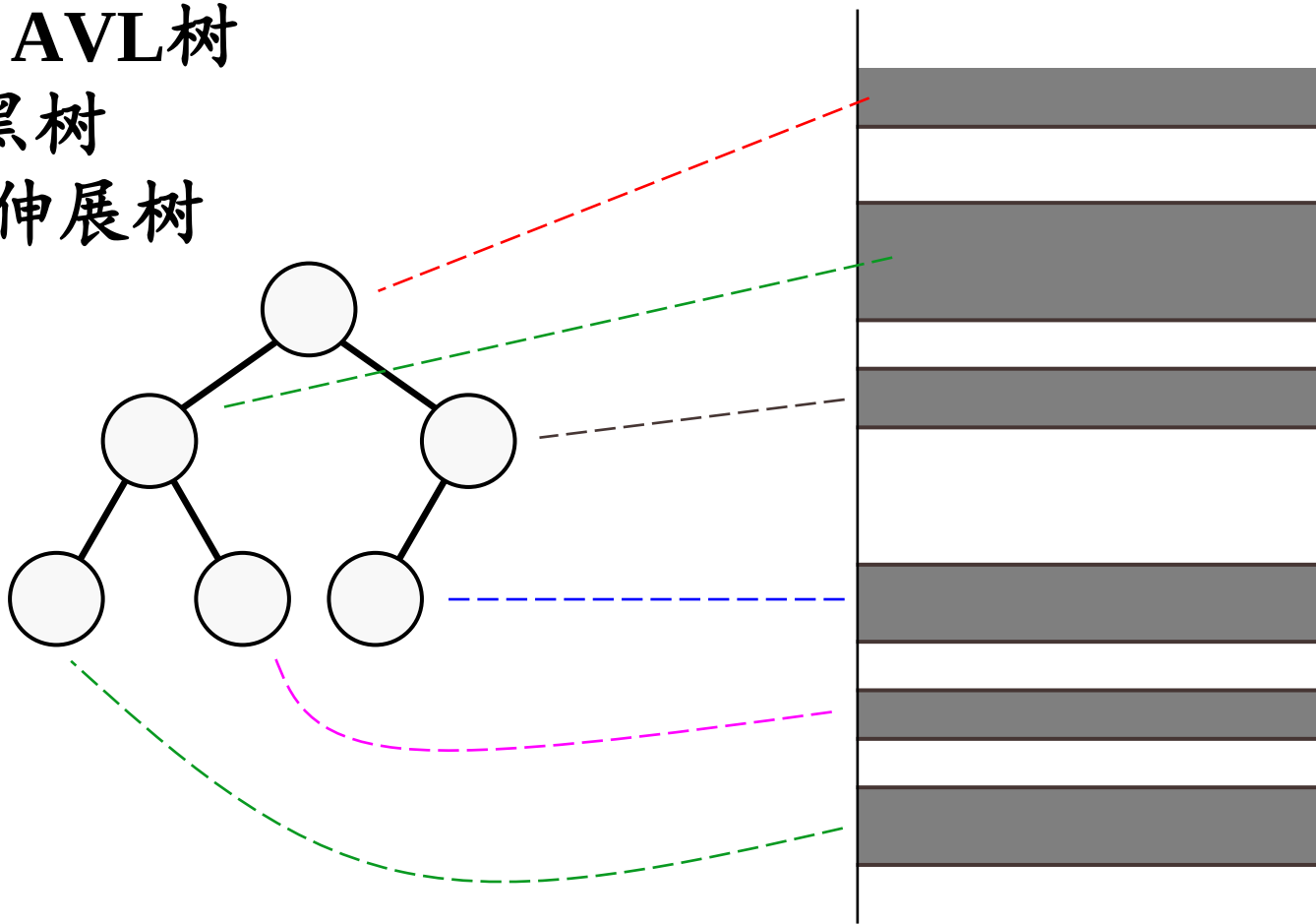
用平衡树组织空闲块  
结点：空闲块  
关键词：空闲块的大小



# 进程虚拟内存地址空间

操作系统中程序内存地址空间管理

- ✓ Windows : AVL树
- ✓ Linux: 红黑树
- ✓ FreeBSD: 伸展树

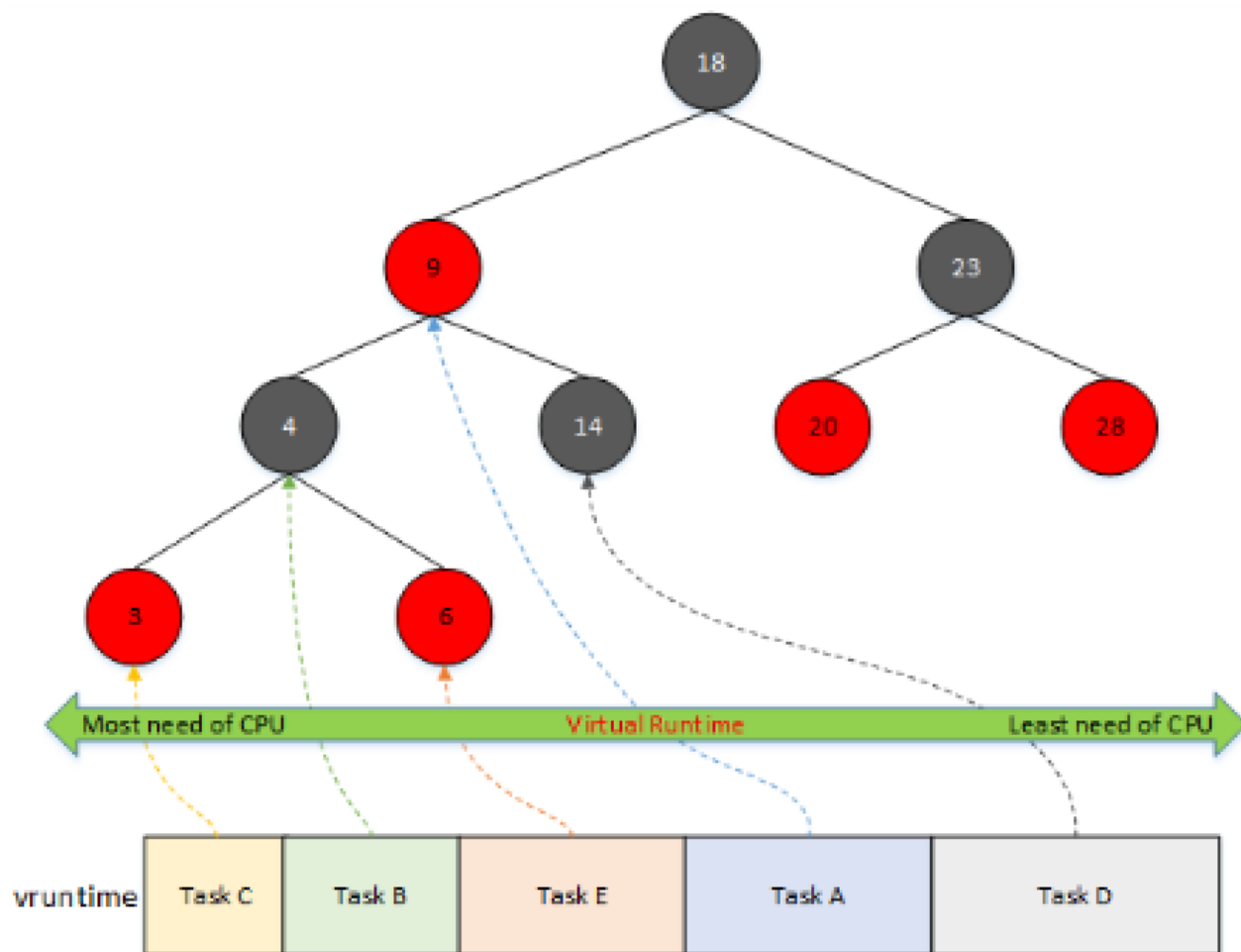




# 自平衡二叉查找树的应用场景

## Linux CFS 进程调度

✓ 红黑树



# 自平衡二叉查找树的应用场景

- C++ STL: map、set和multimap
- Java: TreeMap、TreeSet
- 均基于红黑树实现

